DevOps Question Bank Kubernetes

89 real interview questions - de-duplicated, topic-wise, Basic / Advanced / Scenario-Based
Every answer: plain-English idea · technical answer · example · what to say in the interview

By Priyanka Gupta — 25 September 2026

How to use this question bank

Each question has four parts:

In simple words (green box) - a plain-English analogy. Read this first; if you understand it, the technical part becomes easy to remember.

Detailed answer + Example - the real technical answer, with commands / YAML / code you can show.

What to say to the interviewer (blue box) - 2-3 lines to say out loud. Practise these aloud.

Questions that were asked more than once across the lists were merged into one (the other wordings are shown in grey as 'Also asked as'). Language-specific (Python/PHP) and non-DevOps questions were removed. Where a pasted 'model answer' was wrong, the corrected version is given.

Suggested order: all Basic questions first, then Scenario-Based (most interviews are troubleshooting-heavy), then Advanced. Use the PDF bookmarks panel to jump to any question.

Topic	Basic	Advanced	Scenario	Total
Kubernetes	31	30	28	89

Questions in this topic

Basic **7**

Q1. Explain Kubernetes: what is it, how does it work, and what does each control-plane component do? . 7

Q2. Which component watches your deployment.yaml and keeps the Pods running exactly as defined? . 8

Q3. What is etcd? Can Kubernetes function without etcd? 10

Q4. What are Kubernetes objects? What are the common manifest files/objects and what does each do? 10

Q5. How do you create a Kubernetes object, and what command do you use to get inside a Pod? . . . 12

Q6. What is a Deployment, and how is it different from a ReplicaSet? 12

Q7. What is the difference between a Deployment and a StatefulSet? 13

Q8. How do you create an Nginx Deployment and expose it through a Service? 14

Q9. Why are Services necessary, and how does a Service know which Pods to route traffic to? 15

Q10. What is the difference between ClusterIP, NodePort and LoadBalancer? How do you expose services in Kubernetes? 16

Q11. What is the difference between an Ingress and a LoadBalancer Service? 17

Q12. Why do we use namespaces in Kubernetes? 18

Q13. What are ConfigMaps and Secrets? 19

Q14. What is the rolling update strategy? 20

Q15. How do you auto-scale pods? Explain Horizontal Pod Autoscaling (HPA) and the other types of scaling. 20

Q16. What are liveness, readiness and startup probes? How are they different and how do you use them? 22

Q17. Explain the Kubernetes Pod lifecycle. 23

Q18. What are PV and PVC? Which comes first, and why? 24

Q19. What is RBAC in Kubernetes, what is it used for, and how do you configure it? 25

Q20. What does Kubernetes administration mean to you? 26

Q21. Rapid-fire: Kubernetes concept checks (common trick questions) 27

Q22. What is the NodePort port range? 29

Q23. What is an Ingress, and how do you expose applications on an on-premises Kubernetes cluster? Do LoadBalancer Services work on-prem? 29

Q24. Can we define multiple ports in a Kubernetes Deployment? How do you verify they are correctly defined and working? 30

Q25. The team asks you to pass an extra startup parameter to an app deployed from a Docker image. Where do you define the startup command in Kubernetes? 31

Q26. What is CoreDNS, and what role does it play in Kubernetes? How does DNS resolution work inside a pod? 32

Q27. What is Helm and why is it important? Explain the Helm chart folder structure and the commands you use to deploy. 33

Q28. What is KEDA, and how does it help with event-driven autoscaling? 35

Q29. How many nodes are there in your Kubernetes cluster? How do you decide? 35

Q30. How do you deploy an application in a Kubernetes (EKS) cluster? Explain the end-to-end flow. . . . 36

Q31. When deploying to Kubernetes, do you only update Docker images, or also replicas, storage and CPU allocation? 37

Advanced **38**

Q32. What are master (control-plane) and worker nodes, and how do you create and manage Kubernetes clusters (eksctl / Terraform)? 38

Q33. Explain Amazon EKS architecture. What subnets are used in EKS? 39

Q34. How does authentication work in EKS, and how do pods get AWS permissions and secrets securely?. .

Q35. How do Jenkins and a Kubernetes cluster communicate with each other? 41

Q36. You have Docker images for a frontend, a backend and a database. How do you deploy them on Kubernetes with YAML? 41

Q37. You have 6 Pods and 3 Nodes. How do you ensure exactly 2 Pods run on each Node? 44

Q38. How would you ensure even Pod distribution across Nodes and Availability Zones? 45

Q39. How do you optimize resource requests and limits in a production cluster? 46

Q40. How do you implement auto-scaling when traffic fluctuates heavily? How do you automate node scaling in EKS? 47

Q41. How would you monitor HPA scaling decisions in real time and detect if metrics-server is lagging? 48

Q42. What are the differences between GKE Ingress and the NGINX Ingress Controller? 49

Q43. How do you enforce rules in Kubernetes to control which pods can talk to each other? 50

Q44. How would you secure a Kubernetes cluster at the network, pod and identity/access levels? 51

Q45. How do you secure secrets in Kubernetes? 52

Q46. How do you manage secrets across multi-region Kubernetes clusters? 53

Q47. How do you handle database credential rotation in Kubernetes without downtime? 54

Q48. What is your strategy for backup and restore in a Kubernetes cluster? How do you handle disaster recovery for stateful apps on containers? 55

Q49. How do you migrate data from one MongoDB pod to another? How do you manage stateful workloads in Kubernetes? 56

Q50. Have you upgraded Kubernetes clusters? How do you upgrade a cluster (EKS) safely? 57

Q51. How do you perform zero-downtime deployments in Kubernetes? 57

Q52. How would you do blue-green (and canary) deployments in a Kubernetes / EKS cluster? 59

Q53. How would you set up an automated rollback strategy in Kubernetes for failed deployments? . . . 60

Q54. What are PodDisruptionBudgets and why do you need them? 61

Q55. What does the pod status ContainerStatusUnknown mean, and when does it occur? 62

Q56. How would you configure readiness/liveness probes to catch buffering or lag issues in stream-processing microservices before users notice? 63

Q57. How does data flow in a Kubernetes cluster? Explain the Kubernetes networking model. 64

Q58. What happens if a node with local storage gets scaled down by the autoscaler? 65

Q59. How do you enforce runtime security in Kubernetes? . . . 66

Q60. HPA vs VPA vs Karpenter: when would you NOT use each, and how would you simulate HPA behaviour in staging? . . . 67

Q61. How would you implement autoscaling for a multi-tenant workload, including a strategy for tenant isolation? . . . 69

Scenario-Based **70**

Q62. What common Kubernetes errors have you faced, and how did you fix them? . . . 70

Q63. Rapid-fire: What happens if...? . . . 72

Q64. A pod is stuck in CrashLoopBackOff even though the image is valid. How do you debug and fix it? 74

Q65. A containerized app works on a developer machine (or locally) but fails or crashes in Kubernetes / production. What is your debug checklist? . . . 75

Q66. A deployment rollout is stuck and new pods are not becoming ready. What steps do you follow? . 76

Q67. A kubectl rollout restart did not trigger new pods as expected. What could be wrong? . . . 77

Q68. A pod is failing because of a ConfigMap change. How would you roll back and prevent this next time?

Q69. The dev team pushed a bad Helm chart and the rollback hook fails. What is your live rollback strategy? . . . 79

Q70. Users cannot access an application exposed via Ingress (or the Ingress stopped working). How do you debug it? . . . 80

Q71. Kubernetes pods are healthy, but users are receiving errors (for example 503). How do you troubleshoot the complete request flow? . . . 81

Q72. A Service is running but application requests are timing out. How do you troubleshoot the networking?

Q73. Pod A cannot reach Pod B. How do you debug it? . . . 82

Q74. Service discovery fails across namespaces, but DNS looks green. What is your next step, and how do you confirm the root cause? . . . 83

Q75. Cluster-wide DNS stops resolving randomly across nodes. What is your step-by-step debug path? 84

Q76. One worker node becomes NotReady. What actions do you take? . . . 85

Q77. A node went down suddenly. What happens to the pods running on it? . . . 86

Q78. A node has reached (exhausted) its Pod capacity. What would you check first, and what would you do? . . . 87

Q79. Pods are getting evicted frequently due to resource pressure (for example in EKS). How do you fix and prevent it? . . . 88

Q80. You see high CPU usage in one pod (or one node group), but its logs look clean. What next? . . . 89

Q81. etcd becomes unavailable in a production cluster. What is your recovery strategy? . . . 90

Q82. HPA refuses to scale a critical workload even though Prometheus shows CPU above 90%. What is the root cause and fix? . . . 92

Q83. You are on call: pods are not pulling images (ImagePullBackOff), some are getting evicted, and users are seeing errors or 503s. How do you troubleshoot, and how do you prevent it next time? . . . 93

Q84. An app on AKS (or any managed Kubernetes) fails health checks randomly. How do you debug this end-to-end? . . . 94

Q85. In a canary deployment to production, half the traffic returns 502 while the rest succeeds. How do you troubleshoot? 95

Q86. Users report 10-second delays every 15 minutes in an app running on AKS (or any Kubernetes). No code changes happened. How do you begin RCA? 96

Q87. A kube-proxy update rolls out in the middle of a live event (for example a big match). What is your network rollback plan to avoid packet drops? 97

Q88. You need to containerize a legacy monolith and deploy it to EKS with minimal downtime. What is your approach? 98

Q89. Have you faced production issues in Kubernetes? Tell me about one and how you troubleshot it. . 99

Kubernetes

Basic

Q1. Explain Kubernetes: what is it, how does it work, and what does each control-plane component do?

Also asked as: What are the core components of Kubernetes, and which matter most? Explain Kubernetes architecture. How does Kubernetes work? / How do worker nodes communicate with the control plane?

> **In simple words:** Kubernetes is the manager of a big restaurant kitchen. You tell the manager "I always want 3 cooks making pizza", and the manager keeps checking and hires or replaces cooks so there are always exactly 3.

Detailed answer

Kubernetes (K8s) is a container orchestrator. You describe the **desired state** in YAML (for example "3 replicas of my app, image v2"), and Kubernetes runs **control loops** that keep comparing desired state with actual state and fixing the difference.

A cluster has two parts:

- **Control plane (the brain)**
- **Worker nodes (the muscle)** that actually run your Pods

Component	Where	Job in plain words
kube-apiserver	Control plane	The front door. Every kubectl command, controller and kubelet talks only to it. Validates and stores objects.
etcd	Control plane	The cluster's database (key-value). Stores all desired and current state.
kube-scheduler	Control plane	Picks which node a new Pod should run on (based on CPU/memory requests, affinity, taints).
kube-controller-manager	Control plane	Runs controllers (Deployment, ReplicaSet, Node, Job...) that fix drift between desired and actual state.
cloud-controller-manager	Control plane	Talks to AWS/GCP/Azure to create load balancers, attach volumes, etc.
kubelet	Every node	Agent that starts/stops containers for Pods assigned to its node and reports health.
kube-proxy	Every node	Programs iptables/IPVS rules so Service IPs route to Pod IPs.
Container runtime	Every node	containerd/CRI-O actually runs the containers.

Which matter most? The **API server** (nothing works without it) and **etcd** (lose it without a backup and you lose the cluster's state) are the most critical; on the node side, **kubelet** (no kubelet = node NotReady) and **CoreDNS** (no DNS = services cannot find each other).

Flow of `kubectl apply -f deploy.yaml`: kubectl -> API server -> saved in etcd -> Deployment controller creates a ReplicaSet -> ReplicaSet controller creates Pods -> scheduler assigns each Pod to a node -> kubelet on that node pulls the image and starts the container -> kube-proxy makes it reachable via the Service.

Example:

```
kubectl get componentstatuses        # older clusters
kubectl get pods -n kube-system      # coredns, kube-proxy, aws-node (CNI) ...
kubectl get nodes -o wide
```

How worker nodes talk to the control plane:

- Everything goes through the **API server**. Nodes never talk to etcd, the scheduler or the controller manager directly.
- **Node -> control plane:** the kubelet and kube-proxy open outbound TLS connections to the API server (port 443 or 6443) using the node's credentials (client certificate or, on EKS, an IAM-based token). The kubelet **watches** for pods assigned to its node and reports node and pod status back. kube-proxy watches Services and EndpointSlices.
- **Control plane -> node:** the API server calls the kubelet on **port 10250** for `kubectl logs`, `exec`, `port-forward` and metrics. In EKS this path uses cross-account ENIs placed in your subnets; some setups use the Konnectivity proxy instead.
- The node's **lease** object in `kube-node-lease` is its heartbeat. If renewals stop for about 40 seconds, the node controller marks the node NotReady.

```
kubectl get --raw /api/v1/nodes/ip-10-0-1-23/proxy/healthz   # API server -> kubelet 10250
kubectl get lease -n kube-node-lease                          # node heartbeats
```

> ***What to say to the interviewer:*** *Kubernetes is a declarative container orchestrator: I declare desired state and control loops keep reality matching it. The control plane has the API server as the single entry point, etcd as the state store, the scheduler for placement and the controller manager for reconciliation; each worker node runs kubelet, kube-proxy and a container runtime. On EKS, AWS manages the control plane and we manage the node groups.*

Q2. Which component watches your deployment.yaml and keeps the Pods running exactly as defined?

Also asked as: Walk me through what the controller manager does during a Deployment - the reconciliation logic, not rollout status.

> **In simple words:** You hand the order to the front desk (API server), it is written in the notebook (etcd), and a supervisor (controller manager) keeps walking around making sure reality matches the notebook, all day, forever.

Detailed answer

No single component "watches the file". The YAML file is only sent once (`kubectl apply`). After that:

1. **kube-apiserver** validates it and stores the Deployment object in **etcd**.
2. **kube-controller-manager** runs the **Deployment controller**, which watches Deployment objects (via the API server) and creates/updates a **ReplicaSet**.

3. The **ReplicaSet controller** watches ReplicaSets and Pods; if there are fewer Pods than `replicas` (a Pod crashed, a node died), it creates new Pod objects. If there are too many, it deletes some.
4. **kube-scheduler** assigns each new Pod to a node.
5. **kubelet** on that node starts the containers and restarts them if they die (per `restartPolicy` and liveness probes).

This "observe -> compare -> act" cycle is called the **reconciliation loop**. It is why deleting a Pod manually just makes a new one appear.

How the reconciliation actually works inside the controller manager:

- Controllers do not poll; they use **informers** that **watch** the API server and keep a local cache of objects. Every add/update/delete event puts the object's key (`namespace/name`) onto a **work queue**.
- A worker takes a key, reads the **desired state** (Deployment spec) and the **actual state** (ReplicaSets and Pods from the cache) and computes the difference. The logic is **level-based**: it looks at the current state, not at the history of events, so a missed event is fixed on the next sync.
- **Deployment controller:** hashes the Pod template into a `pod-template-hash` label. If a ReplicaSet with that hash exists, it just scales it; if the template changed, it creates a new ReplicaSet with the new hash, then scales new up and old down within `maxSurge`/`maxUnavailable`. It writes progress into Deployment `status` (replicas, updatedReplicas, availableReplicas, conditions `Progressing`/`Available`).
- **ReplicaSet controller:** counts Pods matching its selector (including the hash label), creates or deletes Pods to match `replicas`, and sets **ownerReferences** so the Pods belong to it (garbage collector deletes them when the owner is deleted).
- **Scheduler** binds Pending Pods to nodes; **kubelet** watches Pods bound to its node, runs containers and reports status back. Those status updates generate new events, and the loop continues.
- If something fails (API error, conflict), the key is **re-queued with back-off**, so the system keeps converging.

Example:

```
kubectl delete pod web-7c9b-abcde
kubectl get pods -w          # a new web-7c9b-xxxxx appears within seconds
kubectl get events --sort-by=.lastTimestamp | tail
# ... ReplicaSet web-7c9b  SuccessfulCreate  Created pod: web-7c9b-xxxxx
```

> ***What to say to the interviewer:*** *The API server stores the Deployment in etcd, and the controllers inside kube-controller-manager, specifically the Deployment and ReplicaSet controllers, keep reconciling actual pods against that desired state using informers, work queues and a level-based diff, with the pod-template-hash deciding whether to scale an existing ReplicaSet or create a new one. The scheduler places new pods and kubelet keeps containers running on the node, so a deleted or crashed pod is replaced automatically.*

Q3. What is etcd? Can Kubernetes function without etcd?

> **In simple words:** etcd is the kitchen manager's notebook. If the notebook is lost, the cooks already cooking keep cooking, but nobody can take new orders or remember what the kitchen is supposed to look like.

Detailed answer

etcd is a distributed, consistent key-value store (uses the Raft consensus algorithm). It holds every object: Deployments, Pods, Secrets, ConfigMaps, node status.

No, Kubernetes cannot function properly without etcd. The API server is stateless and reads/writes everything from etcd. If etcd is down:

- Already running Pods usually **keep running** (kubelet and containers do not need etcd moment to moment).
- But you **cannot** create/update/delete anything, scheduling stops, controllers cannot heal failed Pods, and `kubectl` returns errors.

Production etcd runs as an **odd number of members (3 or 5)** so it keeps quorum when one member fails (3 members tolerate 1 failure, 5 tolerate 2). On EKS/GKE/AKS the provider runs etcd for you.

Example:

```
# self-managed cluster: check etcd health
ETCDCTL_API=3 etcdctl --endpoints=https://127.0.0.1:2379 \
  --cacert=/etc/kubernetes/pki/etcd/ca.crt \
  --cert=/etc/kubernetes/pki/etcd/server.crt \
  --key=/etc/kubernetes/pki/etcd/server.key endpoint health
```

> ***What to say to the interviewer:*** *etcd is the single source of truth for cluster state, so Kubernetes cannot really work without it. Running workloads keep running for a while, but the API becomes unusable and nothing can be scheduled or healed. That is why etcd runs as 3 or 5 members for quorum and is backed up with regular snapshots.*

Q4. What are Kubernetes objects? What are the common manifest files/objects and what does each do?

Also asked as: How do you create a Kubernetes "class" (object/manifest) in your project? Which objects have you used in your project?

> **In simple words:** Objects are the "order forms" you give Kubernetes. Each form type asks for something different: a running app, a network address, a config file, a disk.

Detailed answer

A Kubernetes object is a persistent record of intent stored in etcd. Every object has `apiVersion`, `kind`, `metadata` (name, namespace, labels) and usually `spec` (what you want) and `status` (what exists).

Object	What it does (one line)
Pod	Smallest unit: one or more containers sharing network (IP) and volumes

Object	What it does (one line)
Deployment	Runs stateless pods via ReplicaSets; rolling updates and rollback
StatefulSet	Pods with stable names, stable DNS and their own PVCs (databases, Kafka)
DaemonSet	One pod on every (or selected) node: log agents, node-exporter, CNI
Job	Runs pods until a task completes successfully (migration, batch)
CronJob	Creates Jobs on a schedule (backups, reports)
Service	Stable IP/DNS name + load balancing to pods selected by labels
Ingress	HTTP/HTTPS host and path routing from outside to Services (needs a controller)
ConfigMap	Non-sensitive configuration as key-values or files
Secret	Sensitive data; **base64-encoded, NOT encrypted by default** - enable KMS encryption at rest and RBAC
PersistentVolumeClaim	A pod's request for storage; bound to a PersistentVolume
HorizontalPodAutoscaler	Scales replica count on CPU/memory/custom metrics
ServiceAccount	Identity for pods (used with RBAC and IRSA)
Role	Namespaced set of permissions (verbs on resources)
ClusterRole	Cluster-wide set of permissions
RoleBinding	Grants a Role/ClusterRole to users, groups or ServiceAccounts in a namespace
NetworkPolicy	Firewall rules for which pods may talk to which
Namespace	Logical partition for teams/apps (RBAC, quotas)
PodDisruptionBudget	Limits how many pods can be voluntarily evicted at once

Project example: each microservice had a Deployment + Service + ConfigMap + Secret + HPA; Kafka/Redis-like stateful pieces used StatefulSets with PVCs; the log/metrics agents (Promtail, node-exporter) ran as DaemonSets so one copy runs on every node; nightly jobs used CronJobs.

Example:

```
kubectl api-resources | head
kubectl get deploy,svc,cm,secret,hpa -n my-app
```

> ***What to say to the interviewer:*** *Objects are the declarative records Kubernetes stores in etcd, each with a spec for desired state and a status for actual state; the everyday set is Pod, Deployment, StatefulSet, DaemonSet, Job, Service, Ingress, ConfigMap, Secret, PVC, HPA and the RBAC objects. In my projects every service had a Deployment, Service, ConfigMap, Secret and HPA, monitoring agents ran as DaemonSets, and stateful components used StatefulSets with PVCs.*

Q5. How do you create a Kubernetes object, and what command do you use to get inside a Pod?

Also asked as: How do you write YAML manifests? What is the command to enter a pod?

> **In simple words:** You can either give Kubernetes spoken instructions one at a time (imperative) or hand it a written plan it keeps following (declarative). Getting into a Pod is like opening a remote terminal into that little computer.

Detailed answer

Creating objects - two styles:

Style	Commands	When
Imperative	`kubectl create deployment`, `kubectl run`, `kubectl expose`, `kubectl scale`	Quick tests, learning, generating YAML
Declarative	`kubectl apply -f file.yaml` (or Helm / Argo CD)	Production: files live in Git, repeatable, reviewable

Every manifest needs four top-level fields: `apiVersion`, `kind`, `metadata` (name, namespace, labels) and `spec`. Use `kubectl explain deployment.spec.template.spec.containers` to look up fields instead of memorizing them, and validate with `kubectl apply --dry-run=server -f`.

`kubectl create -f` fails if the object already exists; `kubectl apply -f` creates or updates it. A handy trick is to generate YAML imperatively and then manage it declaratively.

Getting into a Pod: `kubectl exec -it <pod> -- /bin/sh` (or `bash` if the image has it). Use `-c <container>` for multi-container Pods. Distroless images have no shell, so use `kubectl debug` to attach a temporary debug container.

Example:

```
kubectl create deployment web --image=nginx --dry-run=client -o yaml > web.yaml
kubectl apply -f web.yaml
kubectl apply -f k8s/                 # whole folder
kubectl diff -f web.yaml              # preview changes

kubectl exec -it web-7c9b-abcde -- /bin/sh
kubectl exec -it web-7c9b-abcde -c sidecar -- sh
kubectl exec web-7c9b-abcde -- env | grep DB_
kubectl debug -it web-7c9b-abcde --image=busybox:1.36 --target=web
```

> ***What to say to the interviewer:*** *For production I create objects declaratively with kubectl apply -f from Git, usually through Helm or Argo CD, and use imperative commands only for quick tests or to generate YAML with dry-run. To get inside a pod I use kubectl exec -it pod -- sh, adding -c for a specific container, or kubectl debug when the image has no shell.*

Q6. What is a Deployment, and how is it different from a ReplicaSet?

Also asked as: What is Kubernetes Deployment and ReplicaSet?

> **In simple words:** A ReplicaSet is a supervisor who only counts heads: "there must be 3 workers". A Deployment is the HR manager above him who also handles upgrades: it brings in new workers (v2)

and retires old ones (v1) gradually.

Detailed answer

- **ReplicaSet:** makes sure N identical Pods (matching a label selector) are always running. It knows nothing about versions or upgrades.
- **Deployment:** manages ReplicaSets. When you change the Pod template (new image), it creates a **new ReplicaSet** and scales it up while scaling the old one down (rolling update). It keeps old ReplicaSets for **rollback** (`revisionHistoryLimit`).

Feature	ReplicaSet	Deployment
Keeps N pods running	Yes	Yes (via ReplicaSet)
Rolling update	No	Yes
Rollback / history	No	Yes
Pause / resume rollout	No	Yes

You almost never create ReplicaSets directly; you create Deployments.

Example:

```
kubectl create deployment web --image=nginx:1.25 --replicas=3
kubectl set image deployment/web nginx=nginx:1.27
kubectl get rs           # web-6d4f... (old, 0 pods)  web-7c9b... (new, 3 pods)
kubectl rollout history deployment/web
kubectl rollout undo deployment/web
```

What to say to the interviewer: *A ReplicaSet only guarantees a number of identical pods. A Deployment sits on top, creates a new ReplicaSet for every template change, rolls pods over gradually and keeps revision history so I can roll back with kubectl rollout undo. In practice we always use Deployments.*

Q7. What is the difference between a Deployment and a StatefulSet?

In simple words: Deployment Pods are like identical taxis - any taxi will do and they are interchangeable. StatefulSet Pods are like named employees with their own desk and locker - "mongo-0" always comes back as mongo-0 with the same locker (disk).

Detailed answer

Feature	Deployment	StatefulSet
Pod names	Random (`web-7c9b-x2k4`)	Stable, ordered (`mongo-0`, `mongo-1`)
Network identity	Shared Service IP	Stable DNS per pod via headless Service (`mongo-0.mongo`)
Storage	Usually none or shared PVC	`volumeClaimTemplates`: each pod gets its **own** PVC that follows it
Start/stop order	Parallel, any order	Ordered (0, 1, 2 up; reverse down) by default
Rolling update	Any pod, surge allowed	One by one, highest ordinal first; supports `partition`
Use for	Stateless apps: APIs, web, workers	Databases, Kafka, ZooKeeper, Redis, Elasticsearch

If a StatefulSet pod is deleted, it comes back with the **same name and the same PVC**, so its data survives. Deleting the StatefulSet does **not** delete the PVCs by default (protects data).

Example:

```
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: mongo}
spec:
  serviceName: mongo            # headless Service (clusterIP: None)
  replicas: 3
  selector: {matchLabels: {app: mongo}}
  template:
    metadata: {labels: {app: mongo}}
    spec:
      containers:
      - name: mongo
        image: mongo:7
        volumeMounts: [{name: data, mountPath: /data/db}]
  volumeClaimTemplates:
  - metadata: {name: data}
    spec:
      accessModes: [ReadWriteOnce]
      storageClassName: gp3
      resources: {requests: {storage: 20Gi}}
```

> ***What to say to the interviewer:*** *Deployments are for stateless, interchangeable pods, while StatefulSets give each pod a stable name, stable DNS through a headless Service and its own persistent volume, with ordered startup and updates. I use StatefulSets for databases, Kafka or Redis, and Deployments for APIs and web services.*

Q8. How do you create an Nginx Deployment and expose it through a Service?

> **In simple words:** The Deployment starts the nginx "shops"; the Service puts up one permanent signboard and phone number that forwards customers to whichever shop is open.

Detailed answer

Example:

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
      - name: nginx
        image: nginx:1.27
        ports:
        - containerPort: 80
        resources:
          requests: {cpu: 100m, memory: 128Mi}
          limits:   {memory: 256Mi}
---
apiVersion: v1
kind: Service
metadata:
  name: nginx-svc
spec:
  type: ClusterIP          # NodePort or LoadBalancer to expose outside
  selector:
    app: nginx             # must match pod labels
  ports:
  - port: 80
    targetPort: 80
```

```
kubectl apply -f nginx.yaml
kubectl get pods -l app=nginx
kubectl get endpoints nginx-svc      # should list 3 pod IPs
kubectl port-forward svc/nginx-svc 8080:80   # test: curl localhost:8080

# imperative quick way
kubectl create deployment nginx --image=nginx --replicas=3
kubectl expose deployment nginx --port=80 --type=LoadBalancer
```

> ***What to say to the interviewer:*** *I write a Deployment with replicas, a label like app=nginx and containerPort 80, then a Service whose selector matches that label. I verify with kubectl get endpoints to confirm pod IPs are attached, and choose ClusterIP for internal, LoadBalancer or Ingress for external traffic.*

Q9. Why are Services necessary, and how does a Service know which Pods to route traffic to?

> **In simple words:** Pods are like food trucks that keep moving and changing their phone numbers. A Service is the fixed call-center number that always knows where the trucks currently are.

Detailed answer

Why Services: Pods are temporary. When a Pod restarts or scales, it gets a **new IP**. Clients cannot hard-code Pod IPs. A Service gives:

- A **stable virtual IP (ClusterIP)** and **DNS name** (`my-svc.my-ns.svc.cluster.local`)
- **Load balancing** across all healthy Pods
- Service discovery for other apps

How it finds Pods: through **labels and selectors**.

1. The Service has `selector: app: web`.
2. The EndpointSlice controller continuously finds Pods with label `app=web` that are **Ready** (readiness probe passing).
3. It writes their IPs into an **EndpointSlice**.
4. kube-proxy on every node programs iptables/IPVS rules: traffic to the ClusterIP -> one of those Pod IPs.

If a Pod fails its readiness probe, it is removed from endpoints automatically, so no traffic reaches it.

Example:

```
kubectl get svc web -o jsonpath='{.spec.selector}'   # {"app":"web"}
kubectl get pods -l app=web -o wide
kubectl get endpointslices -l kubernetes.io/service-name=web
# empty endpoints = selector/label mismatch or pods not Ready
```

> ***What to say to the interviewer:*** *Pods are ephemeral and their IPs change, so a Service provides a stable IP, DNS name and load balancing. It selects pods by label selector; the endpoint controller keeps only Ready matching pods in the EndpointSlice and kube-proxy routes the ClusterIP to them. An empty endpoints list is my first check when a Service does not work.*

Q10. What is the difference between ClusterIP, NodePort and LoadBalancer? How do you expose services in Kubernetes?

Also asked as: How do you expose an application externally? What are the Service types and how does Ingress work?

> **In simple words:** ClusterIP is an internal office extension number, NodePort is a side door on every building with a fixed door number, and LoadBalancer is a proper public reception desk with its own address.

Detailed answer

Type	Reachable from	How	Typical use
ClusterIP (default)	Inside cluster only	Virtual IP + DNS	Service-to-service calls, DBs
NodePort	Outside via `NodeIP:30000-32767`	Opens same port on every node	Testing, on-prem, behind own LB
LoadBalancer	Internet / VPC	Cloud creates an LB (AWS NLB/CLB) pointing to nodes/pods	Expose one service directly
ExternalName	Inside	DNS CNAME to external host	Point to external DB/API
Headless (`clusterIP: None`)	Inside	DNS returns pod IPs directly	StatefulSets (Kafka, MongoDB)

Each type builds on the previous: LoadBalancer creates a NodePort, which creates a ClusterIP.

Ingress is the usual production way: one load balancer + an Ingress controller (NGINX, AWS Load Balancer Controller -> ALB) routes by host/path to many ClusterIP services, with TLS. Cheaper than one LoadBalancer per service.

Example:

```
kubectl expose deploy web --port=80 --type=NodePort
kubectl get svc web        # 80:31234/TCP -> curl http://<node-ip>:31234

apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: web
  annotations:
    alb.ingress.kubernetes.io/scheme: internet-facing
spec:
  ingressClassName: alb
  rules:
  - host: app.example.com
    http:
      paths:
      - path: /
        pathType: Prefix
        backend: {service: {name: web, port: {number: 80}}}
```

> ***What to say to the interviewer:*** *ClusterIP is internal-only, NodePort opens a static port on every node, and LoadBalancer asks the cloud to provision an external load balancer. In production I keep services as ClusterIP and expose them through an Ingress controller with host and path routing and TLS, so many services share one load balancer.*

Q11. What is the difference between an Ingress and a LoadBalancer Service?

> **In simple words:** A LoadBalancer Service is giving every shop in a mall its own street entrance and security guard (expensive). An Ingress is one main mall entrance with a receptionist who reads your ticket and sends you to the right shop.

Detailed answer

	LoadBalancer Service	Ingress
OSI layer	Layer 4 (TCP/UDP)	Layer 7 (HTTP/HTTPS)
Routing	Just forwards a port to pods	By host (`api.x.com`) and path (`/orders`)
Load balancers created	One per Service	One shared LB for many Services
TLS	Usually terminated at app or NLB	Central TLS termination (cert-manager, ACM)
Extras	None	Redirects, rewrites, auth, rate limits, WAF (ALB)
Cost	Grows with number of Services	One LB cost
Needs a controller	No (cloud controller does it)	Yes (ingress-nginx, AWS Load Balancer Controller)

Use a **LoadBalancer Service** for non-HTTP traffic (MQTT, raw TCP, gRPC on NLB, databases for internal clients) or when you need an NLB with static IPs. Use **Ingress** for HTTP web apps and

APIs. The newer **Gateway API** is the successor to Ingress with richer routing.

Example:

```
# L4: one NLB just for this service
apiVersion: v1
kind: Service
metadata:
  name: mqtt
  annotations:
    service.beta.kubernetes.io/aws-load-balancer-type: external
    service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
spec:
  type: LoadBalancer
  selector: {app: emqx}
  ports: [{port: 1883, targetPort: 1883}]

kubectl get ingress -A     # many hosts/paths, same ADDRESS (one ALB)
```

> ***What to say to the interviewer:*** *A LoadBalancer Service is layer 4 and creates one cloud load balancer per service, while Ingress is layer 7 and lets one load balancer route many hostnames and paths to different services with central TLS. I use Ingress for HTTP APIs to save cost and get path routing, and LoadBalancer or NLB for TCP protocols like MQTT.*

Q12. Why do we use namespaces in Kubernetes?

> **In simple words:** Namespaces are separate floors in the same office building. Each team gets its own floor with its own access cards and its own electricity budget, but they share the building.

Detailed answer

A namespace is a **virtual partition** inside one cluster. Uses:

- **Organization / isolation:** separate teams, apps or environments (`hes`, `mdms`, `monitoring`, `dev`, `staging`). Same object names can exist in different namespaces.
- **Access control:** RBAC Roles/RoleBindings are namespaced, so team A can deploy only in its namespace.
- **Resource limits:** `ResourceQuota` caps total CPU/memory/pod count per namespace; `LimitRange` sets default requests/limits for pods that forget them.
- **Network policy:** NetworkPolicies are applied per namespace, e.g. deny traffic from other namespaces.
- **Cleanup:** `kubectl delete ns feature-x` removes everything in it.

Namespaces are **not** a hard security boundary (same nodes, same kernel); for strong isolation use separate node groups or clusters. Some objects are cluster-wide: Nodes, PersistentVolumes, StorageClasses, ClusterRoles, Namespaces themselves.

Cross-namespace calls use the full name: `svc-name.other-ns.svc.cluster.local`.

Example:

```
kubectl create ns team-a
kubectl apply -n team-a -f - <<EOF
apiVersion: v1
kind: ResourceQuota
metadata: {name: quota}
spec:
  hard: {requests.cpu: "4", requests.memory: 8Gi, pods: "30"}
EOF
kubectl get pods -n team-a
kubectl config set-context --current --namespace=team-a
```

> ***What to say to the interviewer:*** *Namespaces logically divide one cluster by team, app or environment, and they are the unit for RBAC, ResourceQuota, LimitRange and NetworkPolicy. They are not a hard security boundary, so for strict isolation I combine them with separate node groups or separate clusters.*

Q13. What are ConfigMaps and Secrets?

> **In simple words:** A ConfigMap is a notice board with settings anyone can read. A Secret is a locked drawer for passwords. Both let you change settings without rebuilding the app image.

Detailed answer

- **ConfigMap:** non-sensitive config (URLs, feature flags, config files).
- **Secret:** sensitive data (passwords, tokens, TLS certs). Stored **base64-encoded, which is NOT encryption**. You must enable encryption at rest (KMS on EKS) and restrict access with RBAC.

Both can be consumed as **environment variables** or **mounted files**.

Important behavior: env vars are read only at container start, so changing a ConfigMap does **not** update env vars in running Pods (you must restart/roll the Deployment). Mounted files are updated automatically after a short delay (except `subPath` mounts).

Example:

```
kubectl create configmap app-cfg --from-literal=LOG_LEVEL=info
kubectl create secret generic db-cred --from-literal=DB_PASS='S3cr3t!'

spec:
  containers:
  - name: app
    image: myapp:1.0
    envFrom:
    - configMapRef: {name: app-cfg}
    env:
    - name: DB_PASS
      valueFrom:
        secretKeyRef: {name: db-cred, key: DB_PASS}
    volumeMounts:
    - {name: cfg, mountPath: /etc/app}
  volumes:
  - name: cfg
    configMap: {name: app-cfg}

echo 'UzNjcjN0IQ==' | base64 -d   # anyone with read access can decode
```

What to say to the interviewer: *ConfigMaps hold non-sensitive config and Secrets hold sensitive values; both are injected as env vars or mounted files so config is decoupled from the image. Secrets are only base64-encoded, so I enable KMS encryption at rest, lock them down with RBAC, and in production prefer syncing from AWS Secrets Manager or Vault using External Secrets Operator.*

Q14. What is the rolling update strategy?

In simple words: Changing the tyres of a bus one at a time while it is still running, so passengers never have to get off.

Detailed answer

Rolling update (default Deployment strategy) replaces old Pods with new ones **gradually**, so the app stays available. Two knobs:

- `maxSurge`: how many **extra** Pods above desired count can exist during rollout (e.g. 25% or 1).
- `maxUnavailable`: how many Pods can be **down** during rollout. Set `0` for zero downtime.

A new Pod only counts as available when its **readiness probe** passes (plus `minReadySeconds`), so a broken version stops the rollout instead of taking down the app. The other strategy is `Recreate` (kill all, then start new: downtime, used when two versions cannot run together).

Example:

```
spec:
  replicas: 4
  minReadySeconds: 10
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 0
```

```
kubectl set image deploy/web web=myapp:2.0
kubectl rollout status deploy/web
kubectl rollout undo deploy/web          # if something is wrong
```

What to say to the interviewer: *A rolling update creates new pods and removes old ones in small batches controlled by maxSurge and maxUnavailable. With maxUnavailable 0 and a proper readiness probe, capacity never drops and a bad version simply stalls the rollout, and I can revert with kubectl rollout undo.*

Q15. How do you auto-scale pods? Explain Horizontal Pod Autoscaling (HPA) and the other types of scaling.

Also asked as: What types of scaling exist in Kubernetes? How do you autoscale pods on CPU plus custom metrics in EKS? How do you implement HPA in AKS?

In simple words: When the queue at a ticket counter gets long, the manager opens more counters; when it gets quiet, he closes some.

Detailed answer

Types of scaling:

Type	What it changes	Trigger
Manual	Replica count	`kubectl scale deploy web --replicas=5`
HPA	Number of pods	CPU, memory, custom/external metrics
VPA	CPU/memory requests of each pod (bigger/smaller pods)	Historical usage; restarts pods to apply
KEDA	Number of pods, including to zero	Events: Kafka lag, SQS depth, cron, Prometheus
Cluster Autoscaler / Karpenter	Number of nodes	Pending pods that do not fit / empty nodes

Do not run HPA and VPA on the same CPU/memory metric for one workload - they fight each other.

HPA automatically changes the **number of Pod replicas** of a Deployment/StatefulSet based on metrics.

How it works:

1. **metrics-server** collects CPU/memory from kubelets every ~15s.
2. The HPA controller checks every 15s and calculates: `desiredReplicas = ceil(currentReplicas * currentMetric / targetMetric)`.
3. It scales within `minReplicas` and `maxReplicas`, with a **scale-down stabilization window** (default 5 min) to avoid flapping.

Requirements: Pods **must have resource requests** set, because CPU utilization % is calculated against the request.

CPU plus custom metrics (e.g. on EKS):

- `metrics-server` serves the **resource metrics API** (CPU/memory) - install it as an EKS add-on.
- **prometheus-adapter** serves the **custom metrics API** (`custom.metrics.k8s.io`) from Prometheus queries, e.g. requests per second per pod -> use a `Pods` metric.
- **KEDA** serves the **external metrics API** for things outside the cluster (SQS depth, Kafka lag, CloudWatch).
- With several metrics in one HPA, it computes replicas for each and uses the **highest**.

AKS specifics: metrics-server is **built in**; the same `autoscaling/v2` HPA YAML works; event-driven scaling via the **AKS KEDA add-on** (`az aks update --enable-keda`); node scaling with the **cluster autoscaler enabled per node pool** (`az aks nodepool update --enable-cluster-autoscaler --min-count 2 --max-count 10`).

HPA adds Pods; if nodes are full, **Cluster Autoscaler or Karpenter** adds nodes. VPA (Vertical) changes the size of Pods instead of the count.

Example:

```
apiVersion: autoscaling/v2
kind: HorizontalPodAutoscaler
metadata: {name: web}
spec:
  scaleTargetRef: {apiVersion: apps/v1, kind: Deployment, name: web}
  minReplicas: 3
  maxReplicas: 20
  metrics:
  - type: Resource
    resource:
      name: cpu
      target: {type: Utilization, averageUtilization: 70}
  - type: Pods                                # custom metric via prometheus-adapter
    pods:
      metric: {name: http_requests_per_second}
      target: {type: AverageValue, averageValue: "100"}
```

```
kubectl get hpa web     # TARGETS 85%/70%, 140/100  REPLICAS 4 -> scaling up
kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1 | head -c 300
```

> ***What to say to the interviewer:*** *Kubernetes offers manual scaling, HPA for pod count, VPA for pod size, KEDA for event-driven scaling and Cluster Autoscaler or Karpenter for nodes. HPA changes replica count based on metrics like CPU against the pod's request, using the formula current replicas times current over target, and custom metrics come from prometheus-adapter. It needs metrics-server and resource requests, and I pair it with Cluster Autoscaler or Karpenter so new pods have nodes to land on. For event-driven metrics like Kafka lag I use KEDA.*

Q16. What are liveness, readiness and startup probes? How are they different and how do you use them?

Also asked as: What are the different types of probes in a Deployment YAML? What is a startup probe? How does Kubernetes decide a pod is ready?

> **In simple words:** Readiness asks "are you ready to serve customers?" (if no, stop sending people). Liveness asks "are you alive?" (if no, restart you). Startup asks "have you finished waking up?" (until yes, do not run the other checks).

Detailed answer

Probe	Question	On failure	Use for
Readiness	Can it take traffic now?	Removed from Service endpoints (not restarted)	Warm-up, dependency down, overloaded
Liveness	Is it stuck/dead?	Container is **restarted**	Deadlocks, hung process
Startup	Has it finished starting?	Restarted after failureThreshold x period	Slow-starting apps (Java, big caches)

While the startup probe has not succeeded, liveness and readiness are disabled, so a slow app is not killed during boot.

How Kubernetes decides a pod is **Ready**: the pod condition `Ready=True` only when **all containers are running and their readiness probes pass** (plus any custom `readinessGates`). A container with no readiness probe is considered ready as soon as it starts. Only Ready pods are added to Service endpoints, and a Deployment counts a pod as available after it stays Ready for `minReadySeconds`. Probe types: `httpGet`, `tcpSocket`, `exec`, `grpc`.

Good practice:

- Liveness should check only the process itself, **not** downstream DBs (otherwise a DB blip restarts every pod).
- Readiness can check critical dependencies.
- Do not make liveness too aggressive; it causes CrashLoopBackOff under load.

Example:

```
startupProbe:
  httpGet: {path: /healthz, port: 8080}
  failureThreshold: 30
  periodSeconds: 5          # up to 150s to start
readinessProbe:
  httpGet: {path: /ready, port: 8080}
  periodSeconds: 5
  failureThreshold: 3
livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  periodSeconds: 10
  timeoutSeconds: 2
  failureThreshold: 3
```

> ***What to say to the interviewer:*** *Readiness controls traffic, liveness controls restarts, and startup protects slow-booting apps by delaying the other two. I keep liveness light and dependency-free, put dependency checks in readiness, and use a startup probe instead of a huge initialDelaySeconds.*

Q17. Explain the Kubernetes Pod lifecycle.

> **In simple words:** A pod's life is like a flight: waiting at the gate (Pending), boarding checks (init containers), flying (Running), landing normally (Succeeded) or crashing (Failed). When it is time to land, the crew gets a warning (SIGTERM) and some time to finish before the doors are forced open (SIGKILL).

Detailed answer

Pod phases (`status.phase`):

Phase	Meaning
Pending	Accepted, but not all containers are running yet: waiting for scheduling, image pull, volume attach
Running	Bound to a node, at least one container running (or starting/restarting)
Succeeded	All containers exited with 0 and will not restart (Jobs)
Failed	All containers stopped and at least one exited non-zero / was killed
Unknown	Node not reporting, state cannot be obtained

Note: `CrashLoopBackOff`, `ImagePullBackOff`, `OOMKilled` are **container state reasons**, not phases. Container states are **Waiting**, **Running**, **Terminated**.

Startup order:

1. Scheduled to a node -> volumes mounted, images pulled.

2. **Init containers** run one by one, each must succeed (e.g. wait for DB, run migrations, fetch config).
3. App containers start; **postStart** hook runs; **startup probe**, then readiness/liveness probes.
4. Ready -> added to Service endpoints.

Graceful termination:

1. Pod marked Terminating and removed from endpoints (in parallel).
2. **preStop** hook runs (e.g. `sleep 10`).
3. **SIGTERM** sent to the container's main process (PID 1).
4. Kubernetes waits up to `terminationGracePeriodSeconds` (default **30s**, counting the preStop time).
5. Still running -> **SIGKILL** (exit code 137).

`restartPolicy`: Always (Deployments), OnFailure / Never (Jobs).

Example:

```
spec:
  terminationGracePeriodSeconds: 40
  initContainers:
  - name: wait-db
    image: busybox:1.36
    command: ["sh", "-c", "until nc -z postgres 5432; do sleep 2; done"]
  containers:
  - name: app
    image: myapp:1.0
    lifecycle:
      preStop: {exec: {command: ["sh", "-c", "sleep 10"]}}

kubectl get pod app-xyz -o jsonpath='{.status.phase}'
kubectl get pod app-xyz -o jsonpath='{.status.containerStatuses[0].state}'
```

> ***What to say to the interviewer:*** *A pod moves through Pending, Running, then Succeeded or Failed, with Unknown when the node stops reporting; init containers run to completion first, then app containers with hooks and probes. On deletion it is removed from endpoints, preStop runs, SIGTERM is sent, and after terminationGracePeriodSeconds, 30 by default, it gets SIGKILL, so apps must handle SIGTERM gracefully.*

Q18. What are PV and PVC? Which comes first, and why?

> **In simple words:** A PV is an actual storage locker; a PVC is a request slip saying "I need a 10 GB locker". Either the building manager pre-builds lockers (static), or a locker factory (StorageClass) builds one the moment a slip arrives (dynamic).

Detailed answer

- **PersistentVolume (PV):** a piece of real storage in the cluster (EBS volume, EFS, NFS, disk). Cluster-scoped, managed by admins or created automatically.
- **PersistentVolumeClaim (PVC):** a namespaced **request** by a user/pod for storage: size, access mode, StorageClass. The pod mounts the PVC, not the PV.
- **StorageClass:** describes how to create storage dynamically (provisioner `ebs.csi.aws.com`, type gp3, reclaim policy, `allowVolumeExpansion`, `volumeBindingMode`).

Which comes first?

- **Static provisioning:** admin creates the **PV first**, then a PVC claims it (binding by size, access mode, class, selector).
- **Dynamic provisioning (most common in cloud):** you create the **PVC first**; the StorageClass provisioner creates the PV automatically and binds it. With `WaitForFirstConsumer`, the volume is created only when a pod is scheduled, so it lands in the pod's AZ.

Binding is 1-to-1. Access modes: RWO (one node), ROX, RWX (many nodes, e.g. EFS), RWOP. Reclaim policy: **Delete** (volume removed with PVC) or **Retain** (keep data for manual recovery - use for important data).

Example:

```
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata: {name: gp3}
provisioner: ebs.csi.aws.com
parameters: {type: gp3}
reclaimPolicy: Retain
allowVolumeExpansion: true
volumeBindingMode: WaitForFirstConsumer
---
apiVersion: v1
kind: PersistentVolumeClaim
metadata: {name: data}
spec:
  accessModes: [ReadWriteOnce]
  storageClassName: gp3
  resources: {requests: {storage: 20Gi}}

kubectl get pvc,pv       # STATUS Bound
```

> ***What to say to the interviewer:*** *A PV is the actual storage and a PVC is a namespaced request for it that pods mount. With static provisioning the admin creates the PV first; in the cloud we normally use dynamic provisioning where the PVC comes first and a StorageClass such as gp3 on the EBS CSI driver creates and binds the PV, ideally with WaitForFirstConsumer and Retain for important data.*

Q19. What is RBAC in Kubernetes, what is it used for, and how do you configure it?

> **In simple words:** RBAC is the building's access-card system. A Role says which doors exist on a card ("can open deployment and pod doors on floor prod"); a RoleBinding hands that card to a person or robot.

Detailed answer

RBAC (Role-Based Access Control) decides **who can do what on which resources**. Used for least privilege for developers, CI/CD, and applications.

Building blocks:

Object	Scope	Purpose
Role	Namespace	List of allowed verbs on resources in one namespace
ClusterRole	Cluster	Same, cluster-wide or for cluster-scoped resources (nodes, PVs); can be reused in namespaces
RoleBinding	Namespace	Gives a Role/ClusterRole to subjects inside one namespace
ClusterRoleBinding	Cluster	Gives a ClusterRole everywhere
Subjects	-	User, Group (from IAM/OIDC), ServiceAccount (for pods and CI)

Verbs: get, list, watch, create, update, patch, delete. RBAC is **additive only** (no deny rules); anything not granted is denied.

Best practices: namespace-scoped Roles, groups instead of individual users, no wildcard `*`, avoid giving `secrets` list/get and `pods/exec` broadly, separate ServiceAccount per app, audit with `kubectl auth can-i`.

Example:

```
apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata: {name: dev-read-deploy, namespace: team-a}
rules:
- apiGroups: ["", "apps"]
  resources: ["pods", "pods/log", "deployments"]
  verbs: ["get", "list", "watch"]
---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata: {name: devs-read, namespace: team-a}
subjects:
- {kind: Group, name: team-a-devs, apiGroup: rbac.authorization.k8s.io}
roleRef: {kind: Role, name: dev-read-deploy, apiGroup: rbac.authorization.k8s.io}

kubectl auth can-i delete pods -n team-a --as-group=team-a-devs --as=test   # no
kubectl auth can-i --list --as=system:serviceaccount:team-a:api -n team-a
```

> ***What to say to the interviewer:*** *RBAC controls who can perform which verbs on which resources: Roles and ClusterRoles define permissions, and RoleBindings or ClusterRoleBindings grant them to users, groups or ServiceAccounts. I keep it least-privilege and namespace-scoped, map IAM or SSO groups to Kubernetes groups, and verify with kubectl auth can-i.*

Q20. What does Kubernetes administration mean to you?

> **In simple words:** A Kubernetes admin is the building manager, not a tenant. Tenants (developers) run their shops; the manager keeps the lifts, power, security, water and fire exits working, plans for growth, and handles emergencies.

Detailed answer

It is owning the **platform** so application teams can deploy safely:

Area	What it covers
Cluster lifecycle	Provisioning with IaC (Terraform/eksctl), version upgrades one minor at a time, version skew between control plane, kubelet and kubectl, deprecated APIs
Node management	Node groups / Karpenter, AMI updates, cordon/drain, taints and labels, capacity planning
Access & RBAC	SSO/IAM mapping (access entries), namespaced Roles, ServiceAccounts, IRSA/Pod Identity, least privilege
Networking	CNI (IPs, subnets), Ingress controllers, load balancers, CoreDNS, NetworkPolicies, TLS (cert-manager)
Storage	StorageClasses, CSI drivers (EBS/EFS), PVC expansion, reclaim policies, snapshots
Add-ons	CNI, CoreDNS, kube-proxy, metrics-server, CSI, autoscaler, External Secrets - kept on compatible versions
Security	Pod Security Admission, Gatekeeper/Kyverno policies, image scanning, secret encryption (KMS), audit logs
Observability	Prometheus/Grafana metrics, Loki/ELK logs, tracing, alerting with runbooks
Backup & DR	etcd snapshots (self-managed), Velero, GitOps rebuild, restore drills
Capacity & cost	Requests/limits, ResourceQuota/LimitRange, rightsizing, Spot, autoscaling
Incident response	On-call, troubleshooting nodes/pods/network, postmortems

Example:

```
kubectl get nodes -o wide ; kubectl top nodes
kubectl get pods -A | grep -vE "Running|Completed"
kubectl version ; aws eks list-addons --cluster-name prod-eks
kubectl auth can-i --list --as=system:serviceaccount:prod:api -n prod
kubectl get resourcequota -A ; kubectl get pdb -A
velero backup get
```

What to say to the interviewer: *To me Kubernetes administration means owning the platform end to end: cluster provisioning and upgrades, node management, RBAC and identity, networking, ingress and DNS, storage classes, add-ons, security policies, observability, backup and DR, capacity and cost, and incident response. The goal is that application teams can deploy self-service and safely without worrying about the cluster underneath.*

Q21. Rapid-fire: Kubernetes concept checks (common trick questions)

In simple words: These are short "gotcha" questions where the popular answer is slightly wrong. Knowing the precise answer shows real understanding.

Detailed answer

Question	Correct answer	Why
Two nodes, one has pods, one is empty. Where does a new pod go?	Usually the emptier node - it is not random	Scheduler first **filters** (enough requested CPU/memory, taints, affinity, ports), then **scores**; default scoring (LeastAllocated, BalancedAllocation, topology spread) favours less-allocated nodes. It uses **requests**, not actual usage
A container is OOMKilled. Is the container or the whole pod restarted?	Only that **container** is restarted (per `restartPolicy`, default Always)	The pod object, its IP and volumes stay; pod is only replaced if it is evicted (node memory pressure) or deleted
Can env vars / ConfigMap values update without recreating the pod?	Env vars: **no**, need a restart. ConfigMap mounted as a volume: **eventually** (kubelet sync, up to about a minute plus cache TTL), never for `subPath` mounts, and the app must re-read the file	Immutable ConfigMaps never change; common pattern is a checksum annotation or Reloader to trigger a rollout
Once a pod is created, is it stable?	**No**, pods are ephemeral	Node failure, eviction, preemption, drains and rollouts replace them - that is why controllers and Services exist
Does a ClusterIP Service load-balance TCP?	**Yes, per connection (L4)**, not per request	kube-proxy iptables picks a random endpoint, IPVS supports algorithms; long-lived connections (gRPC/HTTP2 keep-alive) stick to one pod - use L7 balancing or a service mesh
How do you collect app logs, and can they be lost?	App writes to **stdout/stderr**; a **DaemonSet agent** (Fluent Bit, Fluentd, Promtail/Alloy) ships them to Loki/ELK/CloudWatch. Prometheus is for **metrics, not logs**	Node-local logs are lost when the pod is deleted or the node dies; `kubectl logs` only keeps current and previous container, within rotation limits
Liveness probe passes - is the app fine?	**No**, it only proves what the probe checks (often just "process responds")	Dependencies and correctness can be broken; use readiness probes, SLO metrics and synthetic checks
How does the app scale with traffic?	**HPA** for pods + **Cluster Autoscaler/Karpenter** for nodes; **KEDA** for event-driven	HPA alone cannot help if no node has room
Is `kubectl exec -it pod -- bash` "logging into the pod"?	Not exactly: it starts a **new process inside one container's namespaces** via API server -> kubelet -> container runtime	Needs a shell in the image (distroless has none -> use `kubectl debug` ephemeral container); use `-c` to pick the container
A container keeps exiting and restarting. What do you do?	`kubectl logs --previous`, `kubectl describe` (Last State, exit code), events, probes, resources	Exit 137 = killed (OOM/SIGKILL), 1 = app error, 0 = main process finished (wrong command for a long-running pod)

Example:

```
kubectl describe pod api-x | grep -A5 "Last State"
kubectl get pod api-x -o jsonpath='{.status.containerStatuses[0].restartCount}'
kubectl debug -it api-x --image=busybox:1.36 --target=api
```

What to say to the interviewer: *These checks are about precision: the scheduler scores nodes by requests and usually picks the less-allocated one, an OOM kill restarts only the container, env vars need a restart while mounted ConfigMaps update eventually, ClusterIP balances per connection not per request, logs go through a DaemonSet shipper and not Prometheus, and a passing liveness probe does not prove the app is healthy.*

Q22. What is the NodePort port range?

In simple words: Kubernetes reserves a block of high-numbered side doors on every node for NodePort services, so they never clash with normal ports like 22 or 443.

Detailed answer

- Default NodePort range: **30000-32767** (TCP/UDP).
- If you do not specify `nodePort`, Kubernetes picks a free one from that range; you can set one explicitly within the range.
- The same port is opened on **every node**, and traffic to any node's IP on that port reaches the Service (even if no pod runs on that node, unless `externalTrafficPolicy: Local`).
- The range can be changed with the API server flag `--service-node-port-range` (not possible on managed EKS/GKE control planes).
- Remember to allow the range in Security Groups/firewalls if clients connect directly.

Example:

```
apiVersion: v1
kind: Service
metadata: {name: web-np}
spec:
  type: NodePort
  selector: {app: web}
  ports:
  - {port: 80, targetPort: 8080, nodePort: 30080}

curl http://<any-node-ip>:30080
```

What to say to the interviewer: *The default NodePort range is 30000 to 32767, configurable with the API server's service-node-port-range flag on self-managed clusters. The port opens on every node, so firewalls or security groups must allow it, and externalTrafficPolicy Local keeps the client IP and avoids an extra hop.*

Q23. What is an Ingress, and how do you expose applications on an on-premises Kubernetes cluster? Do LoadBalancer Services work on-prem?

In simple words: In the cloud, asking for a LoadBalancer is like calling a taxi company - one arrives. On-prem there is no taxi company, so you either run your own taxi service (MetalLB) or use a side door (NodePort) behind your own gatekeeper (HAProxy/F5).

Detailed answer

Ingress is an API object with HTTP/HTTPS routing rules (host, path, TLS). It does nothing alone: an **Ingress controller** (ingress-nginx, Traefik, HAProxy, AWS LB Controller) reads the rules and routes traffic to ClusterIP Services.

On-prem, LoadBalancer Services do not work out of the box - there is no cloud-controller to create an LB, so the Service stays `EXTERNAL-IP <pending>`. Options:

1. **MetalLB** (or **kube-vip**): assigns IPs from a pool you give it on your LAN and announces them via L2 (ARP) or BGP. Then `type: LoadBalancer` works like in the cloud.
2. **NodePort + external load balancer:** HAProxy/NGINX/F5 in front of all nodes on the NodePort (e.g. `node1:30080`, `node2:30080`).
3. **Ingress controller on hostNetwork / DaemonSet** listening on 80/443 of edge nodes, with DNS or a VIP (keepalived) pointing at them.
4. Then create **Ingress** resources for each app, TLS via cert-manager.

Typical design: MetalLB gives one IP to the ingress-nginx controller Service, and all apps are exposed through Ingress rules behind it.

Example:

```
apiVersion: metallb.io/v1beta1
kind: IPAddressPool
metadata: {name: lan-pool, namespace: metallb-system}
spec: {addresses: ["192.168.10.240-192.168.10.250"]}
---
apiVersion: metallb.io/v1beta1
kind: L2Advertisement
metadata: {name: l2, namespace: metallb-system}

kubectl get svc -n ingress-nginx ingress-nginx-controller
# TYPE LoadBalancer  EXTERNAL-IP 192.168.10.240   (instead of <pending>)
```

> ***What to say to the interviewer:*** *An Ingress defines host and path routing that an ingress controller like NGINX implements. On-prem, LoadBalancer Services stay pending because there is no cloud provider, so I install MetalLB or kube-vip to hand out LAN IPs, or put HAProxy or F5 in front of NodePorts, and then expose apps through an ingress controller with cert-manager for TLS.*

Q24. Can we define multiple ports in a Kubernetes Deployment? How do you verify they are correctly defined and working?

> **In simple words:** One shop can have a front door for customers and a back door for deliveries. You just have to label each door clearly.

Detailed answer

Yes. A container can list multiple `containerPorts`, and a Service can expose multiple ports. When a Service has more than one port, **each port must have a name**.

Note: `containerPort` is mostly informational; the app must actually listen on that port. Verification means checking the app, not just the YAML.

Example:

```
containers:
- name: app
  image: myapp:1.0
  ports:
  - {name: http, containerPort: 8080}
  - {name: metrics, containerPort: 9090}
---
kind: Service
spec:
  selector: {app: myapp}
  ports:
  - {name: http, port: 80, targetPort: http}
  - {name: metrics, port: 9090, targetPort: metrics}
```

Verify:

```
kubectl get pod <pod> -o jsonpath='{.spec.containers[*].ports}'
kubectl describe svc myapp              # both ports + Endpoints for each
kubectl get endpoints myapp             # 10.0.1.5:8080,10.0.1.5:9090
kubectl exec <pod> -- ss -lntp          # is the app really listening on both?
kubectl port-forward pod/<pod> 8080 9090
curl localhost:8080/health ; curl localhost:9090/metrics
```

> ***What to say to the interviewer:*** *Yes, a container can declare multiple named ports and the Service must name each port when there is more than one. I verify with describe svc and get endpoints, then exec ss -lnt inside the pod and port-forward plus curl to prove the app actually listens on each port.*

Q25. The team asks you to pass an extra startup parameter to an app deployed from a Docker image. Where do you define the startup command in Kubernetes?

> **In simple words:** The Docker image comes with a default "how to start" instruction. In Kubernetes you can overwrite it in the Pod spec without rebuilding the image.

Detailed answer

In the container spec:

- `command` overrides the image's **ENTRYPOINT**
- `args` overrides the image's **CMD**

To add a parameter while keeping the image's entrypoint, usually set only `args`. Changing `command`/`args` in a Deployment changes the Pod template, so a rolling update happens automatically.

Dockerfile	Kubernetes field
ENTRYPOINT	command
CMD	args

Example:

```
containers:
- name: api
  image: myapi:1.4
  command: ["java"]                              # optional: replaces ENTRYPOINT
  args: ["-Xmx512m", "-jar", "/app/api.jar", "--log-level=debug"]
  env:
  - name: EXTRA_OPTS
    value: "--feature-x=true"
```

```
kubectl edit deploy api        # or change YAML/Helm values and apply
kubectl get pod <pod> -o jsonpath='{.spec.containers[0].args}'
```

> ***What to say to the interviewer:*** *I define it in the container spec: command overrides the Docker ENTRYPOINT and args overrides CMD. For an extra flag I usually add it to args, apply through Git or Helm, and the Deployment rolls out new pods automatically.*

Q26. What is CoreDNS, and what role does it play in Kubernetes? How does DNS resolution work inside a pod?

Also asked as: How does DNS resolution work inside a pod, and what do you check when a service is not reachable by name?

> **In simple words:** CoreDNS is the cluster's phone directory. Apps ask "what is the number for payment-service?" and CoreDNS answers with the Service IP.

Detailed answer

CoreDNS is the cluster DNS server (runs as a Deployment in `kube-system`, exposed by the `kube-dns` Service with a fixed ClusterIP, e.g. `10.100.0.10` on EKS). Every Pod's `/etc/resolv.conf` points to it.

Resolution inside a pod, step by step:

1. kubelet writes the pod's `/etc/resolv.conf` (with the default `dnsPolicy: ClusterFirst`): `nameserver <kube-dns ClusterIP>`, `search <ns>.svc.cluster.local svc.cluster.local cluster.local` (+ VPC domain), `options ndots:5`.
2. The app looks up `payments`. Because the name has fewer than 5 dots, the resolver first tries each **search domain**: `payments.<ns>.svc.cluster.local` -> found if the Service is in the same namespace.
3. The query goes to the kube-dns ClusterIP; kube-proxy DNATs it to one CoreDNS pod.
4. CoreDNS's **kubernetes plugin** answers from its watch of Services/EndpointSlices (ClusterIP for normal Services, pod IPs for headless).
5. Names outside `cluster.local` go to the **forward plugin** -> upstream (node's `/etc/resolv.conf`, i.e. the VPC resolver on AWS).

Corefile plugins you should know: `kubernetes` (cluster records), `forward` (upstream), `cache` (caches answers, e.g. 30s), `loop` (detects forwarding loops and stops CoreDNS - common crash cause when the node resolv.conf points back to itself), `errors`, `health`/`ready` (probes), `prometheus` (metrics on :9153), `reload`, `loadbalance`.

It resolves:

- Services: `my-svc.my-namespace.svc.cluster.local` -> ClusterIP
- Headless Services / StatefulSet pods: `mongo-0.mongo.db.svc.cluster.local` -> pod IP
- External names: forwards to upstream DNS (VPC resolver on AWS)

Within the same namespace, a Pod can just call `http://my-svc`. Because of `ndots:5`, short external names cause several extra lookups; high DNS load is a common source of latency, fixed by scaling CoreDNS or using NodeLocal DNSCache.

If CoreDNS is down, service-to-service calls fail with "could not resolve host", even though Pods and Services are healthy.

Example:

```
kubectl get pods -n kube-system -l k8s-app=kube-dns
kubectl run dnstest --rm -it --image=busybox:1.36 -- nslookup my-svc.my-ns
kubectl exec <pod> -- cat /etc/resolv.conf
# nameserver 10.100.0.10
# search prod.svc.cluster.local svc.cluster.local cluster.local ap-south-1.compute.internal
# options ndots:5
kubectl get svc kube-dns -n kube-system
kubectl get cm coredns -n kube-system -o yaml
#   .:53 { errors; health; ready
#          kubernetes cluster.local in-addr.arpa ip6.arpa { pods insecure; fallthrough in-addr.arpa
    ip6.arpa }
#          prometheus :9153; forward . /etc/resolv.conf; cache 30; loop; reload; loadbalance }
kubectl logs -n kube-system -l k8s-app=kube-dns
```

When a service is not reachable by name, check: short name vs `svc.ns` vs FQDN, the pod's resolv.conf and dnsPolicy, CoreDNS pods/logs, whether the Service exists with endpoints, and NetworkPolicy allowing port 53 (see the scenario questions on cross-namespace discovery and cluster-wide DNS failures).

> ***What to say to the interviewer:*** *CoreDNS is the cluster DNS that turns Service names into ClusterIPs and forwards external names upstream. Inside a pod, resolv.conf points to the kube-dns ClusterIP with namespace search domains and ndots:5, CoreDNS's kubernetes plugin answers cluster names and the forward plugin sends the rest upstream, with cache and loop plugins in the Corefile. When names fail I check the FQDN, resolv.conf, CoreDNS pods and logs and a busybox nslookup test.*

Q27. What is Helm and why is it important? Explain the Helm chart folder structure and the commands you use to deploy.

Also asked as: What is the folder structure of a Helm chart? Which Helm commands do you use?

> **In simple words:** Helm is like an app store and installer for Kubernetes. Instead of copying 10 YAML files and editing them by hand for each environment, you install one package and pass a small settings file.

Detailed answer

Helm is the package manager for Kubernetes.

- **Chart:** a package of templated YAML (Deployment, Service, Ingress, HPA...).
- **values.yaml:** the inputs (image tag, replicas, resources) - one file per environment.
- **Release:** an installed instance of a chart in a cluster, with **revision history**.

Why it matters:

- One chart, many environments (`values-dev.yaml`, `values-prod.yaml`) - DRY.
- Versioned releases and one-command **rollback**.
- Install third-party tools easily (ingress-nginx, Prometheus, cert-manager).
- Works well with Argo CD/Jenkins for GitOps/CI-CD.

Chart folder structure:

```
myapp/
  Chart.yaml          # chart name, version (chart), appVersion (app), dependencies
  values.yaml         # default inputs: image, replicas, resources, ingress...
  values-prod.yaml    # (our convention) per-environment overrides
  charts/             # dependency sub-charts (e.g. redis) pulled by helm dependency update
  templates/
    deployment.yaml   # uses {{ .Values.image.tag }}, {{ .Release.Name }}
    service.yaml
    ingress.yaml
    hpa.yaml
    configmap.yaml
    _helpers.tpl      # reusable named templates: names, labels (not rendered itself)
    NOTES.txt         # message printed after install
  .helmignore
```

Commands I use:

Command	Purpose
`helm lint ./myapp`	Check chart for errors
`helm template myapp ./myapp -f values-prod.yaml`	Render YAML locally to review
`helm install` / `helm upgrade --install`	First install / idempotent install-or-upgrade (used in CI)
`helm diff upgrade ...` (plugin)	Show what will change before upgrading
`helm list -n prod`, `helm history myapp -n prod`	See releases and revisions
`helm rollback myapp 3 -n prod`	Go back to revision 3
`helm uninstall myapp -n prod`	Remove release

Example:

```
helm repo add bitnami https://charts.bitnami.com/bitnami
helm create myapp                          # scaffold a chart
helm lint ./myapp
helm template myapp ./myapp -f values-prod.yaml   # render and review YAML
helm upgrade --install myapp ./myapp -n prod --create-namespace \
  -f values-prod.yaml --set image.tag=1.4.2 --atomic --timeout 5m
helm history myapp -n prod
helm rollback myapp 3 -n prod

# templates/deployment.yaml snippet
image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
replicas: {{ .Values.replicaCount }}
```

> ***What to say to the interviewer:*** *Helm packages Kubernetes manifests into versioned, templated charts: Chart.yaml holds metadata, values.yaml the inputs, templates the YAML with placeholders*

and _helpers.tpl reusable snippets. In CI I run helm lint and helm template, then helm upgrade --install with an environment values file and --atomic so a failed upgrade rolls back automatically, and helm rollback for manual reverts.

Q28. What is KEDA, and how does it help with event-driven autoscaling?

In simple words: Normal HPA looks at how tired the workers are (CPU). KEDA looks at how many parcels are waiting at the door (queue length) and hires workers before they get tired - and can send everyone home when there are zero parcels.

Detailed answer

KEDA (Kubernetes Event-Driven Autoscaling) scales workloads based on **external event sources**: Kafka consumer lag, SQS queue depth, RabbitMQ, Redis lists, Prometheus queries, cron schedules and 60+ others.

How it works:

- You create a `ScaledObject` pointing at a Deployment and a trigger.
- KEDA acts as a **metrics adapter** and creates/manages an HPA under the hood.
- It can **scale to zero** when there are no events and back to 1 when events arrive (plain HPA minimum is 1).

Great for consumers/workers where CPU is a poor signal (a consumer can be idle on CPU while lag grows).

Example:

```
apiVersion: keda.sh/v1alpha1
kind: ScaledObject
metadata: {name: order-consumer}
spec:
  scaleTargetRef: {name: order-consumer}
  minReplicaCount: 0
  maxReplicaCount: 30
  triggers:
  - type: kafka
    metadata:
      bootstrapServers: kafka:9092
      consumerGroup: orders
      topic: orders
      lagThreshold: "100"      # ~1 pod per 100 messages of lag
```

What to say to the interviewer: *KEDA extends HPA to scale on event sources such as Kafka lag, SQS depth or Prometheus queries, and it can scale to zero. I use it for queue consumers where CPU does not reflect backlog, and cap maxReplicaCount at the topic's partition count because extra consumers would sit idle.*

Q29. How many nodes are there in your Kubernetes cluster? How do you decide?

In simple words: There is no magic number - it is like asking how many buses a school needs. It depends on how many students, how big the buses are, and whether you keep a spare.

Detailed answer

Answer honestly with your numbers and the reasoning. A typical answer structure:

- **Control plane:** managed by AWS in EKS (we do not see those nodes); self-managed clusters use 3 control-plane nodes for etcd quorum.
- **Worker nodes:** spread across at least **3 AZs** so one AZ failure does not take down the app; minimum 2-3 per node group for HA.
- Separate **node groups** for different workloads (general apps, memory-heavy, isolated workloads) using taints/labels.
- Count is **not fixed** - Cluster Autoscaler/Karpenter scales between min and max.

Sizing logic: sum of Pod requests + ~20-30% headroom + DaemonSet overhead + the per-node Pod limit (on EKS the VPC CNI limits Pods per node by ENIs/IPs).

Example:

```
kubectl get nodes -L topology.kubernetes.io/zone,eks.amazonaws.com/nodegroup
# NAME                STATUS  ZONE         NODEGROUP
# ip-10-0-1-12...     Ready   ap-south-1a  app-ng
# ip-10-0-2-40...     Ready   ap-south-1b  app-ng
# ip-10-0-3-77...     Ready   ap-south-1c  pull-backend-ng
kubectl top nodes
```

> ***What to say to the interviewer:*** *Our EKS control plane is managed by AWS, and worker nodes run in managed node groups spread over three AZs, autoscaling roughly between a minimum and maximum rather than a fixed number. We size from total pod requests plus headroom, and keep separate node groups for workloads that need isolation.*

Q30. How do you deploy an application in a Kubernetes (EKS) cluster? Explain the end-to-end flow.

> **In simple words:** Developer writes code -> a factory (CI) packs it into a box (image) -> the box goes to a warehouse (ECR) -> the delivery order (manifest/Helm) tells Kubernetes to use the new box -> Kubernetes swaps old boxes for new ones gradually.

Detailed answer

Typical flow:

1. Developer pushes code / merges PR to Git.
2. CI (Jenkins/GitHub Actions) runs tests, builds Docker image, scans it (Trivy), tags it with version/commit SHA.
3. Pushes image to **ECR**.
4. CD updates the image tag in Helm values or manifests:

- Push-based: Jenkins runs `helm upgrade` / `kubectl apply` using a kubeconfig/IAM role.
- GitOps: commit the new tag to a config repo; **Argo CD** syncs it to the cluster.

5. API server stores the new spec; Deployment creates a new ReplicaSet; rolling update starts.
6. Scheduler places Pods; kubelet pulls image from ECR (node IAM role permits it).
7. Readiness probe passes -> Pod added to Service endpoints -> Ingress/ALB sends traffic.
8. Verify: rollout status, dashboards, error rates; rollback if bad.

Example:

```
docker build -t 1234.dkr.ecr.ap-south-1.amazonaws.com/api:1.4.2 .
docker push 1234.dkr.ecr.ap-south-1.amazonaws.com/api:1.4.2
aws eks update-kubeconfig --name prod-eks --region ap-south-1
helm upgrade --install api ./charts/api -n prod --set image.tag=1.4.2 --atomic
kubectl rollout status deploy/api -n prod
```

What to say to the interviewer: *CI builds, tests, scans and pushes a versioned image to ECR; CD updates the image tag in Helm values and either Jenkins runs helm upgrade or Argo CD syncs from Git. Kubernetes then performs a rolling update gated by readiness probes, and I verify with rollout status and metrics, rolling back with helm rollback if error rates rise.*

Q31. When deploying to Kubernetes, do you only update Docker images, or also replicas, storage and CPU allocation?

In simple words: Upgrading a shop is not just new stock (image). Sometimes you add staff (replicas), a bigger storeroom (storage) or more power (CPU/memory).

Detailed answer

All of them, but through the same Git/Helm process, and each behaves differently:

Change	How	Effect
Image tag	values.yaml `image.tag`	Rolling update
Replicas	`replicas` or better HPA min/max	No restart, just add/remove pods (do not hard-set replicas if HPA manages them)
CPU/memory requests/limits	`resources`	Rolling update (pods recreated); resize from real usage (VPA recommendations, Grafana)
Storage	PVC `spec.resources.requests.storage`	Can only **grow**, and only if StorageClass has `allowVolumeExpansion: true`
Config	ConfigMap/Secret	Needs pod restart unless mounted as files (use checksum annotation)

Example:

```
# values-prod.yaml
image: {tag: "1.4.2"}
autoscaling: {minReplicas: 3, maxReplicas: 15}
resources:
  requests: {cpu: 250m, memory: 512Mi}
  limits: {memory: 1Gi}

kubectl patch pvc data-mongo-0 -p '{"spec":{"resources":{"requests":{"storage":"50Gi"}}}}'
```

What to say to the interviewer: *Besides images, we tune replicas or HPA bounds, CPU and memory requests based on observed usage, and expand PVCs when storage grows. All changes go through Helm values in Git so they are reviewed and versioned, and I remember resource changes restart pods while PVCs can only be expanded, not shrunk.*

Advanced

Q32. What are master (control-plane) and worker nodes, and how do you create and manage Kubernetes clusters (eksctl / Terraform)?

Also asked as: How do you create an EKS cluster using Terraform, and how do the Kubernetes core components come into play?

> **In simple words:** The master is the head office that makes decisions; worker nodes are the factories that do the actual work. On EKS, AWS runs the head office for you - you only build and manage the factories.

Detailed answer

- **Master / control-plane nodes:** run API server, etcd, scheduler, controller manager. They decide and store state; normally no app Pods run there.
- **Worker nodes:** run kubelet, kube-proxy, container runtime, CNI - and your application Pods.

On **EKS** the control plane is fully managed (multi-AZ, AWS patches it, you pay per cluster-hour). You manage worker capacity as **managed node groups**, self-managed groups, **Fargate** profiles or **Karpenter**-provisioned nodes.

Ways to create a cluster:

- **eksctl:** quickest; one YAML/command, good for labs.
- **Terraform** (production): usually the `terraform-aws-modules/eks` module - VPC + subnets, IAM roles (cluster role, node role), the EKS cluster, node groups, add-ons (vpc-cni, coredns, kube-proxy, ebs-csi), access entries. State in S3 with locking, reviewed via `plan` in CI.

What happens when Terraform creates EKS: AWS brings up the API server and etcd in its own account, exposes an API endpoint, and injects ENIs in your subnets; when node groups join, their kubelets register with the API server; add-ons deploy CoreDNS and kube-proxy as Pods.

Day-2 management: version upgrades, add-on upgrades, node AMI rotation, access (RBAC/access entries), autoscaling, monitoring.

Example:

```
eksctl create cluster --name demo --region ap-south-1 \
  --nodegroup-name ng1 --node-type t3.medium --nodes 2 --nodes-min 2 --nodes-max 4

module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  version         = "~> 20.0"
  cluster_name    = "prod-eks"
  cluster_version = "1.30"
  vpc_id          = module.vpc.vpc_id
  subnet_ids      = module.vpc.private_subnets
  eks_managed_node_groups = {
    app = { instance_types = ["m6i.large"], min_size = 3, max_size = 10, desired_size = 3 }
  }
}

aws eks update-kubeconfig --name prod-eks --region ap-south-1
kubectl get nodes
```

What to say to the interviewer: *The control plane runs the API server, etcd, scheduler and controllers, and worker nodes run kubelet, kube-proxy and the app pods. On EKS AWS manages the control plane; I create the cluster, VPC, IAM roles, node groups and add-ons with Terraform modules, use eksctl for quick labs, and handle upgrades and node rotation as day-2 operations.*

Q33. Explain Amazon EKS architecture. What subnets are used in EKS?

Also asked as: What is a subnet and which types of subnets are used in Kubernetes/EKS?

In simple words: A subnet is a street inside your private city (VPC). Public streets face the highway (internet); private streets are behind a gate. You put the front desk (load balancer) on the public street and the workers (nodes/pods) on private streets.

Detailed answer

A **subnet** is a range of IPs inside a VPC, tied to one AZ. **Public subnet** = route table has a route to an Internet Gateway. **Private subnet** = no direct internet route; outbound via NAT Gateway or VPC endpoints.

EKS architecture:

- **Control plane** in an AWS-managed VPC, spread over 3 AZs; exposes a public and/or private API endpoint.
- **Cross-account ENIs** placed in your subnets so the control plane can talk to kubelets.
- **Worker nodes** in **private subnets** across 2-3 AZs.
- **VPC CNI:** every Pod gets a **real VPC IP** from the subnet (so subnets must be sized big, e.g. /19; or use prefix delegation / custom networking).
- **Public subnets** hold internet-facing ALB/NLB and NAT Gateways. Tag them `kubernetes.io/role/elb=1`; private ones `kubernetes.io/role/internal-elb=1` so the Load Balancer Controller finds them.
- IAM: cluster role, node role (ECR pull, CNI), IRSA/Pod Identity for apps.
- Add-ons: vpc-cni, CoreDNS, kube-proxy, EBS CSI driver, AWS Load Balancer Controller.

Example:

```
VPC 10.0.0.0/16
  public-a 10.0.0.0/24   -> ALB, NAT GW
  public-b 10.0.1.0/24   -> ALB, NAT GW
  private-a 10.0.32.0/19 -> nodes + pod IPs
  private-b 10.0.64.0/19 -> nodes + pod IPs
  private-c 10.0.96.0/19 -> nodes + pod IPs

aws eks describe-cluster --name prod-eks \
  --query 'cluster.resourcesVpcConfig.{subnets:subnetIds,public:endpointPublicAccess}'
```

What to say to the interviewer: *EKS runs a managed multi-AZ control plane and connects to our VPC with ENIs; worker nodes live in private subnets across at least two AZs, while public subnets host internet-facing load balancers and NAT gateways. Because the VPC CNI gives every pod a VPC IP, I size private subnets generously and tag subnets so the load balancer controller can discover them.*

Q34. How does authentication work in EKS, and how do pods get AWS permissions and secrets securely?

Also asked as: How do you store secrets securely in EKS?

In simple words: To enter the building (cluster), you show your AWS ID card; a guest list (access entries / aws-auth) says which room you can enter (RBAC). Pods get their own temporary badges (IRSA / Pod Identity) instead of sharing a master key.

Detailed answer

Humans and CI -> cluster:

1. `kubectl` calls `aws eks get-token` (built into kubeconfig; older setups used `aws-iam-authenticator`) which creates a signed, short-lived token from your IAM identity.
2. EKS verifies the IAM identity (authentication).
3. It maps the IAM user/role to a Kubernetes user/group via:

- the legacy **aws-auth ConfigMap** in kube-system (easy to break, one typo can lock everyone out), or
- the newer **EKS access entries** (API-managed, with access policies like AmazonEKSClusterAdminPolicy) - preferred.

4. Kubernetes **RBAC** decides what that user/group can do (authorization).

Pods -> AWS services:

- **IRSA** (IAM Roles for Service Accounts): OIDC provider + annotate ServiceAccount with a role ARN; pod gets temporary credentials.
- **EKS Pod Identity:** newer, simpler (agent add-on + association, no OIDC setup per cluster).
- Never put AWS access keys in Secrets or images; avoid relying on the node role for apps.

Secrets: enable **KMS envelope encryption** for Secrets, restrict `get secrets` via RBAC, and keep the source of truth in **AWS Secrets Manager / SSM Parameter Store**, synced by **External Secrets Operator** or the Secrets Store CSI driver using IRSA.

Example:

```
aws eks create-access-entry --cluster-name prod-eks \
  --principal-arn arn:aws:iam::111122223333:role/DevOpsRole
aws eks associate-access-policy --cluster-name prod-eks \
  --principal-arn arn:aws:iam::111122223333:role/DevOpsRole \
  --policy-arn arn:aws:eks::aws:cluster-access-policy/AmazonEKSClusterAdminPolicy \
  --access-scope type=cluster

apiVersion: v1
kind: ServiceAccount
metadata:
  name: orders
  annotations:
    eks.amazonaws.com/role-arn: arn:aws:iam::111122223333:role/orders-s3-read
```

What to say to the interviewer: *kubectl authenticates with a short-lived token from aws eks get-token, EKS maps the IAM principal to Kubernetes identities through access entries or the older aws-auth ConfigMap, and RBAC authorizes actions. Pods get AWS permissions through IRSA or EKS Pod Identity rather than static keys, and secrets stay in Secrets Manager synced by External Secrets with KMS encryption enabled on the cluster.*

Q35. How do Jenkins and a Kubernetes cluster communicate with each other?

Also asked as: How does Jenkins deploy to Kubernetes, and how do you run Jenkins agents on Kubernetes?

In simple words: Jenkins needs a visitor pass to the cluster. You give it a limited pass (a service account or an IAM role) that only opens the doors it needs, instead of the admin master key.

Detailed answer

Two directions:

1. **Jenkins deploys to Kubernetes:**

- Jenkins agent has `kubectl`/`helm` and a **kubeconfig**.

- On EKS: Jenkins runs with an **IAM role** (EC2 instance profile or IRSA if Jenkins is in-cluster); `aws eks update-kubeconfig` generates the kubeconfig, and the role is mapped via access entry to a namespaced RBAC role.

- Non-EKS: a Kubernetes **ServiceAccount** with a Role/RoleBinding limited to the target namespace; its token/kubeconfig stored in **Jenkins Credentials** (never in the Jenkinsfile).

2. **Kubernetes runs Jenkins agents:** the **Kubernetes plugin** creates an ephemeral agent Pod per build and deletes it after, so agents scale on demand.

Alternative (GitOps): Jenkins does not touch the cluster at all - it only pushes a new image tag to Git and **Argo CD** (inside the cluster) pulls the change. This removes cluster credentials from CI.

Example:

```
stage('Deploy') {
  steps {
    withCredentials([file(credentialsId: 'kubeconfig-prod', variable: 'KUBECONFIG')]) {
      sh 'helm upgrade --install api charts/api -n prod --set image.tag=${GIT_COMMIT}'
    }
  }
}

# RBAC for the deployer SA (namespace-scoped)
kubectl create role deployer -n prod --verb=get,list,watch,create,update,patch \
  --resource=deployments,services,configmaps
kubectl create rolebinding jenkins-deployer -n prod --role=deployer \
  --serviceaccount=cicd:jenkins
```

What to say to the interviewer: *Jenkins talks to the cluster API with a kubeconfig backed by either an IAM role mapped through EKS access entries or a namespace-scoped ServiceAccount stored in Jenkins credentials, never an admin kubeconfig. The Kubernetes plugin lets Jenkins run ephemeral agent pods, and in GitOps setups Jenkins just updates Git and Argo CD applies the change from inside the cluster.*

Q36. You have Docker images for a frontend, a backend and a database. How do you deploy them on Kubernetes with YAML?

In simple words: Three tenants move into the building: the frontend gets the ground-floor shop with a public door (Ingress), the backend gets an internal office (ClusterIP), and the database gets a vault with its own locked storage (StatefulSet + PVC).

Detailed answer

Plan:

- **Namespace** `shop`.
- **Secret** for DB password, **ConfigMap** for non-secret settings (DB host, API URL).
- **Database:** StatefulSet + headless Service + `volumeClaimTemplates` (PVC). In production on AWS, a managed DB (RDS) is often better.
- **Backend:** Deployment (replicas 2+, probes, resources) + ClusterIP Service; reads DB host from ConfigMap, password from Secret.
- **Frontend:** Deployment + ClusterIP Service.
- **Ingress:** `/` -> frontend, `/api` -> backend, TLS.

Example:

```
apiVersion: v1
kind: Secret
metadata: {name: db-secret, namespace: shop}
type: Opaque
stringData: {POSTGRES_PASSWORD: "change-me"}
---
apiVersion: v1
kind: ConfigMap
metadata: {name: app-config, namespace: shop}
data: {DB_HOST: "postgres.shop.svc.cluster.local", DB_NAME: "shop"}
---
apiVersion: v1
kind: Service
metadata: {name: postgres, namespace: shop}
spec: {clusterIP: None, selector: {app: postgres}, ports: [{port: 5432}]}
---
apiVersion: apps/v1
kind: StatefulSet
metadata: {name: postgres, namespace: shop}
spec:
  serviceName: postgres
  replicas: 1
  selector: {matchLabels: {app: postgres}}
  template:
    metadata: {labels: {app: postgres}}
    spec:
      containers:
      - name: postgres
        image: postgres:16
        envFrom: [{secretRef: {name: db-secret}}]
        ports: [{containerPort: 5432}]
        volumeMounts: [{name: data, mountPath: /var/lib/postgresql/data}]
  volumeClaimTemplates:
  - metadata: {name: data}
    spec: {accessModes: [ReadWriteOnce], resources: {requests: {storage: 10Gi}}}
---
apiVersion: apps/v1
kind: Deployment
metadata: {name: backend, namespace: shop}
spec:
  replicas: 2
  selector: {matchLabels: {app: backend}}
  template:
    metadata: {labels: {app: backend}}
    spec:
      containers:
      - name: backend
        image: 1234.dkr.ecr.ap-south-1.amazonaws.com/backend:1.0.0
        ports: [{containerPort: 8080}]
        envFrom: [{configMapRef: {name: app-config}}]
```

```
        env:
        - name: DB_PASSWORD
          valueFrom: {secretKeyRef: {name: db-secret, key: POSTGRES_PASSWORD}}
        readinessProbe: {httpGet: {path: /health, port: 8080}}
        resources: {requests: {cpu: 200m, memory: 256Mi}, limits: {memory: 512Mi}}
---
apiVersion: v1
kind: Service
metadata: {name: backend, namespace: shop}
spec: {selector: {app: backend}, ports: [{port: 80, targetPort: 8080}]}
---
# frontend Deployment + Service: same pattern, image frontend:1.0.0, port 80
---
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata: {name: shop, namespace: shop}
spec:
  ingressClassName: nginx
  rules:
  - host: shop.example.com
    http:
      paths:
      - {path: /api, pathType: Prefix, backend: {service: {name: backend, port: {number: 80}}}}
      - {path: /, pathType: Prefix, backend: {service: {name: frontend, port: {number: 80}}}}
```

What to say to the interviewer: *I put the database in a StatefulSet with a headless Service and a PVC, the backend and frontend in Deployments with ClusterIP Services, and inject config from a ConfigMap and the password from a Secret. An Ingress routes slash to the frontend and slash-api to the backend with TLS, and in real AWS production I would usually move the database to RDS.*

Q37. You have 6 Pods and 3 Nodes. How do you ensure exactly 2 Pods run on each Node?

In simple words: You want 6 guests at 3 tables with exactly 2 per table. You set a rule: "no table may have more than 1 guest more than any other table".

Detailed answer

Best option: **topologySpreadConstraints** with `maxSkew: 1` across `kubernetes.io/hostname` and `whenUnsatisfiable: DoNotSchedule`. With 6 replicas and 3 nodes, skew 1 forces a 2-2-2 layout.

Other tools:

- **Pod anti-affinity** only gives "at most 1 per node" (required) or "prefer spreading" (preferred) - it cannot say "exactly 2".
- **DaemonSet** gives exactly 1 per node, not 2.
- Resource sizing hack (requests so only 2 fit per node) works but is fragile.

Caveats: the scheduler only enforces this at **scheduling time**; later imbalance (after a node failure and recovery) is not fixed automatically - use the **descheduler** to rebalance. Also add `nodeSelector`/taints if only those 3 nodes should be used.

Example:

```
spec:
  replicas: 6
  template:
    metadata: {labels: {app: web}}
    spec:
      topologySpreadConstraints:
      - maxSkew: 1
        topologyKey: kubernetes.io/hostname
        whenUnsatisfiable: DoNotSchedule
        labelSelector: {matchLabels: {app: web}}

kubectl get pods -l app=web -o wide --no-headers | awk '{print $7}' | sort | uniq -c
#   2 node-a
#   2 node-b
#   2 node-c
```

> ***What to say to the interviewer:*** *I use topologySpreadConstraints with maxSkew 1 on the hostname topology key and DoNotSchedule, which forces a 2-2-2 split for 6 replicas on 3 nodes. Anti-affinity cannot express exactly two per node, and because spreading is only enforced at scheduling time I would add the descheduler to rebalance after node failures.*

Q38. How would you ensure even Pod distribution across Nodes and Availability Zones?

Also asked as: How do you use multi-zone pod affinity/anti-affinity so a node or zone failure does not impact regional SLAs?

> **In simple words:** Do not keep all your eggs in one basket, and do not keep all your baskets in one room. Spread pods across nodes, and nodes across zones.

Detailed answer

Layered approach:

1. **Nodes in 3 AZs** (node groups spanning subnets in each AZ).
2. **Zone spread:** `topologySpreadConstraints` on `topology.kubernetes.io/zone` with `maxSkew: 1`, `DoNotSchedule` for critical services.
3. **Node spread:** a second constraint on `kubernetes.io/hostname` with `ScheduleAnyway` (soft) so it does not block scaling.
4. **Pod anti-affinity** (required) for replicas that must never share a node (e.g. 3 Kafka brokers, DB replicas).
5. **Pod affinity** to co-locate chatty pairs (app + its cache) in the same zone to cut latency and cross-AZ data cost.
6. **PodDisruptionBudget** so drains/upgrades never take down more than 1 replica.
7. Capacity: enough headroom in each zone to absorb one zone's pods (N+1 zones).

Watch-outs: PVCs on EBS are zone-bound - a StatefulSet pod can only restart in its volume's AZ; use `WaitForFirstConsumer` StorageClass. Cluster Autoscaler node groups should be one per AZ (or use Karpenter) so scale-up happens in the right zone.

Example:

```
topologySpreadConstraints:
- maxSkew: 1
  topologyKey: topology.kubernetes.io/zone
  whenUnsatisfiable: DoNotSchedule
  labelSelector: {matchLabels: {app: stream-edge}}
- maxSkew: 1
  topologyKey: kubernetes.io/hostname
  whenUnsatisfiable: ScheduleAnyway
  labelSelector: {matchLabels: {app: stream-edge}}
affinity:
  podAntiAffinity:
    requiredDuringSchedulingIgnoredDuringExecution:
    - labelSelector: {matchLabels: {app: kafka}}
      topologyKey: kubernetes.io/hostname
```

What to say to the interviewer: *I spread replicas across zones with a hard topology spread constraint and across nodes with a soft one, use required anti-affinity for replicas that must never share a node, and add PodDisruptionBudgets. I also keep enough per-zone headroom, remember EBS volumes are zone-bound, and configure autoscaling per zone so a zone loss is absorbed without breaking SLAs.*

Q39. How do you optimize resource requests and limits in a production cluster?

In simple words: Requests are the seats you reserve on a train; limits are the maximum luggage allowed. Reserve too many seats and the train runs half empty (waste); reserve too few and people get kicked off (evictions, throttling).

Detailed answer

- **Request** = guaranteed amount; used by the **scheduler** to place Pods and by HPA for utilization %.
- **Limit** = hard cap. CPU over limit -> **throttled** (slow). Memory over limit -> **OOMKilled**.

Approach:

1. **Measure** real usage for 1-2 weeks (Prometheus/Grafana: `container_cpu_usage_seconds_total`, `container_memory_working_set_bytes`, p95/p99).
2. **Set requests near p90-p95** of normal usage; memory **limit = request or a bit above** (memory cannot be throttled, only killed).
3. **CPU limits:** often omit or set generous, because CPU limits cause throttling latency even when the node is idle; keep requests accurate instead.
4. Use **VPA in recommendation mode** or Goldilocks/Kubecost for suggestions.
5. **QoS classes:** Guaranteed (request = limit) for critical pods -> last to be evicted; BestEffort (no requests) -> first to be evicted. Never run prod pods as BestEffort.
6. Enforce defaults with **LimitRange** and caps with **ResourceQuota** per namespace.
7. Check node packing (`kubectl describe node` allocated %) and rightsize node types; cluster autoscaler removes underused nodes.

Example:

```
resources:
  requests: {cpu: 300m, memory: 512Mi}
  limits:   {memory: 768Mi}        # no CPU limit to avoid throttling

# PromQL: CPU throttling ratio
sum(rate(container_cpu_cfs_throttled_periods_total{pod=~"api.*"}[5m]))
 / sum(rate(container_cpu_cfs_periods_total{pod=~"api.*"}[5m]))

kubectl top pods -n prod --sort-by=memory
kubectl describe node <node> | grep -A5 "Allocated resources"
```

What to say to the interviewer: *I base requests on observed p90 to p95 usage from Prometheus, set memory limits close to requests, and avoid tight CPU limits because they cause throttling. I use VPA recommendations, QoS Guaranteed for critical pods, LimitRange defaults and ResourceQuotas, and review node allocation so autoscaling can remove wasted capacity.*

Q40. How do you implement auto-scaling when traffic fluctuates heavily? How do you automate node scaling in EKS?

Also asked as: How do you design auto-scaling for very large concurrent traffic without over-provisioning? How do you automate ECS/EKS node scaling?

In simple words: Two levels of hiring: HPA hires more workers (pods) quickly; Cluster Autoscaler/Karpenter builds more rooms (nodes) when workers have no place to sit. For a known big event, you hire in advance.

Detailed answer

1. **Pod level - HPA / KEDA:**

- Scale on the metric that leads load: requests per second or concurrent sessions (via Prometheus Adapter/KEDA), not just CPU.

- Tune `behavior`: fast scale-up (e.g. +100% every 15s), slow scale-down (stabilization 5-10 min) to avoid flapping.

2. **Node level:**

- **Cluster Autoscaler:** adds nodes to ASGs when Pods are Pending; removes underused nodes.

- **Karpenter** (preferred on EKS): watches Pending pods and launches right-sized instances directly in seconds, supports mixed instance types and Spot, consolidates empty nodes.

3. **Kill cold start:**

- **Overprovisioning pods:** low-priority "pause" pods reserve spare capacity; real pods preempt them instantly while new nodes come up.

- Pre-scale before known events (scheduled scaling / raise `minReplicas` via CronJob or KEDA cron trigger).

- Small, pre-pulled images; fast startup probes.

4. **Protect:** PDBs, requests set correctly, max limits to avoid runaway cost, Spot for stateless + On-Demand base.

Example:

```
behavior:
  scaleUp:
    stabilizationWindowSeconds: 0
    policies: [{type: Percent, value: 100, periodSeconds: 15}]
  scaleDown:
    stabilizationWindowSeconds: 600
    policies: [{type: Percent, value: 10, periodSeconds: 60}]

apiVersion: karpenter.sh/v1
kind: NodePool
metadata: {name: default}
spec:
  template:
    spec:
      requirements:
      - {key: karpenter.sh/capacity-type, operator: In, values: [spot, on-demand]}
      nodeClassRef: {group: karpenter.k8s.aws, kind: EC2NodeClass, name: default}
  limits: {cpu: "1000"}
  disruption: {consolidationPolicy: WhenEmptyOrUnderutilized}
```

What to say to the interviewer: *I scale pods with HPA or KEDA on leading metrics like requests per second, with fast scale-up and slow scale-down, and scale nodes with Karpenter or Cluster Autoscaler. To avoid cold starts I keep low-priority overprovisioning pods as buffer and pre-scale before known events, with PDBs and max limits to protect availability and cost.*

Q41. How would you monitor HPA scaling decisions in real time and detect if metrics-server is lagging?

In simple words: HPA is a driver looking at the speedometer (metrics). If the speedometer is stuck, the driver makes wrong decisions. You watch both the driver's actions and whether the speedometer is fresh.

Detailed answer

Watch HPA decisions:

- `kubectl get hpa -w` and `kubectl describe hpa` (Conditions: `AbleToScale`, `ScalingActive`, `ScalingLimited`, plus events "New size: 8; reason: cpu above target").
- **kube-state-metrics** exposes `kube_horizontalpodautoscaler_status_current_replicas`, `..._desired_replicas`, `..._spec_max_replicas`. Graph current vs desired vs max in Grafana; alert when desired == max for 10 min (capped) or desired != current for too long.

Detect metrics-server lag / failure:

- `kubectl top pods` failing or old; `kubectl get apiservice v1beta1.metrics.k8s.io` shows `False (FailedDiscoveryCheck)`.
- HPA shows `<unknown>/70%` targets and `FailedGetResourceMetric` events.
- metrics-server logs (kubelet scrape errors, TLS issues), its own CPU/memory throttling; scrape interval (`--metric-resolution`, default 15s).
- Compare HPA-seen value with Prometheus-seen value; alert on staleness, e.g. `time() - timestamp(...)`.
- For custom metrics: check Prometheus Adapter / KEDA operator logs and `kubectl get --raw /apis/custom.metrics.k8s.io/v1beta1`.

Example:

```
kubectl describe hpa api -n prod | sed -n '/Conditions/,$p'
kubectl get apiservice v1beta1.metrics.k8s.io
kubectl get --raw "/apis/metrics.k8s.io/v1beta1/namespaces/prod/pods" | head -c 400
kubectl logs -n kube-system deploy/metrics-server

# Alert: HPA stuck at max
kube_horizontalpodautoscaler_status_current_replicas
  >= kube_horizontalpodautoscaler_spec_max_replicas
```

> ***What to say to the interviewer:*** *I track HPA current, desired and max replicas from kube-state-metrics in Grafana with alerts when it is pinned at max, and read describe hpa conditions and events for the reason behind each decision. For metrics-server health I check the metrics APIService status, unknown targets on the HPA, metrics-server logs and compare values against Prometheus to spot stale data.*

Q42. What are the differences between GKE Ingress and the NGINX Ingress Controller?

> **In simple words:** GKE Ingress hires Google's own reception at the city gate (Google Cloud Load Balancer). NGINX Ingress puts your own receptionist inside your building (pods), behind a simple door.

Detailed answer

	GKE Ingress (gce class)	NGINX Ingress Controller
Who routes	Google Cloud HTTP(S) Load Balancer (global, outside cluster)	NGINX pods inside the cluster
LB created	External/internal Application LB per Ingress	One L4 LoadBalancer Service in front of NGINX pods
Features	Cloud Armor (WAF), Cloud CDN, IAP, Google-managed certs, global anycast IP	Rewrites, regex paths, rate limiting, auth, canary annotations, custom snippets
Backend	Container-native via NEGs (straight to pods)	kube-proxy/Service to pods
Portability	GCP only	Any cloud/on-prem
Config change speed	Slower (LB provisioning minutes)	Near instant
Cost	Per forwarding rule/LB	Nodes running NGINX + one L4 LB

Choose GKE Ingress for global reach, Cloud Armor/CDN integration and fully managed operation; NGINX for advanced HTTP features, portability and many Ingresses sharing one LB. The same comparison applies on AWS: AWS Load Balancer Controller (ALB) vs ingress-nginx.

Example:

```
metadata:
  annotations:
    kubernetes.io/ingress.class: "gce"            # GKE external LB
    # or with spec.ingressClassName: nginx and:
    nginx.ingress.kubernetes.io/rewrite-target: /$2
    nginx.ingress.kubernetes.io/limit-rps: "20"
```

What to say to the interviewer: *GKE Ingress provisions a Google Cloud load balancer outside the cluster with native Cloud Armor, CDN and managed certificates, while NGINX Ingress runs as pods inside the cluster behind a single L4 load balancer and offers richer HTTP features and portability. I pick GKE Ingress for global, managed edge security and NGINX when I need rewrites, rate limits or cloud-neutral config.*

Q43. How do you enforce rules in Kubernetes to control which pods can talk to each other?

In simple words: By default every room in the building has open doors. NetworkPolicies are door locks: "only the backend may enter the database room".

Detailed answer

Use **NetworkPolicy**. By default all pods can talk to all pods. A NetworkPolicy selects pods (by label) and allows only listed ingress/egress traffic; once a pod is selected by any policy, anything not allowed is denied.

Steps:

1. Make sure the **CNI enforces** NetworkPolicy (Calico, Cilium, EKS VPC CNI with network policy enabled, GKE Dataplane V2). With a CNI that does not enforce it, policies silently do nothing.
2. Apply **default deny** per namespace.
3. Allow specific flows (frontend -> backend:8080, backend -> db:5432).
4. Allow **DNS egress** to kube-dns (UDP/TCP 53) or everything breaks.
5. Test with a temporary pod (`curl`, `nc`).

For L7 rules (HTTP methods/paths) or mTLS identity, use Cilium policies or a service mesh (Istio AuthorizationPolicy).

Example:

```
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: default-deny, namespace: shop}
spec: {podSelector: {}, policyTypes: [Ingress, Egress]}
---
apiVersion: networking.k8s.io/v1
kind: NetworkPolicy
metadata: {name: db-allow-backend, namespace: shop}
spec:
  podSelector: {matchLabels: {app: postgres}}
  policyTypes: [Ingress]
  ingress:
  - from: [{podSelector: {matchLabels: {app: backend}}}]
    ports: [{protocol: TCP, port: 5432}]
---
# allow DNS for all pods
spec:
  podSelector: {}
  policyTypes: [Egress]
  egress:
  - to: [{namespaceSelector: {matchLabels: {kubernetes.io/metadata.name: kube-system}}}]
    ports: [{protocol: UDP, port: 53}, {protocol: TCP, port: 53}]
```

What to say to the interviewer: *I use NetworkPolicies on a CNI that enforces them, start with a default-deny per namespace, then explicitly allow required flows such as backend to database on 5432 and DNS egress to kube-dns. For layer 7 rules or identity-based control I add Cilium policies or an Istio AuthorizationPolicy with mTLS.*

Q44. How would you secure a Kubernetes cluster at the network, pod and identity/access levels?

Also asked as: How would you secure a Kubernetes / GKE cluster?

In simple words: Secure a building in layers: fence and gates (network), rules for how each tenant behaves (pod security), and ID cards with only the rooms you need (identity and access).

Detailed answer

Network level

- Private cluster / private API endpoint (or restrict public endpoint CIDRs); nodes in private subnets.
- NetworkPolicies default-deny; Security Groups (SG for pods on EKS).
- TLS at Ingress + WAF (AWS WAF / Cloud Armor); mTLS between services with a service mesh.
- Restrict egress (egress policies, NAT/firewall allow-lists).

Pod / workload level

- **Pod Security Standards** (`restricted` via Pod Security Admission labels on namespaces).
- `runAsNonRoot`, `readOnlyRootFilesystem`, drop all capabilities, no `privileged`, no hostPath/hostNetwork.
- Signed, scanned images (Trivy in CI, ECR scanning), allow only trusted registries, pin by digest.
- Policy engines **OPA Gatekeeper / Kyverno** to enforce rules (no latest tag, must have limits).
- Resource limits to prevent noisy-neighbor DoS; runtime detection (Falco, GuardDuty EKS Runtime).

Identity & access level

- RBAC least privilege; namespace-scoped Roles; no one uses cluster-admin daily.
- EKS access entries / GKE IAM; short-lived tokens; SSO.
- Pods use **IRSA / Pod Identity / GKE Workload Identity**, not node credentials; `automountServiceAccountToken: false` where not needed.
- Secrets encrypted with KMS, external secret stores.
- **Audit logging** enabled to CloudWatch / Cloud Logging; regular version upgrades and CIS benchmark (kube-bench).

Example:

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted

securityContext:
  runAsNonRoot: true
  runAsUser: 10001
  readOnlyRootFilesystem: true
  allowPrivilegeEscalation: false
  capabilities: {drop: ["ALL"]}
  seccompProfile: {type: RuntimeDefault}

kubectl auth can-i --list --as=system:serviceaccount:prod:api -n prod
```

> ***What to say to the interviewer:*** *At the network level I use private endpoints, private subnets, default-deny NetworkPolicies, WAF and TLS or mTLS. At the pod level I enforce the restricted Pod Security Standard, non-root read-only containers, scanned images and Gatekeeper or Kyverno policies. For identity I apply least-privilege RBAC, IRSA or Workload Identity for pods, KMS-encrypted secrets and audit logging.*

Q45. How do you secure secrets in Kubernetes?

Also asked as: How do you manage secrets across 50+ services without exposing Vault access?

> **In simple words:** A Kubernetes Secret by default is a letter in a see-through envelope (base64). Securing it means a locked safe (KMS encryption), a short list of key holders (RBAC), and ideally keeping the real valuables in a bank (Secrets Manager / Vault).

Detailed answer

Kubernetes Secrets are only **base64-encoded** and stored in etcd. Layers of protection:

1. **Encryption at rest:** EKS KMS envelope encryption (GKE: application-layer secrets encryption with Cloud KMS).
2. **RBAC:** only the workloads/people that need them can `get`/`list` secrets; note that anyone who can create pods in a namespace can read its secrets through the pod.
3. **External source of truth:** AWS Secrets Manager / SSM, HashiCorp Vault, GCP Secret Manager.

- **External Secrets Operator** syncs them into K8s Secrets on a refresh interval.
- **Secrets Store CSI Driver** mounts them as files directly from the store (can avoid a K8s Secret object).
- Vault Agent injector sidecar for dynamic secrets.

4. **Git:** never commit plain Secrets; use **SOPS** or **Sealed Secrets** if they must live in Git (GitOps).
5. **Access from pods** via IRSA/Workload Identity (no static keys).
6. Rotation + audit logging (who read which secret).
7. Prefer mounting as files over env vars (env vars leak into crash dumps, `kubectl describe`, child processes).

Example:

```
apiVersion: external-secrets.io/v1beta1
kind: ExternalSecret
metadata: {name: db-cred, namespace: prod}
spec:
  refreshInterval: 1h
  secretStoreRef: {name: aws-secrets-manager, kind: ClusterSecretStore}
  target: {name: db-cred}
  data:
  - secretKey: DB_PASSWORD
    remoteRef: {key: prod/orders/db, property: password}
```

50+ services without handing out Vault tokens:

- Every service gets its own **ServiceAccount**. Vault's **Kubernetes auth method** maps that SA (plus namespace) to a Vault role, which maps to a policy allowing only that service's paths.
- Secrets are delivered by the **Vault Agent Injector** sidecar (annotations) or the **Secrets Store CSI driver**. The app just reads a file, and never holds a long-lived or broad Vault token.
- Use **dynamic secrets** for databases (Vault creates a DB user per pod with a short TTL), so a leak expires quickly.
- An alternative is the **External Secrets Operator** with a SecretStore per namespace, if you prefer native K8s Secrets synced from Vault or AWS Secrets Manager.
- Turn on Vault audit logs, and template policies so onboarding service 51 is a PR, not a manual task.

```
metadata:
  annotations:
    vault.hashicorp.com/agent-inject: "true"
    vault.hashicorp.com/role: "meter-api"                      # bound to SA meter-api
    vault.hashicorp.com/agent-inject-secret-db: "database/creds/meter-api"   # dynamic
```

> ***What to say to the interviewer:*** *Native Secrets are just base64, so I enable KMS encryption at rest, restrict access with RBAC, and keep the real values in AWS Secrets Manager or Vault synced by External Secrets Operator or the CSI driver using IRSA. For GitOps I use Sealed Secrets or SOPS, and I rotate secrets and audit who accesses them.*

Q46. How do you manage secrets across multi-region Kubernetes clusters?

> **In simple words:** Keep one master copy of each key in a central safe that automatically makes copies in each city's branch vault, and let each city's workers fetch from their local vault.

Detailed answer

Design:

- **Single source of truth:** AWS Secrets Manager with **multi-region replica secrets** (primary in region A, read replicas in B/C; same name, replicated automatically, each encrypted with a regional KMS key). Alternatively Vault with performance replication.
- **Per-cluster sync:** each cluster runs **External Secrets Operator** with a `ClusterSecretStore` pointing to its **local region** endpoint, authenticated by **IRSA / Pod Identity** (per-cluster IAM role scoped to specific secret ARNs/paths like `prod/orders/*`).
- **Consistent naming:** `env/app/secret` so the same ExternalSecret manifest (via Helm/Argo CD ApplicationSet) works in every region.

- **Rotation:** rotate once at the primary (Lambda rotation); replicas update; ESO `refreshInterval` pulls new values; apps reload (Reloader or checksum annotation roll).
- **Failover:** if the primary region is down, replicas can be **promoted** to standalone.
- Audit with CloudTrail in each region; alert on access-denied spikes.

Example:

```
aws secretsmanager replicate-secret-to-regions --secret-id prod/orders/db \
  --add-replica-regions Region=ap-southeast-1 Region=eu-west-1

apiVersion: external-secrets.io/v1beta1
kind: ClusterSecretStore
metadata: {name: aws-sm}
spec:
  provider:
    aws:
      service: SecretsManager
      region: ap-southeast-1          # local region per cluster
      auth:
        jwt: {serviceAccountRef: {name: external-secrets, namespace: external-secrets}}
```

What to say to the interviewer: *I keep secrets in AWS Secrets Manager with multi-region replicas and run External Secrets Operator in every cluster, each reading from its local region using a scoped IRSA role. Rotation happens once at the primary, replicas propagate, ESO refreshes the Kubernetes Secret and Reloader restarts the pods; replicas can be promoted if the primary region fails.*

Q47. How do you handle database credential rotation in Kubernetes without downtime?

In simple words: Change the lock on a door while people are still using it: first make the new key work too, hand out new keys, then retire the old key.

Detailed answer

Steps:

1. Store credentials in **AWS Secrets Manager** (or Vault) - not hard-coded.
2. Enable **automatic rotation** (Secrets Manager rotation Lambda for RDS). Use the **alternating users** strategy (two DB users, rotate one while the other is active) or keep AWSPREVIOUS valid briefly, so existing connections do not break.
3. **Sync to K8s:** External Secrets Operator with a short `refreshInterval`, or Secrets Store CSI driver with rotation enabled.
4. **App picks up new value:**

- If mounted as a file, the app can re-read it on auth failure.
- If env var, pods must restart: use **Stakater Reloader** (restarts Deployment when Secret changes) or a checksum annotation in Helm -> rolling restart, zero downtime.

5. Connection pools must reconnect gracefully on auth errors.
6. Even better: **Vault dynamic secrets** (short-lived credentials generated per pod) or **RDS IAM authentication** (token instead of password).
7. Monitor: alert on DB auth failures after rotation.

Example:

```
aws secretsmanager rotate-secret --secret-id prod/orders/db --rotation-lambda-arn <arn> \
  --rotation-rules AutomaticallyAfterDays=30

metadata:
  annotations:
    secret.reloader.stakater.com/reload: "db-cred"    # Reloader rolls pods on change
```

> ***What to say to the interviewer:*** *Credentials live in Secrets Manager with automatic rotation using an alternating-users strategy, External Secrets Operator syncs the new value into Kubernetes and Reloader performs a rolling restart so pods pick it up with no downtime. Where possible I prefer RDS IAM auth or Vault dynamic secrets so there is no long-lived password at all.*

Q48. What is your strategy for backup and restore in a Kubernetes cluster? How do you handle disaster recovery for stateful apps on containers?

Also asked as: What is your approach to disaster recovery for stateful apps running on containers?

> **In simple words:** Back up two things: the building's blueprint (Kubernetes objects) and the contents of the vaults (persistent data). A backup only counts if you have practised rebuilding from it.

Detailed answer

What to back up:

- **Cluster objects:** ideally already in **Git** (GitOps) + Helm charts, so a cluster can be rebuilt with Terraform + Argo CD.
- **etcd snapshots** (self-managed clusters only; EKS/GKE manage etcd).
- **Persistent volumes / data:** EBS snapshots, application-level dumps.
- **Velero:** backs up namespaces/resources to S3 and volumes via snapshots or file-level (Kopia), on a schedule, with restore to the same or another cluster (also used for migrations).

Stateful DR strategy:

- Prefer **managed data services** (RDS Multi-AZ, cross-region read replica, automated backups/PITR) over DBs in pods.
- For DBs in K8s: app-consistent backups (`pg_dump`, `mongodump`, operator backups like CloudNativePG/Percona to S3), plus volume snapshots.
- Copy backups **cross-region/cross-account**, S3 versioning + Object Lock.
- Define **RPO/RTO**; choose backup & restore, pilot light, warm standby, or active-active accordingly.
- **Test restores regularly** (monthly restore drill into a scratch namespace, verify checksums / row counts).

Example:

```
velero install --provider aws --bucket k8s-backups --backup-location-config region=ap-south-1 \
  --snapshot-location-config region=ap-south-1 --plugins velero/velero-plugin-for-aws:v1.10.0
velero schedule create daily-prod --schedule="0 1 * * *" --include-namespaces prod --ttl 720h
velero backup get
velero restore create --from-backup daily-prod-20260920010000 --namespace-mappings prod:prod-restore
    -test

# self-managed etcd
ETCDCTL_API=3 etcdctl snapshot save /backup/etcd-$(date +%F).db ...
```

> ***What to say to the interviewer:*** *Manifests live in Git so the cluster can be rebuilt with Terraform and Argo CD, Velero takes scheduled backups of namespaces and volume snapshots to S3, and databases get application-consistent backups copied cross-region. I prefer managed databases for state, define RPO and RTO per service, and run regular restore drills because an untested backup is not a backup.*

Q49. How do you migrate data from one MongoDB pod to another? How do you manage stateful workloads in Kubernetes?

> **In simple words:** Moving house for a database: either copy the boxes while the house is closed (dump/restore), or build the new house as an extension that keeps itself in sync and then just move in (replica set).

Detailed answer

Stateful workloads need StatefulSets (stable identity), PVCs (persistent storage), headless Services (stable DNS), anti-affinity, PDBs, and ideally an **operator** (MongoDB Community/Percona Operator) that handles replication, failover, backups.

Migration options:

1. **Replica set method (near zero downtime - preferred):**

- Add the new pod (new StatefulSet / new cluster) as a **secondary** member: `rs.add("mongo-new-0.mongo-new:27017")`.
- Wait for initial sync (`rs.status()` shows SECONDARY, low lag).
- `rs.stepDown()` to make the new member primary, update app connection string (or keep it through the Service), then `rs.remove()` old members.

2. **mongodump / mongorestore (simple, needs write freeze):**

- Stop writes or accept a maintenance window, dump from old pod, restore into new pod, switch connection string.

3. **Volume-level:** snapshot the PVC (VolumeSnapshot / EBS snapshot) and create a new PVC from it for the new pod (same data, same engine version).

Always: take a backup first, verify counts/checksums after, keep the old copy until verified.

Example:

```
kubectl exec -it mongo-0 -n db -- mongosh --eval 'rs.add("mongo-new-0.mongo-new.db.svc:27017")'
kubectl exec -it mongo-0 -n db -- mongosh --eval 'rs.status().members.map(m=>[m.name,m.stateStr])'

# dump and restore
kubectl exec mongo-0 -n db -- mongodump --archive --gzip > dump.gz
kubectl exec -i mongo-new-0 -n db -- mongorestore --archive --gzip < dump.gz
```

What to say to the interviewer: *Stateful apps run as StatefulSets with PVCs, headless Services, anti-affinity and ideally an operator. To migrate MongoDB with near-zero downtime I add the new pod as a replica set secondary, wait for sync, step down the old primary and remove old members; for simpler cases I use mongodump and mongorestore in a maintenance window or restore from a PVC snapshot, always verifying data before deleting the old copy.*

Q50. Have you upgraded Kubernetes clusters? How do you upgrade a cluster (EKS) safely?

In simple words: Upgrade the brain first, then replace the workers one by one while the factory keeps running - and check the recipe book (APIs) is still valid for the new version.

Detailed answer

Steps for EKS (one minor version at a time, e.g. 1.29 -> 1.30):

1. **Prepare:** read release notes; find **deprecated/removed APIs** (`kubent`, `pluto`, EKS upgrade insights); check add-on and Helm chart compatibility (ingress controller, cert-manager, CSI, Karpenter).
2. **Test** in dev/staging first.
3. **Upgrade control plane** (Terraform `cluster_version` change or `eksctl upgrade cluster`). AWS does it in-place with no API downtime.
4. **Upgrade add-ons:** vpc-cni, CoreDNS, kube-proxy, EBS CSI to versions matching the new K8s.
5. **Upgrade nodes:** managed node group rolling update (new AMI) - it cordons and drains nodes respecting **PDBs**; or blue/green node groups (create new group, drain old); Karpenter drift replaces nodes.
6. **Validate:** nodes Ready, pods running, dashboards, error rates, smoke tests.

Rollback: control plane cannot be downgraded, so testing first and blue/green node groups (or a blue/green cluster for big jumps) is the safety net. Node versions may lag the control plane within the supported skew.

Example:

```
kubent                                  # find deprecated APIs
eksctl upgrade cluster --name prod-eks --version 1.30 --approve
aws eks update-addon --cluster-name prod-eks --addon-name coredns --addon-version <ver>
eksctl upgrade nodegroup --cluster prod-eks --name app-ng --kubernetes-version 1.30
kubectl get nodes -o wide     # VERSION column shows v1.30.x
```

What to say to the interviewer: *I upgrade one minor version at a time after scanning for deprecated APIs and testing in staging: control plane first, then add-ons like VPC CNI, CoreDNS and kube-proxy, then node groups with a rolling or blue-green node replacement that respects PDBs. Because the control plane cannot be downgraded, preparation and validation matter most.*

Q51. How do you perform zero-downtime deployments in Kubernetes?

Also asked as: You need to implement zero-downtime deployment for a service in Kubernetes. Walk me through your plan. / How do you design zero-downtime deployments for STATEFUL apps on Kubernetes?

In simple words: Open new checkout counters and only close the old ones after the new ones are actually serving customers - and let the old cashier finish the customer already at the counter.

Detailed answer

Checklist:

1. **Rolling update** with `maxUnavailable: 0`, `maxSurge: 1` (or 25%).
2. **Readiness probe** that truly reflects "can serve traffic"; `minReadySeconds` to catch instant crashes.
3. **Graceful shutdown:**

- App handles **SIGTERM** (stop accepting, finish in-flight requests).

- `preStop` hook `sleep 5-15` so load balancers/kube-proxy remove the pod from endpoints before it stops.

- `terminationGracePeriodSeconds` longer than the longest request.

4. **Enough replicas** (>=2) spread across nodes/AZs + **PDB** so drains do not take all replicas.
5. **Backward-compatible changes:** DB migrations in expand/contract style so old and new versions both work during rollout.
6. For AWS ALB ingress: use IP target mode, set deregistration delay, pod readiness gates.
7. **Progressive delivery** for risky releases: canary/blue-green with Argo Rollouts, automatic rollback on metrics.
8. Verify with `rollout status` + error-rate dashboards; `rollout undo` or `helm rollback` if needed.

Example:

```
spec:
  strategy: {type: RollingUpdate, rollingUpdate: {maxSurge: 1, maxUnavailable: 0}}
  minReadySeconds: 10
  template:
    spec:
      terminationGracePeriodSeconds: 45
      containers:
      - name: api
        readinessProbe: {httpGet: {path: /ready, port: 8080}, periodSeconds: 5}
        lifecycle:
          preStop: {exec: {command: ["sh", "-c", "sleep 10"]}}
---
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: api-pdb}
spec: {minAvailable: 2, selector: {matchLabels: {app: api}}}
```

Stateful apps (databases, Kafka, Redis, apps with sessions):

- **StatefulSet rolling updates** go one pod at a time in reverse order. Use `updateStrategy.rollingUpdate.partition` for a canary: only pods with an ordinal >= partition get the new version.

- **Readiness probes that mean "caught up":** for a replica, ready only when replication lag is low, not just when the port is open.

- **PodDisruptionBudget** (`maxUnavailable: 1`), so drains and upgrades never take down the quorum.

- **preStop hook + terminationGracePeriodSeconds** to hand over leadership or drain connections before SIGTERM kills the process.
- **Migration sequencing:** apply expand-only, backward-compatible schema changes first, roll the app, then contract in a later release.
- Prefer **operators** (CloudNativePG, Strimzi for Kafka, Redis operators) that handle switchover and failover properly.

```
apiVersion: apps/v1
kind: StatefulSet
spec:
  updateStrategy:
    type: RollingUpdate
    rollingUpdate: { partition: 2 }     # only pod-2 (and above) gets the new version first
  template:
    spec:
      terminationGracePeriodSeconds: 60
      containers:
        - name: db
          lifecycle:
            preStop: { exec: { command: ["/scripts/step-down-and-drain.sh"] } }
          readinessProbe:
            exec: { command: ["/scripts/replica-lag-ok.sh"] }
```

> ***What to say to the interviewer:*** *I use a rolling update with maxUnavailable 0, accurate readiness probes, a preStop sleep plus SIGTERM handling and enough grace period so in-flight requests finish, and PDBs with multiple replicas across zones. Database changes are backward compatible, risky releases go through canary with automatic rollback, and I watch rollout status and error rates to revert quickly.*

Q52. How would you do blue-green (and canary) deployments in a Kubernetes / EKS cluster?

Also asked as: How do you handle blue/green deployment for EKS? What would that look like in practice?

> **In simple words:** Blue-green is building a complete second stage behind the curtain, testing it, then turning the spotlight to it in one move - and turning it back instantly if something is wrong. Canary is letting a few audience members try the new show first.

Detailed answer

Blue-green with plain Kubernetes:

1. `app-blue` Deployment (v1, label `version: blue`) is live; Service selector points to `version: blue`.
2. Deploy `app-green` (v2, `version: green`) at full capacity alongside it.
3. Test green via a separate test Service/host (`green.internal`).
4. **Switch:** patch the Service selector to `version: green` - all traffic moves at once.
5. **Rollback:** patch selector back to `blue` (seconds). Keep blue running until confident, then scale it down.

Gradual traffic shifting (canary style):

- **Istio VirtualService weights:** 90/10 -> 50/50 -> 100/0 between subsets; rollback = set weights back to 100% old, no pod restarts.

- **NGINX Ingress** canary annotations (`canary-weight: "10"`) or **ALB weighted target groups**.
- **Argo Rollouts / Flagger** automate steps and run **analysis** (Prometheus error rate, latency) and auto-rollback.

Watch-outs: double capacity during blue-green (cost), DB schema must work for both versions, sticky sessions/long connections, and consumers (Kafka workers) should not both process in parallel unexpectedly.

Example:

```
kubectl patch svc app -p '{"spec":{"selector":{"app":"app","version":"green"}}}'
kubectl patch svc app -p '{"spec":{"selector":{"app":"app","version":"blue"}}}'   # rollback

apiVersion: networking.istio.io/v1beta1
kind: VirtualService
metadata: {name: app}
spec:
  hosts: [app]
  http:
  - route:
    - {destination: {host: app, subset: blue},  weight: 90}
    - {destination: {host: app, subset: green}, weight: 10}   # then 50, then 100
```

> ***What to say to the interviewer:*** *For blue-green I run blue and green Deployments side by side, test green on an internal route, then switch the Service selector or ingress target to green and roll back by switching it back. For gradual shifts I use Istio VirtualService weights such as 10, 50 then 100 percent, or Argo Rollouts with Prometheus analysis for automatic rollback, keeping database changes compatible with both versions.*

Q53. How would you set up an automated rollback strategy in Kubernetes for failed deployments?

Also asked as: Explain how you would handle a failed rollout during a deployment.

> **In simple words:** Give the deployment a safety net and a referee: if the new version does not become healthy in time, or the error rate goes up, the referee automatically puts the old version back.

Detailed answer

Layers of automation:

1. **Kubernetes-native detection:** `progressDeadlineSeconds` marks rollout `Failed` (ProgressDeadlineExceeded) if new pods do not become ready; readiness probes stop bad pods getting traffic; old ReplicaSet stays serving because `maxUnavailable: 0`.
2. **CI/CD gate:** `kubectl rollout status --timeout` or `helm upgrade --atomic --timeout 5m` -> on failure Helm **automatically rolls back** to the previous release and the pipeline fails.
3. **Metric-based rollback (progressive delivery): Argo Rollouts** or Flagger run canary steps with an `AnalysisTemplate` querying Prometheus (5xx rate, p99 latency); if the analysis fails, it aborts and returns traffic to stable automatically.
4. **GitOps:** Argo CD - revert the Git commit to roll back; the cluster follows.

5. Post-rollback: alert the team, keep the failed ReplicaSet/pods logs for RCA, block re-deploy until fixed.

Example:

```
helm upgrade --install api charts/api -n prod --set image.tag=$TAG --atomic --timeout 5m

apiVersion: argoproj.io/v1alpha1
kind: AnalysisTemplate
metadata: {name: error-rate}
spec:
  metrics:
  - name: error-rate
    interval: 1m
    failureLimit: 1
    successCondition: result[0] < 0.02
    provider:
      prometheus:
        address: http://prometheus.monitoring:9090
        query: |
          sum(rate(http_requests_total{app="api",status=~"5.."}[2m]))
          / sum(rate(http_requests_total{app="api"}[2m]))
```

> ***What to say to the interviewer:*** *I rely on progressDeadlineSeconds and readiness probes so a bad version never takes traffic, and deploy with helm upgrade --atomic so a failed rollout is reverted automatically by the pipeline. For metric-driven safety I use Argo Rollouts canaries with Prometheus analysis on error rate and latency, which abort and restore the stable version on their own.*

Q54. What are PodDisruptionBudgets and why do you need them?

> **In simple words:** A PDB is a rule for maintenance crews: "you may take workers off the line for repairs, but at least 2 must always stay working".

Detailed answer

A **PodDisruptionBudget** limits how many pods of an app can be down at the same time due to **voluntary disruptions**: node drains, cluster upgrades, Cluster Autoscaler/Karpenter consolidation, `kubectl drain`.

- `minAvailable: 2` or `maxUnavailable: 1` (number or %).
- The eviction API refuses evictions that would break the budget; the drain waits.
- It does **not** protect against involuntary disruptions (node crash, OOM kill).

Gotchas:

- A PDB with `maxUnavailable: 0` or `minAvailable` = replicas blocks drains forever -> node upgrades hang.
- Single-replica apps with a strict PDB block maintenance; either run 2+ replicas or allow 1 unavailable.

Example:

```
apiVersion: policy/v1
kind: PodDisruptionBudget
metadata: {name: api-pdb, namespace: prod}
spec:
  maxUnavailable: 1
  selector: {matchLabels: {app: api}}

kubectl get pdb -n prod
# NAME     MIN AVAILABLE  MAX UNAVAILABLE  ALLOWED DISRUPTIONS
# api-pdb  N/A            1                1
```

What to say to the interviewer: *A PodDisruptionBudget caps how many replicas can be evicted at once during voluntary disruptions like drains, upgrades or autoscaler consolidation. I set maxUnavailable 1 for most services with at least two replicas, and avoid budgets that allow zero disruptions because they block node upgrades.*

Q55. What does the pod status ContainerStatusUnknown mean, and when does it occur?

In simple words: The manager lost contact with a worker's walkie-talkie. He does not know if the worker is still working, has left, or has collapsed - so the status is "unknown".

Detailed answer

`ContainerStatusUnknown` means the kubelet **could not determine the container's state**, usually because the container disappeared and its status could not be retrieved (it shows with reason "The container could not be located when the pod was terminated", exit code 137).

Common causes:

- **Node problems:** node became NotReady / lost contact with control plane, kubelet restarted or crashed.
- **Eviction under resource pressure** (disk/memory) where the container was killed and cleaned up.
- **Container runtime issues** (containerd restarted, runtime state lost).
- Node was terminated (Spot interruption, scale-down) while pods were running.

What to do:

1. `kubectl describe pod` and `kubectl get events` - look for Evicted, NodeNotReady, OOM.
2. Check node: `kubectl describe node` conditions (MemoryPressure, DiskPressure), kubelet/containerd logs.
3. Usually the controller has already created a replacement; delete the stale pod (`kubectl delete pod`).
4. Fix root cause: resource requests/limits, disk cleanup, node health, Spot handling.

Example:

```
kubectl get pods -A | grep ContainerStatusUnknown
kubectl describe pod <pod> | sed -n '/State/,/Events/p'
kubectl get pods -A --field-selector=status.phase=Failed -o name | xargs kubectl delete
journalctl -u kubelet --since "1 hour ago"      # on the node
```

What to say to the interviewer: *ContainerStatusUnknown means kubelet lost track of the container's state, typically after node pressure evictions, a kubelet or containerd restart, or a node going away. I check pod events and node conditions, clean up the stale pod since the controller usually replaced it, and fix the underlying node or resource issue.*

Q56. How would you configure readiness/liveness probes to catch buffering or lag issues in stream-processing microservices before users notice?

In simple words: Do not just ask the worker "are you breathing?" - ask "are you keeping up with the conveyor belt?" If he is falling behind, stop giving him new boxes before the pile hits customers.

Detailed answer

Plain "process is up" checks miss slowness. Design:

- **Readiness = "can I serve well right now?":** expose `/ready` that fails if internal lag exceeds a threshold, e.g. consumer lag > N messages, internal buffer/queue depth > X%, segment processing time > target, dependency (Kafka/Redis) unreachable. Failing readiness removes the pod from the load balancer so users go to healthy pods; it is **not restarted**.
- **Liveness = "am I stuck?":** a **watchdog** timestamp updated by the processing loop; fail only if no progress for, say, 60s (deadlock). Do **not** tie liveness to lag, or a traffic spike restarts all pods and makes it worse.
- **Startup probe** covers warm-up (loading codecs, caches) so pods are not killed at boot.
- Tuned timing: `periodSeconds: 5`, `failureThreshold: 2-3` for readiness (fast reaction), longer for liveness.
- Probes are only a safety net: also export **metrics** (lag, buffer ratio, rebuffer rate, p99 latency) to Prometheus with alerts and HPA/KEDA scaling on lag, plus synthetic playback checks.

Example:

```
readinessProbe:
  httpGet: {path: /ready, port: 8080}     # 503 if lag > 2s or buffer > 80%
  periodSeconds: 5
  failureThreshold: 2
  timeoutSeconds: 1
livenessProbe:
  httpGet: {path: /live, port: 8080}      # 500 only if loop idle > 60s
  periodSeconds: 10
  failureThreshold: 6
startupProbe:
  httpGet: {path: /live, port: 8080}
  periodSeconds: 5
  failureThreshold: 24
```

What to say to the interviewer: *I make readiness reflect quality of service, failing when processing lag or buffer depth crosses a threshold so the pod is taken out of rotation, while liveness only detects a stuck processing loop via a heartbeat so load spikes do not trigger restart storms. A startup probe covers warm-up, and lag metrics also drive alerts and KEDA scaling.*

Q57. How does data flow in a Kubernetes cluster? Explain the Kubernetes networking model.

In simple words: Every pod gets its own phone number and can call any other pod directly, with no switchboard operator changing numbers in between. Services are shared hotline numbers that forward to whichever pod is free, and the Ingress is the reception desk for calls from outside.

Detailed answer

The networking model (rules every CNI must follow):

- Every pod gets its **own IP**; containers in a pod share it (talk over `localhost`).
- **Pod-to-pod without NAT**, across all nodes: the IP a pod sees for itself is the IP others use.
- Nodes can reach all pods and vice versa.

How packets move:

1. **Same node:** pod eth0 -> **veth pair** -> node bridge / routing table -> other pod's veth. Never leaves the node.
2. **Across nodes:** handled by the **CNI plugin**:

- **Overlay** (Flannel VXLAN, Calico IPIP/VXLAN): packet is encapsulated with the node IPs, sent across, decapsulated. Pod IPs come from a separate range.
- **Native routing** (AWS VPC CNI, Azure CNI, GKE VPC-native): pods get **real VPC IPs** on node ENIs, so the VPC routes them directly - no encapsulation, but subnets must have enough IPs.

3. **Pod -> Service (east-west):** app resolves `svc.ns` via **CoreDNS** -> gets the ClusterIP (a virtual IP that exists on no interface) -> **kube-proxy** rules (iptables or IPVS) on the source node **DNAT** it to a Ready endpoint pod IP -> normal pod-to-pod path. **Cilium** can replace kube-proxy with eBPF.
4. **External -> app (north-south):** client -> DNS (Route 53) -> cloud load balancer (ALB/NLB) -> ingress controller pods (or directly to pod IPs in IP target mode) -> Service -> pod.
5. **Pod -> internet (egress):** pod -> node -> SNAT to node IP -> NAT Gateway -> internet.

Example:

```
                Client
                  |  DNS: app.example.com -> ALB
                  v
          +------------------+
          |  Cloud LB (ALB)  |
          +--------+---------+
                   v
          +------------------+      Ingress rules: host/path
          | Ingress ctrl pod |
          +--------+---------+
                   |  Service "api" ClusterIP 10.100.5.20
                   |  kube-proxy DNAT -> endpoint pod IP
      +------------+-------------+
      v                          v
 Node A                      Node B
 [api pod 10.0.1.15]         [api pod 10.0.2.33]
   | veth                        | veth
 [bridge/routes]             [bridge/routes]
   +------- CNI (VPC routing or VXLAN overlay) -------+
 api pod -> "db.data" -> CoreDNS -> ClusterIP -> db pod IP

kubectl get pods -o wide              # pod IPs and nodes
kubectl get endpointslices -l kubernetes.io/service-name=api
sudo iptables -t nat -L KUBE-SERVICES | grep api      # on a node (iptables mode)
```

For troubleshooting along this path, see the scenario questions on Ingress, request-flow 503s and pod-to-pod connectivity.

> ***What to say to the interviewer:*** *Kubernetes uses a flat network where every pod has its own IP and can reach any other pod without NAT; same-node traffic goes over veth pairs and the bridge, while cross-node traffic is carried by the CNI either as an overlay like VXLAN or with native VPC IPs as in the AWS VPC CNI. Services are virtual IPs that kube-proxy DNATs to Ready pod endpoints, CoreDNS maps names to them, and external traffic flows DNS, load balancer, ingress controller, Service, pod.*

Q58. What happens if a node with local storage gets scaled down by the autoscaler?

> **In simple words:** If you keep your files in a hotel room's drawer (local disk) and the hotel removes that room to save money, the files are gone. Keep important things in a bank locker (network storage) that you can take to any room.

Detailed answer

- Data in **emptyDir**, **hostPath** or **local PersistentVolumes** lives on that node's disk. When the node is removed, **that data is lost**. A pod using a local PV also cannot move to another node (the PV is tied to that node).
- **Cluster Autoscaler** by default **skips scaling down** nodes that run pods with local storage (emptyDir/hostPath), unless the pod is annotated `cluster-autoscaler.kubernetes.io/safe-to-evict: "true"` (or the flag `--skip-nodes-with-local-storage=false` is set). Pods without a controller also block scale-down.
- **Karpenter** consolidation **does** drain and replace nodes; it respects PDBs and the pod annotation `karpenter.sh/do-not-disrupt: "true"`, which blocks voluntary disruption.

- Spot interruptions and node failures ignore all of this - the node just disappears.

How to design for it:

- Keep durable data on **network volumes** (EBS via PVC for single-writer, EFS for shared) with **StatefulSets**; the volume reattaches to the pod on another node (same AZ for EBS).
- Use emptyDir only for **scratch/cache** data that can be rebuilt.
- **PDBs** to limit simultaneous evictions; graceful shutdown to flush data.
- **Backups** (Velero, app-level) for anything important.
- If you must use local NVMe (high-performance DBs), rely on app-level replication (Kafka, Cassandra) and use the do-not-disrupt / safe-to-evict annotations deliberately.

Example:

```
metadata:
  annotations:
    karpenter.sh/do-not-disrupt: "true"                        # Karpenter
    cluster-autoscaler.kubernetes.io/safe-to-evict: "false"   # Cluster Autoscaler
spec:
  volumes:
  - name: scratch
    emptyDir: {sizeLimit: 2Gi}        # lost when pod/node goes away
```

> ***What to say to the interviewer:*** *Local data in emptyDir, hostPath or local PVs is lost when the node is removed; Cluster Autoscaler skips such nodes by default unless pods are marked safe-to-evict, while Karpenter consolidation drains them unless pods carry the do-not-disrupt annotation. I keep durable data on EBS or EFS through StatefulSets and PVCs, use local disk only for rebuildable scratch data, and add PDBs and backups.*

Q59. How do you enforce runtime security in Kubernetes?

> **In simple words:** Security at the gate (admission) stops dangerous tenants from moving in; security guards inside (runtime detection) watch for someone doing something suspicious after they have moved in.

Detailed answer

1. Before the pod runs (admission):

- **PodSecurityPolicy was removed in Kubernetes 1.25.** The built-in replacement is **Pod Security Admission** enforcing the **Pod Security Standards** (privileged / baseline / restricted) per namespace via labels.
- **OPA Gatekeeper or Kyverno** for custom policies: only images from our ECR, no `latest` tag, must have limits, no hostPath, required labels.
- **Image signing verification** (cosign/Sigstore with Kyverno or Gatekeeper, AWS Signer + Ratify) so only images built by our pipeline run; vulnerability scanning in CI and in the registry.

2. How the container runs (hardening via securityContext):

- `runAsNonRoot`, fixed non-zero UID, `allowPrivilegeEscalation: false`
- `readOnlyRootFilesystem: true` (writable emptyDir only where needed)
- `capabilities: drop: ["ALL"]`, no `privileged`, no hostNetwork/hostPID
- `seccompProfile: RuntimeDefault`; **AppArmor/SELinux** profiles

- Minimal/distroless images; no service account token unless needed.

3. While it runs (detection and response):

- **Falco** (eBPF-based rules: shell spawned in container, write to /etc, unexpected network connection) or **GuardDuty EKS Runtime Monitoring**, Sysdig, Aqua.
- Alerts to on-call; automated response (kill pod, isolate with NetworkPolicy).
- Audit logs + NetworkPolicies to limit lateral movement.

Example:

```
kubectl label ns prod pod-security.kubernetes.io/enforce=restricted \
  pod-security.kubernetes.io/warn=restricted

apiVersion: kyverno.io/v1
kind: ClusterPolicy
metadata: {name: disallow-latest-tag}
spec:
  validationFailureAction: Enforce
  rules:
  - name: require-pinned-tag
    match: {any: [{resources: {kinds: [Pod]}}]}
    validate:
      message: "Image tag latest is not allowed"
      pattern: {spec: {containers: [{image: "!*:latest"}]}}

# Falco rule idea: alert on "Terminal shell in container"
```

What to say to the interviewer: *PodSecurityPolicy was removed in 1.25, so I enforce the restricted Pod Security Standard with Pod Security Admission, plus Gatekeeper or Kyverno for custom rules and signed image verification. Containers run non-root with read-only filesystems, all capabilities dropped and seccomp RuntimeDefault, and at runtime Falco or GuardDuty EKS Runtime Monitoring detects suspicious behaviour like shells in containers.*

Q60. HPA vs VPA vs Karpenter: when would you NOT use each, and how would you simulate HPA behaviour in staging?

In simple words: HPA hires more workers, VPA gives each worker a bigger desk, Karpenter builds more office floors. Each is wrong for some jobs - you do not hire ten managers for a one-manager role, and you do not rebuild the office every hour.

Detailed answer

Tool	What it does	When NOT to use it
HPA	Changes number of pods	Singletons or leader-based/stateful apps that cannot safely run multiple copies; when the bottleneck is elsewhere (DB), so more pods just add load; when CPU is a poor signal (use KEDA or custom metrics)
VPA	Changes CPU/memory requests (recreates pods to apply)	Spiky stateless services needing instant reaction (VPA is slow and restarts pods); together with HPA on the same CPU/memory metric (they fight); restart-sensitive apps. Use `updateMode: "Off"` (recommend only) instead
Karpenter	Adds/removes nodes for pending pods, consolidates	It does not scale pods at all; very steady workloads where consolidation churn (draining nodes) adds disruption without savings; strict stateful/local-storage pods (use do-not-disrupt)

A good combination: **HPA** on replicas, **VPA in recommendation mode** to right-size requests, **Karpenter** for nodes.

Simulate HPA in staging:

1. Same HPA spec and same requests/limits as prod (HPA math depends on requests).
2. Generate load with **k6, hey or Locust** (ramp-up, spike and soak patterns).
3. Watch `kubectl get hpa -w`, `kubectl describe hpa` events, pod counts, latency and node scale-up time.
4. Tune the `behavior` block (scale-up speed, stabilization windows) and repeat; record time-to-scale.

Example:

```
hey -z 5m -c 200 http://api.staging.svc.cluster.local/orders
k6 run --vus 500 --duration 10m spike-test.js
kubectl get hpa api -n staging -w
# NAME  REFERENCE      TARGETS   MINPODS MAXPODS REPLICAS
# api   Deployment/api 180%/70%  3       20      3 -> 8 -> 14

apiVersion: autoscaling.k8s.io/v1
kind: VerticalPodAutoscaler
metadata: {name: api-vpa}
spec:
  targetRef: {apiVersion: apps/v1, kind: Deployment, name: api}
  updatePolicy: {updateMode: "Off"}      # recommendations only
```

> ***What to say to the interviewer:*** *I avoid HPA for singletons or stateful leaders, avoid VPA for spiky services and never combine HPA and VPA on the same CPU or memory metric, and I remember Karpenter only scales nodes and can add churn for steady workloads. My usual combination is HPA for replicas, VPA in recommendation-only mode for right-sizing and Karpenter for nodes, and I validate HPA in staging with k6 or hey while watching kubectl get hpa -w and tuning the behavior block.*

Q61. How would you implement autoscaling for a multi-tenant workload, including a strategy for tenant isolation?

Also asked as: How do you isolate traffic between environments (dev/test/prod) in a shared AKS cluster?

> **In simple words:** Many families share one apartment building. Each family gets its own flat (namespace) with a fixed electricity quota, VIP families can get their own floor, and the building adds floors when needed - but no family may use up everyone else's electricity.

Detailed answer

Isolation (so one tenant cannot hurt others):

- **Namespace per tenant** (or per environment) with RBAC.
- **ResourceQuota** caps total CPU/memory/pods per tenant - this also implicitly caps how far that tenant's HPAs can scale. **LimitRange** sets default requests/limits so nothing runs BestEffort.
- **PriorityClasses**: premium tenants and system components preempt lower tiers under pressure.
- **Dedicated node pools** for noisy or premium tenants: taints + tolerations and nodeSelector/affinity; **Karpenter NodePools per tenant tier** (for example `premium` On-Demand, `standard` Spot) with their own limits.
- **NetworkPolicy** default-deny between tenant namespaces.
- **Hard isolation** when required (compliance, untrusted code): **vCluster** (virtual control plane per tenant) or **cluster per tenant**.

Environments in a shared AKS cluster: default-deny NetworkPolicy per environment namespace (Azure Network Policy, Calico or Cilium), separate ingress classes/controllers per environment, separate node pools with taints per environment, and Azure RBAC for Kubernetes scoped per namespace. Real advice: **prod should be its own cluster** (blast radius, upgrades, compliance); share only dev/test.

Scaling:

- **HPA per tenant Deployment** on CPU or per-tenant custom metrics (requests per second with a `tenant` label via prometheus-adapter).
- **KEDA** for queue-based tenants (per-tenant queue depth / Kafka lag).
- **VPA in recommend-only mode** to right-size requests; do not mix HPA and VPA on the same metric.
- Cluster level: **Karpenter / Cluster Autoscaler** adds nodes in the right pool.
- **Fairness:** quotas + max replicas per tenant + priority so a single tenant's spike cannot starve others; per-tenant rate limiting at the ingress/API gateway.
- Observe per-tenant usage and cost (Kubecost labels) for chargeback.

Example:

```
apiVersion: v1
kind: ResourceQuota
metadata: {name: tenant-a-quota, namespace: tenant-a}
spec:
  hard: {requests.cpu: "20", requests.memory: 40Gi, limits.memory: 60Gi, pods: "100"}
---
apiVersion: karpenter.sh/v1
kind: NodePool
metadata: {name: premium}
spec:
  template:
    spec:
      taints: [{key: tier, value: premium, effect: NoSchedule}]
      requirements:
      - {key: karpenter.sh/capacity-type, operator: In, values: [on-demand]}
      nodeClassRef: {group: karpenter.k8s.aws, kind: EC2NodeClass, name: default}
  limits: {cpu: "200"}
---
# tenant-a pods (premium)
tolerations: [{key: tier, operator: Equal, value: premium, effect: NoSchedule}]
nodeSelector: {karpenter.sh/nodepool: premium}
priorityClassName: tenant-premium
```

What to say to the interviewer: *I isolate tenants with a namespace each, ResourceQuota and LimitRange, PriorityClasses, default-deny NetworkPolicies and dedicated tainted node pools or Karpenter NodePools per tier, moving to vCluster or a cluster per tenant when hard isolation is required, and for environments I keep prod in its own cluster. Each tenant scales with its own HPA or KEDA trigger on per-tenant metrics, VPA runs in recommendation mode, Karpenter adds nodes, and quotas plus max replicas keep one tenant from starving the others.*

Scenario-Based

Q62. What common Kubernetes errors have you faced, and how did you fix them?

In simple words: Every pod error name is a clue that tells you which stage of the pod's life went wrong: could not be placed, could not get its image, could not start, or started and then died.

Detailed answer

Error / status	What it means	First checks	Typical fixes
Pending	Scheduler cannot place the pod	`kubectl describe pod` events: Insufficient cpu/memory, taints, node affinity, unbound PVC, pod limit per node	Lower requests, add nodes (autoscaler), fix selectors/tolerations, fix StorageClass/AZ
ImagePullBackOff / ErrImagePull	Node cannot pull the image	Events: not found, unauthorized, timeout	Fix tag/repo name, ECR permissions on node role, imagePullSecrets, NAT/VPC endpoints for ECR, rate limits (Docker Hub)
CreateContainerConfigError	Pod spec references something missing	Events: `secret "db-cred" not found`, `configmap ... not found`, missing key	Create the Secret/ConfigMap/key, fix names, check namespace
CrashLoopBackOff	Container starts then keeps exiting	`kubectl logs --previous`, exit code, describe	Fix app config/env, dependencies, command/args, probes
OOMKilled (exit 137)	Container exceeded memory limit	describe -> Last State: OOMKilled; memory graph	Raise memory limit, fix leak, tune JVM/heap (`-XX:MaxRAMPercentage`)
Error / exit 1	App crashed with an error	logs --previous	Fix code/config
RunContainerError / exec format error	Wrong command or wrong CPU architecture	logs, image arch	Fix entrypoint, build multi-arch (amd64/arm64)
Evicted	Node under memory/disk pressure	node conditions, `kubectl get pods --field-selector=status.phase=Failed`	Set requests, clean disk, bigger nodes, log rotation
Terminating forever	Finalizers or unreachable node	describe, finalizers	Fix node, remove stuck finalizer carefully, `--force --grace-period=0` last resort

Example:

```
kubectl get pods -A | grep -vE "Running|Completed"
kubectl describe pod <pod> -n <ns> | sed -n '/Events/,$p'
kubectl logs <pod> -n <ns> --previous
kubectl get events -n <ns> --sort-by=.lastTimestamp | tail -20
```

> ***What to say to the interviewer:*** *The status tells me which stage failed: Pending is scheduling, ImagePullBackOff is registry access or tag, CreateContainerConfigError is a missing Secret or ConfigMap, CrashLoopBackOff is the app exiting, and OOMKilled is memory limit. I always start with describe pod events and logs --previous, then fix the specific cause such as requests, IAM pull permissions, missing config or memory limits.*

Q63. Rapid-fire: What happens if...?

In simple words: Knowing how Kubernetes behaves when each piece breaks lets you guess the cause from the symptom in seconds.

Detailed answer

Situation	What you see	What to do
A pod keeps crashing repeatedly	CrashLoopBackOff; kubelet restarts it with exponential back-off 10s, 20s, 40s... capped at 5 minutes (reset after 10 min of healthy running)	`logs --previous`, exit code, describe events; fix config/dependency/probe
CoreDNS fails	Pods and Services look healthy but apps get "could not resolve host"; calls by IP still work	Check kube-system CoreDNS pods/logs, scale it, check NetworkPolicy on port 53, NodeLocal DNSCache
A node runs out of memory	Node condition MemoryPressure, new pods not scheduled there, kubelet evicts pods (BestEffort first, then Burstable over their requests); containers over their own limit get OOMKilled	Set proper requests/limits, fix leaks, bigger/more nodes, keep system reserved memory
Service selector matches no pods	Service exists but `kubectl get endpoints` is empty; clients get connection refused or timeout	Fix label/selector mismatch or make pods Ready; check targetPort
An unready app gets traffic	No readiness probe, so pod is added to endpoints as soon as the container starts; users get errors during startup/deploys	Add readiness (and startup) probes
etcd fails / loses quorum	API server errors, kubectl fails, nothing can be created, scaled or healed; **already running pods keep running** and serve traffic	Restore quorum (fix/replace member) or restore from snapshot; on EKS/GKE the provider handles etcd
Rolling deploy with bad config	With `maxUnavailable: 0`, new pods fail readiness, old pods keep serving; rollout stalls and after `progressDeadlineSeconds` (default 600s) is marked failed; no probe = bad pods take traffic	`kubectl rollout undo`, fix config; always use readiness probes
Kubelet loses contact with API server	Node goes NotReady/Unknown after the grace period (~40-50s); node is tainted; after the default toleration of 300s (5 min) pods are evicted and Deployments recreate them elsewhere; containers may still run on the isolated node	Fix kubelet/network; StatefulSet pods need node confirmed dead before replacement
HPA scales too aggressively	Replica count spikes, costs jump, node autoscaler adds many nodes, possibly hits ResourceQuota (pods fail to create)	Tune `behavior.scaleUp` policies, `stabilizationWindowSeconds`, sane `maxReplicas`, better metric
Container exits 0 but app is broken	Deployment (restartPolicy Always) restarts it anyway -> may still loop as CrashLoopBackOff "Completed"; if process stays up but app is broken, nothing happens without probes	Liveness probe for hung app, readiness for broken dependency; check why main process exits

Example:

```
kubectl get endpoints <svc>                  # selector problem?
kubectl describe node <node> | grep -A8 Conditions
kubectl get deploy <app> -o jsonpath='{.status.conditions[?(@.type=="Progressing")]}'
kubectl describe hpa <app>
```

> ***What to say to the interviewer:*** *I match symptoms to components: empty endpoints means a selector or readiness problem, name resolution failures with healthy pods point to CoreDNS, an etcd failure freezes the API but does not kill running pods, and a node losing its API connection has its pods evicted after about five minutes. Readiness probes plus maxUnavailable 0 are what save a bad rolling deploy.*

Q64. A pod is stuck in CrashLoopBackOff even though the image is valid. How do you debug and fix it?

Also asked as: Your pod is stuck in CrashLoopBackOff, what is your next step? Half your pods are in CrashLoopBackOff and the team says it works on my machine - you are on call, what do you do live?

> **In simple words:** The car (image) is fine, but it keeps stalling as soon as it starts - maybe no fuel (missing secret), road blocked (database unreachable), or the inspector (liveness probe) keeps switching it off too early.

Detailed answer

CrashLoopBackOff = the container starts, exits, and kubelet keeps restarting it with increasing delay (up to 5 min). The image pulls fine, so the problem is **runtime**.

Live triage order:

1. **Scope first:** which pods/nodes, since when, all replicas or some? Only new ReplicaSet pods -> recent deploy. Only one node -> node issue. Half the pods -> maybe the new version during a rollout, or pods on a specific node group/arch.
2. **What changed?** `kubectl rollout history`, recent Helm/Argo CD sync, ConfigMap/Secret edits.
3. **Logs of the crashed attempt:** `kubectl logs <pod> --previous` (current logs are often empty).
4. **Exit code and reason:** `kubectl describe pod` -> Last State. 1 = app error, 137 = OOMKilled/SIGKILL, 139 = segfault, 143 = SIGTERM, 127 = command not found.
5. **Config/env:** missing `DB_PASSWORD` env from Secret, wrong key name, wrong DB host (localhost vs Service name) - compare with the working environment (`kubectl exec env`, diff values files).
6. **Dependencies:** DB/Kafka reachable? Test from a debug pod (`nc -zv postgres 5432`); NetworkPolicy, SG.
7. **Probes:** liveness too aggressive (short timeout, no startup probe) kills a slow-starting app - events show "Liveness probe failed".
8. **Resources:** OOMKilled -> raise memory limit or fix heap.
9. **Platform differences:** image built for arm64 but nodes amd64 (`exec format error`), `readOnlyRootFilesystem`/non-root breaking writes, missing volume mounts.
10. **Mitigate first if prod is hurting:** `kubectl rollout undo`, then fix calmly.

Example:

```
kubectl get pods -o wide -l app=api           # which nodes, which ReplicaSet hash
kubectl logs api-6f9c-abcde --previous
kubectl describe pod api-6f9c-abcde | grep -A6 "Last State"
#   Last State: Terminated  Reason: Error  Exit Code: 1
kubectl exec -it api-6f9c-xyz -- env | grep DB_  # compare with healthy pod
kubectl get secret db-cred -o jsonpath='{.data}' | head -c 200
kubectl debug -it api-6f9c-abcde --image=nicolaka/netshoot --target=api
kubectl rollout undo deploy/api
```

What to say to the interviewer: *I scope the problem first, check what changed, and read kubectl logs --previous and the exit code from describe pod. With a valid image the usual culprits are missing env or secrets like DB_PASSWORD, an unreachable database, an over-aggressive liveness probe, OOMKilled or an architecture mismatch; if users are affected I roll back first with kubectl rollout undo and then fix the root cause.*

Q65. A containerized app works on a developer machine (or locally) but fails or crashes in Kubernetes / production. What is your debug checklist?

In simple words: The dish tastes fine in your home kitchen but not in the restaurant. Same recipe, different kitchen: different ingredients (config), different stove (CPU architecture), smaller pots (memory limits), stricter health inspector (probes, security rules).

Detailed answer

Checklist - compare local vs cluster:

1. **Same image?** Local may use a locally-built or `latest` tag; cluster pulls a different digest. Pin tags/digests.
2. **Env and config:** missing env vars, Secrets, ConfigMaps; `.env` file exists only on the laptop.
3. **Networking/DNS:** app connects to `localhost:5432` locally; in K8s it must be `postgres.db.svc.cluster.local`. NetworkPolicies, SGs, egress to external APIs.
4. **Resources:** laptop has 16 GB, pod limit 256Mi -> OOMKilled; CPU limit -> slow startup -> liveness kills it.
5. **Probes:** wrong path/port, too short timeouts.
6. **Security context:** runs as non-root / read-only root filesystem in K8s; app tries to write to `/app/tmp` -> permission denied. Mount an `emptyDir`.
7. **Architecture:** built on Apple Silicon (arm64), nodes are amd64 -> `exec format error`. Build multi-arch with buildx.
8. **Filesystem/volumes:** files expected in a path that is not mounted; case-sensitive paths.
9. **Time/locale, certificates:** missing CA certs in slim image for HTTPS calls.
10. **Signals:** app ignores SIGTERM or runs as a shell script child, so it gets killed on each rollout.

Example:

```
kubectl logs <pod> --previous
kubectl describe pod <pod>                    # OOMKilled? probe failures? mount errors?
kubectl get pod <pod> -o jsonpath='{.spec.containers[0].image}'
docker manifest inspect myrepo/app:1.2 | grep architecture
kubectl exec -it <pod> -- sh -c 'id; env | sort; ls -ld /app'
docker buildx build --platform linux/amd64,linux/arm64 -t myrepo/app:1.2 --push .
```

What to say to the interviewer: *I diff the two environments: exact image digest, env vars and secrets, service DNS names instead of localhost, memory and CPU limits, probes, security context like non-root and read-only filesystems, and CPU architecture. logs --previous and describe pod usually point to one of these within minutes.*

Q66. A deployment rollout is stuck and new pods are not becoming ready. What steps do you follow?

In simple words: New workers were hired but they never pass the "ready to work" check, so the manager will not let the old workers go home - the handover is frozen.

Detailed answer

1. **See the rollout state:** `kubectl rollout status` and Deployment conditions (`ProgressDeadlineExceeded`?).
2. **Which pods and what state:** new ReplicaSet pods Pending, CrashLoopBackOff, or Running but not Ready?
3. **Pending:** insufficient CPU/memory (maxSurge needs extra capacity), node selector/taints, PVC not bound, ResourceQuota exceeded (check ReplicaSet events: `exceeded quota`).
4. **Running but 0/1 Ready:** readiness probe failing - wrong path/port, app not listening, dependency down, probe timeout too short. `kubectl describe` shows `Readiness probe failed: HTTP probe failed with statuscode: 503`.
5. **Crashing:** logs --previous (see CrashLoopBackOff answer).
6. **Image:** ImagePullBackOff - wrong tag.
7. **Is the Deployment paused?** `kubectl rollout resume`.
8. **Decide:** if production impact or deadline -> `kubectl rollout undo` (or `helm rollback`). Old pods are still serving if `maxUnavailable: 0`.
9. **Prevent:** `progressDeadlineSeconds`, Helm `--atomic`, test probes in staging, canary.

Example:

```
kubectl rollout status deploy/api --timeout=120s
kubectl get rs -l app=api                        # new RS: DESIRED 1, READY 0
kubectl get pods -l app=api
kubectl describe pod <new-pod> | sed -n '/Conditions/,$p'
kubectl describe rs <new-rs> | tail              # quota / admission errors
kubectl rollout undo deploy/api
kubectl rollout history deploy/api
```

What to say to the interviewer: *I check rollout status and the new ReplicaSet's pods: Pending points to capacity, quota or scheduling constraints, Running but not Ready points to the readiness probe or a dependency, and crashes send me to logs --previous. If production is impacted I roll back immediately, since old pods are still serving, and then fix the probe, config or capacity issue.*

Q67. A kubectl rollout restart did not trigger new pods as expected. What could be wrong?

In simple words: You pressed the restart button, but either you pressed it on the wrong machine, the machine was paused, or the new parts could not be installed, so the old ones stayed.

Detailed answer

`kubectl rollout restart` works by adding/updating the annotation `kubectl.kubernetes.io/restartedAt` in the Pod template, which triggers a normal rolling update. Possible causes:

1. **Wrong target:** wrong namespace/context/cluster, or wrong resource name (restarted a different Deployment).
2. **Deployment is paused** (`spec.paused: true`) - template changes are recorded but not rolled out.
3. **New pods cannot become Ready/scheduled** - they are Pending (no capacity, quota) or failing readiness; with `maxUnavailable: 0` the old pods remain, so it looks like nothing happened.
4. **GitOps/Helm overwrote it:** Argo CD with self-heal reverts the annotation immediately (drift), or an operator manages the Deployment and resets the template.
5. **Resource is not a Deployment/StatefulSet/DaemonSet** (plain pods or a Job cannot be rolled out-restarted).
6. **StatefulSet with `OnDelete` update strategy** or a `partition` set - pods are not replaced automatically.
7. **Old kubectl version** (restart was added in 1.15) or RBAC denies `patch`.
8. Pods were restarted but you are looking at pod age of other pods, or the rollout is just slow (large `minReadySeconds`).

Example:

```
kubectl config current-context; kubectl get deploy api -n prod
kubectl rollout restart deploy/api -n prod
kubectl get deploy api -n prod -o jsonpath='{.spec.paused}{"\n"}'
kubectl get deploy api -n prod -o jsonpath='{.spec.template.metadata.annotations}'
kubectl rollout status deploy/api -n prod
kubectl get rs -n prod -l app=api
kubectl get sts db -o jsonpath='{.spec.updateStrategy}'
```

What to say to the interviewer: *rollout restart just patches a restartedAt annotation into the pod template, so I check whether I hit the right context and namespace, whether the Deployment is paused, whether new pods are stuck Pending or not Ready, and whether Argo CD self-heal or an operator reverted the change. For StatefulSets I also check for an OnDelete strategy or partition.*

Q68. A pod is failing because of a ConfigMap change. How would you roll back and prevent this next time?

In simple words: Someone changed the recipe card on the kitchen wall and now every new dish is wrong. Put the old card back quickly, then make a rule that recipe changes are tested and versioned like everything else.

Detailed answer

Roll back now:

1. Identify the change: `kubectl describe cm`, Git history / Argo CD diff, audit logs.
2. **Restore the previous ConfigMap** (from Git: revert the commit and let Argo CD sync, or `kubectl apply -f` the old version, or `helm rollback` the release that contains it).
3. **Restart pods** so they pick it up (env vars are read only at start): `kubectl rollout restart deploy/<app>`. Note: `kubectl rollout undo` alone does **not** restore ConfigMap content, because ConfigMaps are not part of Deployment revisions.
4. Verify pods Ready and errors gone.

Prevent:

- Manage ConfigMaps in **Git + Helm/Argo CD**, reviewed PRs, no `kubectl edit` in prod.
- **Immutable, versioned ConfigMaps:** name includes a hash (`app-config-7f9c2`, Kustomize `configMapGenerator`), so a change creates a new ConfigMap and a new Deployment revision -> `rollout undo` then really rolls back config too.
- Helm **checksum annotation** so config changes trigger a proper rolling update gated by readiness probes (bad config stops at the first pod instead of silently breaking all pods on next restart).
- Validate config in CI (schema, `helm template`, app `--validate-config`), test in staging, canary.
- `immutable: true` on ConfigMaps to prevent in-place edits.

Example:

```
# templates/deployment.yaml
spec:
  template:
    metadata:
      annotations:
        checksum/config: {{ include (print $.Template.BasePath "/configmap.yaml") . | sha256sum }}

# kustomization.yaml
configMapGenerator:
- name: app-config
  files: [app.properties]        # generates app-config-<hash>

git revert <commit> && git push        # Argo CD syncs old config
kubectl rollout restart deploy/api -n prod
```

> ***What to say to the interviewer:*** *I restore the previous ConfigMap from Git or helm rollback and restart the Deployment, remembering rollout undo alone does not revert ConfigMap data. To prevent it, config lives in Git with review, and I use hashed or immutable ConfigMaps or a Helm checksum annotation so every config change becomes a normal rolling update gated by readiness probes and easy to roll back.*

Q69. The dev team pushed a bad Helm chart and the rollback hook fails. What is your live rollback strategy?

In simple words: The emergency exit door (rollback hook) is jammed. You do not stand there pushing it - you use the side door (skip hooks or roll the Deployment back directly), get everyone out, and fix the door later.

Detailed answer

Step by step:

1. **See release state:** `helm history <rel>` and `helm status` - find the last good revision and whether the release is `failed` or stuck in `pending-upgrade` / `pending-rollback`.
2. **Rollback without hooks:** `helm rollback <rel> <good-rev> --no-hooks --wait --timeout 5m`. This skips the broken pre/post-rollback hook Job.
3. **Stuck in pending state** ("another operation is in progress"):

- First try `helm rollback` to the last good revision.

- Last resort: the release state is stored as Secrets `sh.helm.release.v1.<rel>.v<N>` in the release namespace; delete (after backing up) the Secret of the stuck pending revision, so Helm sees the last deployed revision again, then roll back/upgrade.

4. **Fastest app-level escape:** if Helm itself is broken, `kubectl rollout undo deploy/<app>` restores the previous ReplicaSet in seconds and stops user impact; reconcile Helm afterwards (next `helm upgrade` or rollback aligns it).
5. **Clean up failed hook Jobs** (`kubectl delete job <hook-job>`) that block reruns.
6. **GitOps:** if Argo CD manages it, **revert the Git commit** (or roll back the Application to the previous synced revision; pause auto-sync first) - otherwise Argo CD will re-apply the bad chart.
7. Verify: pods Ready, error rate, then communicate.

Prevent: `helm lint`, `helm template` + kubeconform, `helm diff` in CI; `--atomic --wait --timeout`; `helm.sh/hook-delete-policy: before-hook-creation,hook-succeeded`; hooks idempotent; deploy to staging first; canary.

Example:

```
helm history api -n prod
# REV  STATUS           CHART       DESCRIPTION
# 11   superseded       api-1.8.0   Upgrade complete
# 12   pending-upgrade  api-1.9.0   Preparing upgrade
helm rollback api 11 -n prod --no-hooks --wait --timeout 5m

kubectl get secret -n prod -l owner=helm,name=api
kubectl get secret sh.helm.release.v1.api.v12 -n prod -o yaml > backup-v12.yaml
kubectl delete secret sh.helm.release.v1.api.v12 -n prod     # last resort

kubectl rollout undo deploy/api -n prod                    # fastest app-level escape
```

What to say to the interviewer: *I check helm history, then run helm rollback to the last good revision with --no-hooks so the broken hook is skipped; if the release is stuck in pending-upgrade I roll back or, as a last resort, remove that revision's sh.helm.release secret after backing it up. If Helm is unusable, kubectl rollout undo restores service in seconds, and under Argo CD I revert the Git commit; prevention is lint, template and diff in CI plus --atomic and proper hook delete policies.*

Q70. Users cannot access an application exposed via Ingress (or the Ingress stopped working). How do you debug it?

Also asked as: Your service is not accessible externally - where do you start troubleshooting? If an Ingress stops working, how would you fix it?

In simple words: Follow the path of a letter: street address (DNS) -> building gate (load balancer) -> receptionist (Ingress controller) -> office (Service) -> person (Pod). Find the first place where the letter stops.

Detailed answer

Work outside-in (or inside-out - both fine, just be systematic):

1. **DNS:** `dig app.example.com` - does it resolve to the current LB address? (LB recreated -> new DNS name; Route 53 record stale.)
2. **Load balancer:** exists? listeners 80/443? **target group health** - unhealthy targets mean health-check path/port or SG problem. Security Group allows client traffic; subnets tagged correctly.
3. **TLS:** certificate valid and matches host (`curl -vk`); ACM/cert-manager renewal failed?
4. **Ingress controller:** pods running? `kubectl logs` of ingress-nginx / aws-load-balancer-controller - errors reconciling the Ingress (bad annotation, missing subnets, IAM permissions)? `ingressClassName` correct?
5. **Ingress rules:** `kubectl describe ingress` - host/path correct, backend service and port exist, pathType, rewrite rules.
6. **Service:** `kubectl get endpoints` - empty = selector mismatch or pods not Ready; `targetPort` matches container port.
7. **Pods:** Ready? app listening on that port? `kubectl port-forward` to pod and Service to test, bypassing ingress.
8. **Network policy** blocking traffic from ingress controller namespace; WAF rules blocking (403).
9. Check what changed recently (deploy, cert, annotation, controller upgrade).

Example:

```
dig +short app.example.com
curl -sv https://app.example.com/health -o /dev/null
kubectl get ingress -n shop; kubectl describe ingress shop -n shop
kubectl logs -n kube-system deploy/aws-load-balancer-controller | tail -50
kubectl get svc,endpoints backend -n shop
kubectl port-forward svc/backend 8080:80 -n shop; curl localhost:8080/health
aws elbv2 describe-target-health --target-group-arn <tg-arn>
```

What to say to the interviewer: *I trace the request path hop by hop: DNS resolution, load balancer listeners and target health, TLS, ingress controller logs and the Ingress rules, then Service endpoints and finally the pods with port-forward. Empty endpoints, wrong targetPort, unhealthy targets from a bad health-check path, and controller errors from annotations or IAM are the most common causes.*

Q71. Kubernetes pods are healthy, but users are receiving errors (for example 503). How do you troubleshoot the complete request flow?

Also asked as: You are paged for 503 errors but all pods look healthy - where do you look next: metrics, probes, Ingress?

In simple words: Every worker says "I am fine", yet customers are still turned away at the door. The problem is somewhere on the path between the door and the workers, or the workers are "fine" only because the health check is too easy.

Detailed answer

Request flow: **Client -> DNS -> CDN/WAF -> Load balancer -> Ingress controller -> Service -> Pod -> downstream (DB/cache/APIs).**

1. **Where are errors generated?** Look at who returns the 503: LB metrics (ELB 5xx vs Target 5xx), ingress controller access logs (`upstream connect error`, `no live upstreams`), or the app itself.
2. **Endpoints/readiness:** `kubectl get endpoints` - maybe only a few pods are Ready; readiness flapping; probes too shallow (pod "healthy" but app returns 503 for real requests).
3. **LB target health:** targets draining/unhealthy; target type (instance vs IP); `externalTrafficPolicy: Local` nodes without pods fail health checks.
4. **Capacity:** HPA already at `maxReplicas`? pods saturated (CPU throttling, thread/connection pool exhausted), ingress `max connections`/rate limits, app queue full -> 503.
5. **Timeouts and keep-alive:** app keep-alive timeout shorter than LB/ingress idle timeout -> intermittent 502/503 when the app closes a connection the proxy reuses. Set app keep-alive > LB idle timeout.
6. **Deploys/drains:** errors only during rollouts -> missing preStop/graceful shutdown; PDB missing during node drains/Karpenter consolidation.
7. **Downstream:** DB max connections, Redis latency, third-party API failing - app returns 503 when a dependency is down. Traces (Tempo/Jaeger) show the slow span.
8. **Network:** NetworkPolicy, DNS latency, conntrack table full, SNAT/NAT port exhaustion.
9. **Segment the errors:** by path, pod, node, AZ, version, client region - a pattern points to the cause.

Example:

```
kubectl get endpoints api -n prod -o wide
kubectl logs -n ingress-nginx deploy/ingress-nginx-controller | grep -E ' 50[234] ' | tail
kubectl get hpa -n prod; kubectl top pods -n prod
kubectl get events -n prod --sort-by=.lastTimestamp | tail
# PromQL: error rate by pod
sum by (pod) (rate(http_requests_total{status=~"5.."}[5m]))
aws cloudwatch get-metric-statistics --namespace AWS/ApplicationELB \
  --metric-name HTTPCode_ELB_5XX_Count ...
```

What to say to the interviewer: *I follow the request through DNS, load balancer, ingress, Service and pods, first identifying which layer generates the 503 from LB metrics and ingress logs. Then I check endpoints and readiness, LB target health, HPA at max or saturated pods, keep-alive and timeout mismatches, graceful shutdown during deploys and downstream dependencies, segmenting errors by pod, node and zone to find the pattern.*

Q72. A Service is running but application requests are timing out. How do you troubleshoot the networking?

In simple words: The phone number exists and the line rings, but nobody answers or the call drops. Either the call is not reaching the right person, or the person is too slow to pick up.

Detailed answer

Timeouts (not immediate refusal) usually mean packets are dropped or the app is too slow. Steps:

1. **Endpoints:** `kubectl get endpoints` - pods listed? If empty, clients may hang/refuse.
2. **Bypass layers to localize:**

- Call the **pod IP:port** directly from another pod -> works? then Service/kube-proxy layer.
- Call the **ClusterIP** -> works? then DNS or client side.
- Call the **DNS name** -> slow? DNS issue.

3. **Port mapping:** Service `port` -> `targetPort` must match the port the app listens on (`ss -lnt` in the pod). App bound to `127.0.0.1` instead of `0.0.0.0` is a classic.
4. **NetworkPolicy:** a default-deny drops packets silently -> timeout.
5. **kube-proxy/CNI:** kube-proxy pods healthy, iptables/IPVS rules present; CNI (aws-node) errors, IP exhaustion in subnet.
6. **Security Groups / NACLs** (EKS SG for pods, node SG between nodes), cross-node traffic blocked.
7. **App slowness:** CPU throttling, thread pool exhaustion, DB slow queries; check latency metrics and traces - "network timeout" is often a slow app.
8. **Conntrack table full** (`nf_conntrack: table full, dropping packet` in dmesg), MTU issues (large responses hang).
9. **Client timeouts** too short vs normal response time.

Example:

```
kubectl run net --rm -it --image=nicolaka/netshoot -- bash
  curl -m 3 -v http://10.0.45.12:8080/health      # pod IP
  curl -m 3 -v http://10.100.12.34/health          # ClusterIP
  curl -m 3 -v http://api.prod.svc.cluster.local/health
  nc -zv api.prod 80 ; dig api.prod.svc.cluster.local
kubectl exec <api-pod> -- ss -lntp                 # 0.0.0.0:8080 or 127.0.0.1:8080?
kubectl get networkpolicy -n prod
kubectl logs -n kube-system -l k8s-app=kube-proxy | tail
```

What to say to the interviewer: *I localize the timeout by testing pod IP, then ClusterIP, then DNS name from a netshoot pod, which tells me if it is the app, kube-proxy and Service layer, or DNS. Then I check targetPort versus the actual listening port and bind address, NetworkPolicies and security groups that silently drop packets, kube-proxy and CNI health, conntrack, and finally whether the app itself is just slow.*

Q73. Pod A cannot reach Pod B. How do you debug it?

In simple words: Two neighbours cannot visit each other. Check the address (DNS), whether the door is open (port), whether a guard blocks the gate (NetworkPolicy/SG), and whether the road exists (CNI/routing).

Detailed answer

1. **Are both pods Running and Ready,** and what are their IPs/nodes? (`-o wide`)
2. **How is A calling B?** Pod IP, Service name, or FQDN? Across namespaces it must be `svc-b.ns-b` or `svc-b.ns-b.svc.cluster.local`.
3. **DNS:** from pod A: `nslookup svc-b.ns-b.svc.cluster.local`.
4. **Service:** selector matches B's labels, endpoints populated, port/targetPort correct.
5. **Direct pod IP test:** `curl <podB-IP>:<port>` from A.

- Fails only across nodes but works on same node -> CNI/overlay/SG between nodes.
- Fails everywhere -> app not listening or NetworkPolicy.

6. **NetworkPolicy** on B's namespace (ingress) or A's (egress, including DNS egress).
7. **CNI health:** aws-node/calico/cilium pods, IP allocation errors; `cilium connectivity test` / Hubble for flow drops.
8. **Security Groups for pods / node SGs** on EKS; host firewall.
9. **Service mesh:** mTLS STRICT mode and pod A without sidecar -> connection reset.

Example:

```
kubectl get pod -o wide -n ns-a; kubectl get pod -o wide -n ns-b
kubectl exec -n ns-a podA -- nslookup svc-b.ns-b.svc.cluster.local
kubectl exec -n ns-a podA -- curl -m 3 -sv http://svc-b.ns-b:8080/health
kubectl exec -n ns-a podA -- curl -m 3 -sv http://<podB-ip>:8080/health
kubectl get endpoints svc-b -n ns-b
kubectl get networkpolicy -n ns-b -o yaml
kubectl get pods -n kube-system -l k8s-app=aws-node
```

> ***What to say to the interviewer:*** *I confirm both pods are ready, then test from pod A by FQDN, by Service ClusterIP and by pod B's IP to isolate DNS, Service and network layers. After that I check selectors and endpoints, NetworkPolicies in both namespaces including DNS egress, CNI health and security groups between nodes, and mesh mTLS settings.*

Q74. Service discovery fails across namespaces, but DNS looks green. What is your next step, and how do you confirm the root cause?

Also asked as: A pod cannot resolve service names, yet DNS looks fine - what is your next move?

> **In simple words:** The phone directory works for people on your own floor, but when you look up someone on another floor using only their first name, you get nothing - you need their full name and floor, and the corridor door must be open.

Detailed answer

"DNS looks green" (CoreDNS pods Running, no errors) does not mean the **name the app uses** resolves or the service behind it works. Next steps:

1. **Short name vs FQDN:** a short name `payments` only resolves inside the same namespace because of `search` domains in `/etc/resolv.conf` (`<ns>.svc.cluster.local svc.cluster.local cluster.local`). Cross-namespace needs `payments.billing` or `payments.billing.svc.cluster.local`. Check the app config.
2. **Test from the failing pod itself** (not from your laptop or a random pod):

- `nslookup payments.billing.svc.cluster.local` -> returns ClusterIP? Then DNS is really fine and the issue is further along.
- NXDOMAIN on short name but FQDN works -> config issue (wrong name/namespace).

3. **Pod's resolv.conf / dnsPolicy:** `hostNetwork` pods or `dnsPolicy: Default` use node DNS, not CoreDNS; custom `dnsConfig` may override search domains.
4. **Service has endpoints?** `kubectl get endpointslices -n billing -l kubernetes.io/service-name=payments` - empty = selector mismatch or pods not Ready. DNS still resolves the ClusterIP, but connections fail.
5. **Ports:** Service port vs `targetPort` vs container port; named port mismatch.
6. **NetworkPolicy** in the target namespace allowing only same-namespace traffic (namespaceSelector missing), or egress policy in the source namespace blocking.
7. **Curl the pod IP directly** from the source pod - works? then Service/kube-proxy (iptables rules) problem; check kube-proxy logs.
8. **Mesh / headless:** headless Service returns pod IPs; ExternalName misconfigured.

Confirm root cause by reproducing: FQDN nslookup -> ClusterIP, EndpointSlice contents, curl pod IP vs ClusterIP, and a temporary NetworkPolicy allow test in a non-prod namespace.

Example:

```
kubectl exec -n orders deploy/api -- cat /etc/resolv.conf
# search orders.svc.cluster.local svc.cluster.local cluster.local
# options ndots:5
kubectl exec -n orders deploy/api -- nslookup payments                    # NXDOMAIN
kubectl exec -n orders deploy/api -- nslookup payments.billing.svc.cluster.local
kubectl get endpointslices -n billing -l kubernetes.io/service-name=payments
kubectl exec -n orders deploy/api -- curl -m 3 -sv http://payments.billing:8080/health
kubectl exec -n orders deploy/api -- curl -m 3 -sv http://<payments-pod-ip>:8080/health
kubectl get networkpolicy -n billing -o yaml | grep -A5 namespaceSelector
```

> ***What to say to the interviewer:*** *I test from the failing pod itself: the short name only resolves within its own namespace through the search domains, so cross-namespace calls need name.namespace or the full svc.cluster.local FQDN. If the FQDN resolves, I check EndpointSlices for a selector mismatch, targetPort, NetworkPolicies that block cross-namespace traffic and kube-proxy, confirming by comparing curl to the ClusterIP versus the pod IP.*

Q75. Cluster-wide DNS stops resolving randomly across nodes. What is your step-by-step debug path?

> **In simple words:** The phone directory office answers most calls, but randomly drops some. Either too few operators (under-scaled), too many pointless calls (ndots), the phone lines get crossed (conntrack race), or the head office (upstream resolver) limits how many calls it takes.

Detailed answer

1. **Confirm and measure:** run a loop test from dnsutils pods on several nodes; note failure rate, timeouts vs NXDOMAIN/SERVFAIL, and whether failures are ~5 seconds long.
2. **CoreDNS health:** pods Running, restarts, OOMKilled, CPU throttling; logs (`SERVFAIL`, `i/o timeout` to upstream); metrics `coredns_dns_request_duration_seconds`, `coredns_dns_responses_total{rcode="SERVFAIL"}`, `coredns_forward_*`.

3. **Under-scaled:** only 2 CoreDNS replicas for a big cluster -> use **cluster-proportional-autoscaler** / more replicas, spread across nodes/AZs, raise limits.
4. **ndots:5 amplification:** every external name like `api.stripe.com` is tried with each search domain first (4-5 extra queries, A and AAAA). Fix: use FQDN with a trailing dot, or set `dnsConfig.options ndots: 2` for heavy callers; enable CoreDNS `autopath`/cache.
5. **Conntrack race (the famous 5-second timeouts):** glibc sends A and AAAA queries in parallel over UDP from the same socket; a Linux conntrack race drops one, and the client retries after 5 seconds. Fixes: **NodeLocal DNSCache** (node-local cache, upstream over TCP), `options single-request-reopen` (glibc), or avoiding musl/alpine quirks.
6. **Upstream limits:** on AWS the **VPC resolver allows 1024 packets per second per ENI**; CoreDNS pods concentrated on few nodes exceed it and packets drop. Check `linklocal_allowance_exceeded` in ENA stats (`ethtool -S`). Fix: NodeLocal DNSCache, more CoreDNS replicas spread across nodes, caching.
7. **NetworkPolicy** blocking UDP/TCP 53 to kube-system for some namespaces (big responses fall back to TCP).
8. **kube-proxy health:** stale iptables rules pointing to deleted CoreDNS pod IPs after restarts; kube-proxy errors.
9. **Node-level:** conntrack table full (`dmesg | grep conntrack`), CNI issues on specific nodes.
10. Fix, then keep **alerts** on CoreDNS latency/SERVFAIL and a synthetic DNS probe.

Example:

```
kubectl run dnsutils --rm -it --image=registry.k8s.io/e2e-test-images/jessie-dnsutils:1.3 -- bash
  for i in $(seq 1 200); do
    start=$(date +%s%N); getent hosts api.prod.svc.cluster.local >/dev/null || echo FAIL
    echo $(( ($(date +%s%N)-start)/1000000 ))ms
  done | sort | uniq -c | sort -rn | head
  dig +search +stats payments.billing
kubectl get pods -n kube-system -l k8s-app=kube-dns -o wide
kubectl logs -n kube-system -l k8s-app=kube-dns --tail=100 | grep -iE "servfail|timeout"
kubectl top pods -n kube-system -l k8s-app=kube-dns
# on a node:
ethtool -S eth0 | grep linklocal_allowance_exceeded
dmesg | grep -i conntrack
```

What to say to the interviewer: *I measure with a DNS loop test from several nodes, then check CoreDNS pods, logs and latency and SERVFAIL metrics, scaling CoreDNS if it is under-provisioned. Classic causes are ndots:5 query amplification, the UDP conntrack race that causes 5-second timeouts, and on AWS the VPC resolver limit of 1024 packets per second per ENI; NodeLocal DNSCache fixes most of these, along with checking NetworkPolicies on port 53 and kube-proxy health.*

Q76. One worker node becomes NotReady. What actions do you take?

In simple words: A branch office stopped answering the head office's calls. Is the office closed (machine dead), is the manager sick (kubelet), or is the phone line cut (network)? Meanwhile, move urgent work to other offices.

Detailed answer

1. **Confirm and scope:** one node or many? same AZ/node group? `kubectl get nodes`, `kubectl describe node` -> Conditions (Ready, MemoryPressure, DiskPressure, PIDPressure, NetworkUnavailable) and events.
2. **Protect workloads:** `kubectl cordon` to stop new pods; after ~5 minutes pods are evicted automatically (taint + toleration 300s); for faster recovery `kubectl drain --ignore-daemonsets --delete-emptydir-data`.
3. **Check the machine:** EC2 status checks, instance still running? Spot interruption? (cloud console / `aws ec2 describe-instance-status`).
4. **On the node (SSM Session Manager/SSH):**

- `systemctl status kubelet`, `journalctl -u kubelet` - crashed? certificate expired? cannot reach API endpoint?

- `systemctl status containerd` - runtime down.

- Disk full (`df -h`, images/logs), memory exhaustion (`free -m`, `dmesg | grep -i oom`), PID exhaustion.

- CNI: aws-node pod on that node failing, out of IPs.

- Network: SG/NACL change, route, DNS to API endpoint.

5. **Fix or replace:** restart kubelet/containerd, clean disk; in the cloud it is usually faster and safer to **terminate the node** and let the node group/Karpenter replace it (cattle, not pets).
6. **Root cause + prevention:** kubelet `system-reserved`/`kube-reserved`, eviction thresholds, log rotation, image GC, node problem detector, alerts on NotReady.

Example:

```
kubectl get nodes
kubectl describe node ip-10-0-2-40.ap-south-1.compute.internal | sed -n '/Conditions/,/Addresses/p'
kubectl cordon ip-10-0-2-40...
kubectl drain ip-10-0-2-40... --ignore-daemonsets --delete-emptydir-data --timeout=5m
aws ssm start-session --target i-0abc123
  sudo systemctl status kubelet containerd
  sudo journalctl -u kubelet --since "30 min ago" | tail -50
  df -h ; free -m ; dmesg | grep -iE "oom|conntrack" | tail
aws ec2 terminate-instances --instance-ids i-0abc123      # let ASG/Karpenter replace
```

> ***What to say to the interviewer:*** *I check node conditions and events, cordon and drain the node to protect workloads, then look at the instance status and, on the node, kubelet and containerd logs, disk, memory and CNI. If it is not a quick fix I terminate it and let the node group or Karpenter replace it, then add prevention like reserved resources, eviction thresholds and NotReady alerts.*

Q77. A node went down suddenly. What happens to the pods running on it?

In simple words: When a shop suddenly closes, the head office waits a few minutes in case it reopens, then reassigns its staff to other shops. Staff with their own locked cabinets (StatefulSets with disks) have to wait until it is certain the old shop is truly closed.

Detailed answer

Timeline (default settings):

1. Kubelet stops sending heartbeats (Lease objects).

2. After the **node monitor grace period (~40s, 50s in newer versions)** the node controller marks it **NotReady/Unknown** and adds taints `node.kubernetes.io/unreachable:NoExecute` (or `not-ready`).
3. Pods have a default toleration of **300 seconds** for these taints. After ~5 minutes they are **evicted** (marked Terminating; the kubelet is unreachable so they cannot actually be killed).
4. **Deployment/ReplicaSet pods:** controller creates replacements on healthy nodes right away after eviction. Traffic stops going to the dead pods much earlier, because endpoints are updated when the node is NotReady (pods marked not ready).
5. **StatefulSet pods:** NOT recreated automatically while the old pod still exists (Terminating), to avoid two pods with the same identity writing to the same disk (split brain). You must confirm the node is dead, then `kubectl delete pod --force --grace-period=0` or delete the Node object; in the cloud, the node object is removed when the instance is terminated.
6. **Volumes:** EBS (RWO) must detach from the dead node before attaching elsewhere - can take several minutes (force detach after ~6 min).
7. **DaemonSet pods:** not rescheduled (they belong to that node).
8. Bare pods (no controller) are simply lost.

To recover faster: lower `tolerationSeconds` for critical pods, multiple replicas spread across nodes/AZs, PDBs, fast node replacement.

Example:

```
tolerations:
- {key: node.kubernetes.io/unreachable, operator: Exists, effect: NoExecute, tolerationSeconds: 60}
- {key: node.kubernetes.io/not-ready,   operator: Exists, effect: NoExecute, tolerationSeconds: 60}

kubectl get pods -o wide | grep <dead-node>          # Terminating / Unknown
kubectl delete pod mongo-1 --force --grace-period=0  # only after confirming node is dead
```

> ***What to say to the interviewer:*** *The node is marked NotReady after about 40 to 50 seconds and tainted, pods are removed from endpoints, and after the default 300-second toleration they are evicted and Deployments recreate them on healthy nodes. StatefulSet pods are not replaced until the node is confirmed gone, to avoid split brain, and EBS volumes need to detach first, so I tune tolerationSeconds for critical apps and always spread replicas.*

Q78. A node has reached (exhausted) its Pod capacity. What would you check first, and what would you do?

> **In simple words:** A parking lot has a fixed number of spots. Even if there is space on the ground (free CPU/memory), when all numbered spots are used, no more cars can park.

Detailed answer

What "pod capacity" means: each node has a **max pods** limit (`kubectl describe node` -> Capacity: pods). Default kubelet limit is 110; on **EKS with the VPC CNI** it is based on ENIs x IPs per ENI (e.g. t3.medium = 17 pods, m5.large = 29), because every pod needs a VPC IP.

Check first:

1. `kubectl describe node` -> Capacity/Allocatable pods vs running pods; pending pod events say `Too many pods`.

2. Is it the **pod limit** or **IP exhaustion** (subnet has no free IPs - aws-node logs `failed to assign an IP address`)?

3. What is using the slots: completed/failed pods not cleaned up, many small DaemonSets, a runaway Job/CronJob creating pods, too many tiny replicas?

Fixes:

- Short term: clean up Completed/Evicted pods; let autoscaler add nodes (Cluster Autoscaler/Karpenter respect the pod limit); cordon and spread.

- **EKS prefix delegation** (`ENABLE_PREFIX_DELEGATION=true`) - assigns /28 prefixes, raising pods per node (e.g. up to 110 on many Nitro instances); update node max-pods accordingly.

- Larger instance types (more ENIs/IPs), larger subnets or custom networking (secondary CIDR for pods).

- Set `ttlSecondsAfterFinished` for Jobs, `successfulJobsHistoryLimit` for CronJobs; review replica counts.

- Topology spread so pods do not pile onto one node.

Example:

```
kubectl describe node ip-10-0-1-12... | grep -A6 -E "Capacity|Allocatable"
#  pods: 17
kubectl get pods -A --field-selector spec.nodeName=ip-10-0-1-12... | wc -l
kubectl get pods -A --field-selector=status.phase==Succeeded -o name | xargs kubectl delete
kubectl logs -n kube-system -l k8s-app=aws-node --tail=50 | grep -i "ip address"
kubectl set env daemonset aws-node -n kube-system ENABLE_PREFIX_DELEGATION=true
```

> ***What to say to the interviewer:*** *I first check describe node for the pods capacity versus running count and whether pending pods say Too many pods or the CNI is out of subnet IPs. Short term I clean up completed pods and let the autoscaler add nodes; long term on EKS I enable prefix delegation or use bigger instances and larger subnets, and set TTLs on Jobs so finished pods do not waste slots.*

Q79. Pods are getting evicted frequently due to resource pressure (for example in EKS). How do you fix and prevent it?

> **In simple words:** When the lifeboat is overloaded, the captain throws off the passengers without tickets first. Give every important passenger a proper ticket (requests), do not overload the boat, and keep it clean.

Detailed answer

Eviction happens when a **node** is under pressure (memory, disk/ephemeral storage, PIDs). The kubelet evicts pods in order: **BestEffort** (no requests) first, then **Burstable** pods using more than their request, **Guaranteed** last. (Different from OOMKilled, which is a single container exceeding its own limit.)

Investigate:

1. `kubectl get pods -A | grep Evicted` and `kubectl describe pod` -> reason: `The node was low on resource: memory` / `ephemeral-storage`.

2. Which nodes? `kubectl describe node` conditions and allocated resources; overcommitted limits?

3. Metrics: node memory, disk usage (container logs, images, emptyDir), which pods consume most.

Fix / prevent:

- **Set requests for every pod** (requests close to actual usage), so the scheduler does not overpack nodes. Use LimitRange defaults so nothing runs as BestEffort.
- Memory limits close to requests for heavy apps; critical pods as **Guaranteed** QoS + **PriorityClass**.
- **Ephemeral storage:** set `ephemeral-storage` requests/limits, log rotation, avoid writing big files to container filesystem, bigger root volumes, image garbage collection.
- **Kubelet reservations:** `kube-reserved`, `system-reserved`, eviction thresholds (EKS AMIs set defaults; tune via bootstrap/launch template).
- Right-size node types; autoscaler to add capacity; fix memory leaks.
- Spread with topology constraints; PDBs.
- Alerts on node MemoryPressure/DiskPressure and on eviction counts.
- Clean up evicted pods (they stay as Failed objects).

Example:

```
kubectl get pods -A --field-selector=status.phase=Failed | grep Evicted
kubectl describe pod <evicted-pod> | grep -i -A2 "Reason"
kubectl describe nodes | grep -A10 "Allocated resources"
kubectl top pods -A --sort-by=memory | head

resources:
  requests: {cpu: 250m, memory: 512Mi, ephemeral-storage: 1Gi}
  limits:   {memory: 512Mi, ephemeral-storage: 2Gi}
```

> ***What to say to the interviewer:*** *Evictions mean the node itself is under memory or disk pressure, and BestEffort and over-request Burstable pods go first. I fix it by giving every pod realistic requests, making critical pods Guaranteed with a PriorityClass, setting ephemeral-storage limits and log rotation, reserving resources for kubelet and system, right-sizing nodes, and alerting on node pressure.*

Q80. You see high CPU usage in one pod (or one node group), but its logs look clean. What next?

Also asked as: CPU usage on one EKS node group is consistently high - Grafana shows the spike but logs are normal. A critical microservice in production has high CPU usage - diagnose and resolve it in real time.

> **In simple words:** A worker is sweating heavily but his diary says everything is normal. You need to watch what he is actually doing (profiling), not just read his diary.

Detailed answer

Real-time mitigation first if users are affected: scale out (`kubectl scale` or raise HPA min/max), temporarily raise the CPU limit if it is throttled, or **roll back** if a recent deploy caused it. Then diagnose:

1. **Confirm it is real and scope:** `kubectl top pod --containers`, Grafana per-container CPU; only one replica or all? If only one -> something specific to that pod (hot key, stuck request, bad node). All replicas -> load or code. For a **node group**: `kubectl top nodes`, then which

pods on those nodes use the CPU (often one workload, a DaemonSet, or pods packed onto that group by nodeSelector).

2. **What changed?** Recent deploy (`kubectl rollout history`), config change, traffic increase or new client, a batch job/CronJob landing on that node group.

3. **Is it load?** Compare request rate to that pod - uneven load balancing (long-lived keep-alive/gRPC connections pinned to one pod), a hot partition or hot key.

4. **Is it throttling?** Check `container_cpu_cfs_throttled_periods_total` and compare usage vs **requests vs limits**: usage at the limit = throttled (slow responses, probe timeouts); requests far below usage = node overcommitted.

5. **HPA status:** `kubectl get hpa` - at maxReplicas? not scaling because of missing requests or metrics? (see the HPA scenario).

6. **Look inside the process:** `kubectl exec` -> `top -H` for threads; profilers: Java `jstack`/`jcmd Thread.print` or async-profiler, Go `pprof`, Node `--inspect`, py-spy style samplers; continuous profiling (Pyroscope). GC thrash from memory near heap limit causes high CPU with clean logs.

7. **Traces/APM:** Tempo/Jaeger/Datadog showing slow endpoints.

8. **Background work:** cron-like tasks inside the app, retry storms, infinite loop after a specific input, sidecar (Envoy, log shipper) using CPU rather than the app container - check per-container CPU.

9. **Noisy neighbours / node:** another pod on the same node without CPU requests, kernel issues, instance type too small (burstable t3 out of CPU credits); move the pod (`kubectl delete pod`) to compare.

10. Fix root cause (code, config, balancing, right-sized requests/instance type) and add alerts on throttling and per-pod CPU.

Example:

```
kubectl top pod -n prod --containers | sort -k3 -rn | head
kubectl exec -it api-7d9f-abcde -n prod -- top -H -b -n1 | head -20
kubectl exec -it api-7d9f-abcde -n prod -- jcmd 1 Thread.print > threads.txt
# PromQL: per-pod CPU and throttling
sum by (pod,container) (rate(container_cpu_usage_seconds_total{namespace="prod"}[5m]))
sum by (pod) (rate(container_cpu_cfs_throttled_periods_total{namespace="prod"}[5m]))
```

> ***What to say to the interviewer:*** *If users are affected I first mitigate by scaling out, raising a throttled limit or rolling back a recent deploy. Then I check whether it is one pod, all pods or one node group, what changed, and whether request distribution explains it, such as sticky connections or a hot key. Then I profile inside the container with top -H and language tools like jstack or pprof, check GC pressure, per-container CPU in case a sidecar is the culprit, and throttling metrics; I mitigate by restarting or scaling and then fix the code or balancing issue.*

Q81. etcd becomes unavailable in a production cluster. What is your recovery strategy?

> **In simple words:** The cluster's memory notebook is missing pages. Running work continues, but no new instructions can be written. Fix the notebook if possible; if not, restore it from last night's photocopy.

Detailed answer

First, understand impact: API server errors (`etcdserver: request timed out`, `leader changed`), kubectl fails, no scheduling/healing - but **running pods keep serving**. Do not panic-restart workloads.

Managed (EKS/GKE/AKS): etcd is the provider's responsibility - check the provider health dashboard, open a high-priority support case, avoid mass changes; ensure your own recovery path (GitOps + Velero) is ready.

Self-managed:

1. **Check members:** `etcdctl endpoint health --cluster`, `endpoint status` (leader, DB size, raft index), `member list`.
2. **Common causes:** disk full or DB size exceeded quota (`mvcc: database space exceeded` -> compact + defrag + disarm alarm), slow disk (fsync latency - etcd needs fast SSD), expired certificates, network partition, clock issues.
3. **One member down, quorum intact:** fix or `member remove` the bad one and `member add` a fresh one.
4. **Quorum lost / data corrupt:** restore from the latest **snapshot**:

- stop kube-apiserver (and etcd) on all control-plane nodes,

- `etcdutl snapshot restore` (older: `etcdctl snapshot restore`) on each member with the right `--name`, `--initial-cluster`, new data dir,

- start etcd, verify health, then start API servers,

- reconcile: objects created after the snapshot are lost; GitOps re-applies them.

5. **Prevention:** scheduled snapshots (every few hours, copied off-cluster), 3/5 members across zones, monitoring (`etcd_server_has_leader`, `etcd_disk_wal_fsync_duration_seconds`, DB size), cert rotation, restore drills.

Example:

```
ETCDCTL_API=3 etcdctl --endpoints=$EPS --cacert=ca.crt --cert=etcd.crt --key=etcd.key \
  endpoint status --cluster -w table
etcdctl alarm list
etcdctl compact $(etcdctl endpoint status -w json | jq '.[0].Status.header.revision')
etcdctl defrag --cluster && etcdctl alarm disarm

etcdutl snapshot restore /backup/etcd-2026-09-24.db --name cp1 \
  --initial-cluster cp1=https://10.0.1.10:2380,cp2=https://10.0.2.10:2380,cp3=https://10.0.3.10:2380
     \
  --initial-advertise-peer-urls https://10.0.1.10:2380 --data-dir /var/lib/etcd-restored
```

> ***What to say to the interviewer:*** *Running pods keep serving, so I focus on restoring the control plane: on EKS that is AWS's responsibility and I raise a support case, while self-managed I check member health, fix disk space or quota alarms with compact and defrag, and replace a failed member if quorum still exists. If quorum is lost I restore all members from the latest snapshot and let GitOps re-apply newer changes, and I prevent it with frequent off-cluster snapshots, fast disks and etcd monitoring.*

Q82. HPA refuses to scale a critical workload even though Prometheus shows CPU above 90%. What is the root cause and fix?

In simple words: The fire alarm in the control room (Prometheus) is ringing, but the sprinkler system (HPA) reads a different sensor that says everything is fine - or the sprinkler is already at full blast, or the water pipes (nodes, quota) are blocked.

Detailed answer

Check `kubectl describe hpa` first - conditions and events tell you why. Common root causes:

1. **Different measurement:** HPA CPU utilization = usage / **request**. Prometheus may show % of node CPU or % of **limit**. Example: request 1 core, limit 2 cores, usage 1.8 cores = 90% of limit but HPA sees 180% (would scale) - or the reverse if the dashboard is node-based. Compare apples to apples.
2. **Missing requests:** if any container in the pod (including a sidecar) has no CPU request, HPA cannot compute utilization -> `FailedGetResourceMetric` / `missing request for cpu`.
3. **metrics-server broken/lagging:** targets show <unknown>, APIService unavailable.
4. **Already at maxReplicas** -> `ScalingLimited: TooManyReplicas`.
5. **Scaled, but new pods Pending:** no node capacity (autoscaler not working, max nodes reached), **ResourceQuota** exceeded (events on ReplicaSet), IP exhaustion.
6. **Wrong target:** `scaleTargetRef` points to a different Deployment name, or the Deployment is managed by another tool that resets replicas (Helm/Argo CD setting `replicas`, fighting HPA).
7. **Behavior/stabilization policies** too conservative; unready pods and CPU during startup (`cpu-initialization-period`) are ignored in the calculation.
8. **HPA based on the wrong metric** (memory or custom metric) while CPU is high.
9. Target averaged across pods: one hot pod at 90% but average 50%.

Fix accordingly: set requests on all containers, repair metrics-server, raise maxReplicas and node group max, remove `replicas` from Helm when HPA manages it, adjust quota, tune behavior.

Example:

```
kubectl describe hpa api -n prod
#  AbleToScale     True    ReadyForNewScale
#  ScalingActive   False   FailedGetResourceMetric  missing request for cpu in container "envoy"
#  ScalingLimited  True    TooManyReplicas
kubectl get hpa api -n prod -o yaml | grep -A3 scaleTargetRef
kubectl get pods -n prod -l app=api | grep Pending
kubectl describe quota -n prod
```

What to say to the interviewer: *I read describe hpa conditions first. Typical causes are a container or sidecar without a CPU request, metrics-server not serving metrics, HPA already at maxReplicas, scaled pods stuck Pending due to node capacity or ResourceQuota, a wrong scaleTargetRef, or Helm resetting replicas; I also make sure I compare HPA's usage-over-request with what Prometheus is actually graphing.*

Q83. You are on call: pods are not pulling images (ImagePullBackOff), some are getting evicted, and users are seeing errors or 503s. How do you troubleshoot, and how do you prevent it next time?

Also asked as: Pods show ImagePullBackOff in EKS right after a new image was pushed - what do you check?

In simple words: Three alarms at once. First stop the bleeding for customers, then find whether one root problem (for example a full disk on nodes) is causing all three before fixing each one.

Detailed answer

1. Stabilize and triage (first 5-10 minutes):

- Declare incident, check dashboards: which services, namespaces, nodes, AZs? Recent changes (deploys, node upgrades, registry credential rotation)?
- If a recent deploy caused it -> roll back. Keep healthy old pods serving (do not delete running pods).

2. Image pulls:

- `kubectl describe pod` events: `not found` (wrong tag), `unauthorized`/`no basic auth credentials` (ECR permissions on node role, expired imagePullSecret), `toomanyrequests` (Docker Hub rate limit), `i/o timeout` (NAT Gateway/VPC endpoint issue, registry outage).
- Often linked to evictions: **disk pressure** on nodes triggers image garbage collection and pulls fail due to no space.
- **EKS-specific checks after a new image push:**
- Does the exact **tag/digest exist** in the ECR repo and region the manifest points to? (`aws ecr describe-images`) - typo, tag not pushed yet, or tag overwritten/lifecycle-expired.
- **Node IAM role** has ECR read (`AmazonEC2ContainerRegistryReadOnly` managed policy).
- **Cross-account ECR:** the repository policy in the registry account must allow the node role of the cluster account (`ecr:BatchGetImage`, `ecr:GetDownloadUrlForLayer`, `ecr:BatchCheckLayerAvailability`).
- **Private nodes without NAT:** need VPC endpoints `ecr.api`, `ecr.dkr` and the **S3 gateway endpoint** (image layers are stored in S3), with SGs allowing 443.
- **Architecture:** image built only for arm64 but nodes are amd64 (or the reverse) -> `no matching manifest for linux/amd64`; build multi-arch.

3. Evictions:

- Node conditions MemoryPressure/DiskPressure; which pods consume most; BestEffort pods without requests.
- Short term: add nodes, cordon bad nodes, clean disk, move critical workloads.

4. User errors / 503s:

- Endpoints for key services - enough Ready pods? HPA at max? Ingress/LB target health.
- Scale critical services up, prioritize with PriorityClass.

5. Prevention:

- Pull from **ECR** (pull-through cache for Docker Hub), pin image tags/digests, VPC endpoints for ECR/S3, `imagePullPolicy: IfNotPresent`.
- Requests/limits everywhere, LimitRange, PriorityClasses, PDBs, ephemeral-storage limits, larger root volumes, log rotation.
- Alerts on ImagePullBackOff, evictions, node pressure, and SLO-based user error alerts.
- Postmortem with timeline and action items.

Example:

```
kubectl get pods -A | grep -E "ImagePull|ErrImage|Evicted|CrashLoop" | awk '{print $1,$4}' | sort |
    uniq -c
kubectl describe pod <pod> | sed -n '/Events/,$p'
#  Failed to pull image ... 403 Forbidden / no space left on device
kubectl describe nodes | grep -E "Name:|DiskPressure|MemoryPressure"
kubectl get endpoints -n prod
kubectl get hpa -A
aws ecr describe-images --repository-name api --image-ids imageTag=1.4.2 --region ap-south-1
aws iam list-attached-role-policies --role-name <eks-node-role>
docker manifest inspect <acct>.dkr.ecr.ap-south-1.amazonaws.com/api:1.4.2 | grep architecture
```

What to say to the interviewer: *I stabilize first by rolling back any recent change and protecting the critical services, then look for a common root cause such as disk pressure on nodes that causes both evictions and failed image pulls. I check pull errors for the tag or digest, node role ECR permissions, cross-account repository policy, ECR and S3 VPC endpoints, image architecture and rate limits, node pressure conditions, and endpoints behind the 503s; prevention is ECR pull-through caching, pinned tags, requests and limits everywhere, priority classes, PDBs and alerts on these states.*

Q84. An app on AKS (or any managed Kubernetes) fails health checks randomly. How do you debug this end-to-end?

Also asked as: How do you configure and troubleshoot health probes in AKS?

In simple words: The doctor checks the patient's pulse every 10 seconds, but gives up if the pulse takes more than 1 second to find. If the patient is sometimes busy, the doctor wrongly declares him sick. Find out whether the patient is really sick or the check is too strict.

Detailed answer

1. **Which health check fails?** Kubernetes probes (events `Liveness/Readiness probe failed`) or the **Azure Load Balancer / Application Gateway** health probe (backend health in portal)? They are different.
2. **Pattern:** random pods or specific nodes? Correlates with traffic peaks, GC, deployments, node upgrades, time of day?
3. **Probe config:** default `timeoutSeconds: 1` is too tight for many apps; probe hitting an expensive endpoint (checks DB); liveness checking dependencies -> restarts during DB blips. Relax timeout/threshold, make probes lightweight, add startup probe.
4. **App/resource side:** CPU throttling (CPU limit too low -> probe response slow), GC pauses, thread pool exhaustion (probe waits in the same queue as user requests), memory pressure.
5. **Node side:** node pressure, kubelet busy, noisy neighbours, SNAT port exhaustion on outbound, conntrack.

6. **Load balancer probe side:** `externalTrafficPolicy: Local` makes nodes without pods fail LB probes (expected); App Gateway probe path/host header/port mismatch, TLS/SNI, probe interval vs app warm-up.
7. **Network:** NSG rules, CNI (Azure CNI IP exhaustion), DNS delays in dependency checks.
8. **Correlate** with metrics/logs (Container Insights / Prometheus): probe failures vs CPU throttling vs latency vs GC.
9. Fix, then load test to confirm.

Example:

```
kubectl get events -A --field-selector reason=Unhealthy --sort-by=.lastTimestamp | tail -20
kubectl describe pod <pod> | grep -A3 -E "Liveness|Readiness"
kubectl top pod <pod> --containers
kubectl exec <pod> -- sh -c 'time wget -qO- localhost:8080/healthz'

livenessProbe:
  httpGet: {path: /healthz, port: 8080}
  timeoutSeconds: 3
  periodSeconds: 10
  failureThreshold: 3
```

> ***What to say to the interviewer:*** *I first separate Kubernetes probe failures from cloud load balancer probe failures, then look for a pattern by node, time and load. Most often the cause is a probe timeout that is too tight combined with CPU throttling, GC pauses or a heavy probe endpoint, so I make probes lightweight, relax timeouts, add a startup probe and fix resources, and verify LB probe settings like path, port and externalTrafficPolicy.*

Q85. In a canary deployment to production, half the traffic returns 502 while the rest succeeds. How do you troubleshoot?

> **In simple words:** Half the customers go to the new counter and get turned away, the other half go to the old counter and are served. The first suspicion is the new counter - but also check the signpost that splits the queue.

Detailed answer

1. **Stop the bleeding:** 50% errors is too much - set canary weight back to 0 / abort the rollout (Argo Rollouts abort, Istio weights 100/0). Then debug calmly.
2. **Confirm the split:** break errors down by **version/subset label**, pod and node. If all 502s come from canary pods -> the canary is broken. If spread across both -> infra/route issue.
3. **502 meaning:** the proxy (ingress/Envoy/ALB) got an **invalid or no response from upstream**: connection refused/reset, upstream closed connection, protocol mismatch.
4. **Canary pods:** Ready? Listening on the right port (targetPort changed in new version)? Crashing under load? Logs of the canary pods and of the ingress/Envoy (`upstream connect error or disconnect/reset before headers`, response flags `UF`, `UC`, `URX`).
5. **Routing config:** canary Service selector correct; Istio **DestinationRule subsets** match pod labels (`version: v2`); mTLS mode mismatch (canary pods missing sidecar injection -> resets); NGINX canary annotations pointing to right Service; ALB weighted target group health.
6. **Protocol/timeouts:** new version switched to HTTP/2, gRPC or HTTPS; app keep-alive timeout lower than proxy idle timeout (connection reuse -> 502); new version slower than proxy

timeout.

7. **Dependencies:** canary uses a new config/secret/DB migration that is failing.
8. Fix, re-run canary at 1-5% with **automated analysis** (error rate, latency) to catch this before 50%.

Example:

```
# errors by version
sum by (version) (rate(istio_requests_total{destination_app="api",response_code="502"}[1m]))
kubectl get pods -l app=api -L version -o wide
kubectl logs -l app=api,version=v2 --tail=50
kubectl logs -n istio-system deploy/istio-ingressgateway | grep " 502 " | tail
kubectl get destinationrule api -o yaml | grep -A3 subsets
kubectl argo rollouts abort api -n prod
```

> ***What to say to the interviewer:*** *I abort or roll the canary weight back to zero first, then split the 502s by version; if they all come from canary pods I check their readiness, ports, logs and new config. I also verify the routing layer, such as DestinationRule subset labels, sidecar and mTLS consistency, protocol changes and keep-alive timeouts, and then retry the canary at a small weight with automated metric analysis.*

Q86. Users report 10-second delays every 15 minutes in an app running on AKS (or any Kubernetes). No code changes happened. How do you begin RCA?

> **In simple words:** Something happens like clockwork every 15 minutes. Periodic problems come from periodic things - find everything that runs on a 15-minute timer.

Detailed answer

1. **Confirm the pattern precisely:** exact timestamps, duration, which endpoints/users/regions; is it all pods or some? Plot p99 latency at 1-minute resolution.
2. **List everything periodic** and correlate with the timestamps:

- CronJobs in the cluster (backups, reports, cleanup), DB maintenance/vacuum, cache expiry (TTL 15 min -> cache stampede).
- Token/credential refresh (managed identity, OAuth tokens, Key Vault/secret rotation, CSI driver sync interval).
- DNS TTLs / cache expiry, connection pool recycle (`maxLifetime`), idle connection timeouts (Azure Load Balancer idle timeout default 4 min, NAT idle timeout) -> first request after reconnect is slow.
- JVM full GC cycles, log rotation, metrics scrapes of heavy endpoints, HPA/autoscaler scale events, Karpenter/cluster autoscaler consolidation.
- Node-level: image GC, kernel tasks, antivirus/security agents, **SNAT port exhaustion** on outbound connections.

3. **Trace a slow request** (Application Insights / OpenTelemetry): which span takes 10 seconds - DNS lookup, TCP connect, TLS, DB query, external API?
4. **10 seconds exactly** is suspicious: typical of a timeout + retry (e.g. 5s DNS timeout x2, connect timeout), so check DNS (conntrack race), and connect timeouts.
5. Check platform events: AKS node image auto-upgrade, Azure maintenance, LB health.

6. Test hypothesis: change one variable (disable CronJob, adjust pool settings, enable NodeLocal DNSCache) and watch the next cycle.

Example:

```
kubectl get cronjobs -A
kubectl get events -A --sort-by=.lastTimestamp | grep -iE "scaled|killing|pull" | tail
# PromQL: p99 latency at 1m resolution
histogram_quantile(0.99, sum by (le) (rate(http_request_duration_seconds_bucket{app="api"}[1m])))
kubectl exec <pod> -- sh -c 'time nslookup db.internal'
```

What to say to the interviewer: *A 15-minute rhythm means a periodic cause, so I line up the exact latency spikes with everything periodic: CronJobs, cache TTLs, token or secret refresh, connection pool recycling, idle timeouts, GC and autoscaler events. I trace a slow request to see which span takes the 10 seconds, and an exact 10 seconds often points to timeouts and retries such as DNS or connection timeouts, then I test one hypothesis at a time.*

Q87. A kube-proxy update rolls out in the middle of a live event (for example a big match). What is your network rollback plan to avoid packet drops?

In simple words: Someone started rewiring the traffic lights city-wide during the rush hour. Stop the rewiring, put back the old lights one junction at a time, and next time only rewire at night, one junction first.

Detailed answer

Immediately:

1. **Pause the change:** stop the pipeline/add-on update; if it is a DaemonSet rollout in progress, check `kubectl rollout status ds/kube-proxy -n kube-system`.
2. **Assess impact:** packet drops/5xx/latency per node - are only updated nodes affected? Compare updated vs old nodes.
3. **Roll back kube-proxy:**

- Self-managed DaemonSet: `kubectl rollout undo ds/kube-proxy -n kube-system` (updates one node at a time with `maxUnavailable: 1`).

- EKS managed add-on: `aws eks update-addon --addon-name kube-proxy --addon-version <previous> --resolve-conflicts PRESERVE` (or pin the previous version in Terraform).

4. **Protect traffic during rollback:** cordon/drain affected nodes in the worst case so traffic shifts to healthy nodes; external LBs route away from unhealthy nodes (health checks). Keep existing connections: kube-proxy restarts do not flush conntrack by default, so established flows usually survive; avoid changing proxy **mode** (iptables <-> IPVS) mid-event since that rewrites all rules.
5. **Verify:** iptables/IPVS rules sync (`kubeproxy_sync_proxy_rules_duration_seconds`), Service connectivity tests, error rate back to normal.

Prevention:

- **Change freeze** during major events; add-on version pinned in IaC (no auto-update).
- Canary node group first, `maxUnavailable: 1`, soak time, automatic halt on error metrics.
- Test version compatibility (kube-proxy must match or be close to control plane version) in staging under load.

- Run a pre-event game day and keep a written rollback runbook.

Example:

```
kubectl -n kube-system rollout status ds/kube-proxy
kubectl -n kube-system rollout history ds/kube-proxy
kubectl -n kube-system rollout undo ds/kube-proxy
kubectl -n kube-system get ds kube-proxy -o jsonpath='{.spec.updateStrategy}'
aws eks describe-addon --cluster-name prod-eks --addon-name kube-proxy --query addon.addonVersion
aws eks update-addon --cluster-name prod-eks --addon-name kube-proxy \
  --addon-version v1.30.0-eksbuild.3 --resolve-conflicts PRESERVE
```

What to say to the interviewer: *I halt the rollout, compare updated versus not-yet-updated nodes to confirm impact, and roll kube-proxy back with rollout undo or by pinning the previous EKS add-on version, one node at a time, draining nodes if drops are severe. Established connections generally survive because conntrack is not flushed, and prevention is a change freeze for big events, pinned add-on versions and a canary node group with automatic halt.*

Q88. You need to containerize a legacy monolith and deploy it to EKS with minimal downtime. What is your approach?

In simple words: Move a shop to a new mall without closing it. Open the new shop while the old one still runs, send a few customers there first, then more and more, and only close the old shop when the new one is proven.

Detailed answer

1. Assess the monolith:

- Where is **state** kept: local disk (uploads, generated files), in-memory **sessions**, local cron jobs, config files with hard-coded hosts/secrets?
- Dependencies (DB, message queues, external IPs whitelisted), OS packages, startup time, memory needs.

2. Make it container-friendly (12-factor basics):

- **Externalize config** to env vars/ConfigMaps and secrets to Secrets Manager (External Secrets).
- **Sessions** to Redis/ElastiCache (or sticky sessions temporarily); files to **S3/EFS**.
- **Logs to stdout/stderr** instead of local files.
- Add **health endpoints** (`/health`, `/ready`) and graceful SIGTERM handling.
- Local crons -> Kubernetes CronJobs.

3. Build and deploy:

- Dockerfile (pinned base image, multi-stage if compiled, non-root), scan, push to ECR.
- Helm chart: Deployment with probes and realistic resources (monoliths are big - start generous, tune later), Service, Ingress (ALB), HPA if it is stateless now, PDB.
- Test in staging with production-like data and a load test.

4. Run in parallel and shift traffic gradually (strangler fig):

- Legacy stays live; the EKS version runs alongside, pointing to the **same database** initially (no data migration on day one).

- Shift traffic with **ALB weighted target groups** or **Route 53 weighted records**: 5% -> 25% -> 50% -> 100%, watching error rate, latency and business metrics at each step. Rollback = weight back to legacy.
- Lower DNS TTLs before starting.

5. After cut-over: keep legacy on standby for a few days, then decommission; later peel off features into microservices (strangler fig pattern) and plan the DB migration separately.

Example:

```
# ALB listener rule with weighted forward (Terraform / console)
forward {
  target_group { arn = aws_lb_target_group.legacy_ec2.arn  weight = 90 }
  target_group { arn = aws_lb_target_group.eks_ingress.arn weight = 10 }
}

# Route 53 weighted alternative
aws route53 change-resource-record-sets --hosted-zone-id Z123 --change-batch file://weights.json
kubectl rollout status deploy/monolith -n app
```

What to say to the interviewer: *I first assess where the monolith keeps state, then externalize config, sessions and files, send logs to stdout, add health endpoints and build a hardened image with a Helm chart including probes, resources and a PDB. The EKS version runs in parallel with the legacy system on the same database, and I shift traffic gradually with ALB weighted target groups or Route 53 weighted records, rolling back by weight if metrics degrade, then decommission the legacy system and peel off services over time.*

Q89. Have you faced production issues in Kubernetes? Tell me about one and how you troubleshot it.

Also asked as: Tell me about a recent production issue you handled in Kubernetes. What was the last Kubernetes outage you debugged, and what was the RCA?

In simple words: Tell a short story with a clear structure: what broke, how you noticed, how you found the cause, how you fixed it, and what you changed so it does not happen again (STAR: Situation, Task, Action, Result).

Detailed answer

Structure your answer: **symptom -> impact -> investigation steps -> root cause -> fix -> prevention**. Use a real example from your work and keep it to 2 minutes.

RCA structure interviewers like:

1. **Detection:** which alert fired (or did users report it first - then alerting is an action item).
2. **Timeline:** start, detection, mitigation, resolution times (MTTD, MTTR).
3. **Evidence:** metrics (error rate, latency, CPU/memory, restarts), logs (`kubectl logs --previous`, Loki queries), events (`kubectl get events`), traces, and the change log (deploys, node upgrades).
4. **Root cause** (5 whys - technical cause and why it was not caught) and **contributing factors**.
5. **Fix + prevention:** action items with owners (PDBs, probes, alerts, tests, runbooks).
6. **Blameless postmortem** document shared with the team, and follow-up on action items.

Sample answers you can adapt (from real EKS monitoring work):

Example 1 - logs missing for one service: A Loki/Grafana dashboard showed no application logs for one service while others were fine. I checked that the pods were Running and producing logs with `kubectl logs`, so the app was fine. Then I compared the dashboard's container label filter with the actual container name in the pod spec (`kubectl get pod -o jsonpath='{.spec.containers[*].name}'`) and found the real container name differed from what the dashboard query expected. Fix: corrected the label/query. Prevention: dashboards use variables populated from real label values instead of hard-coded names.

Example 2 - SLA breach after a node group update: A service availability dip lined up with a managed node group update. Pods were rescheduled as nodes were replaced, and one service did not have enough replicas/PDB protection, so it had a short outage. Fix: PDB with `maxUnavailable: 1`, at least 2 replicas spread across nodes, readiness probes; schedule node upgrades in low-traffic windows.

Example 3 - alerts silently not delivered: An alert pipeline looked healthy but notifications never reached Teams because a sidecar forwarder had silently died. Fix: moved to Alertmanager's native Teams receiver and added an end-to-end test alert (a "dead man's switch") so a broken alert path is itself detected.

Example:

```
kubectl get pods -n <ns> -o wide
kubectl logs <pod> -n <ns> --tail=50
kubectl get pod <pod> -n <ns> -o jsonpath='{.spec.containers[*].name}'
kubectl get events -n <ns> --sort-by=.lastTimestamp | tail
kubectl get pdb -n <ns>
```

> ***What to say to the interviewer:*** *Pick one real incident and tell it as symptom, impact, how it was detected, the specific commands and evidence from metrics, logs and events you used, the root cause, the fix and the prevention you added, and mention the blameless postmortem with owned action items. Interviewers care most that you debugged systematically, communicated, and made a lasting improvement like a PDB, better probes or an end-to-end alert test.*